



视频播放器配置错误

错误 153

在 YouTube 上观看此视频

[了解详情](#)

Introduction



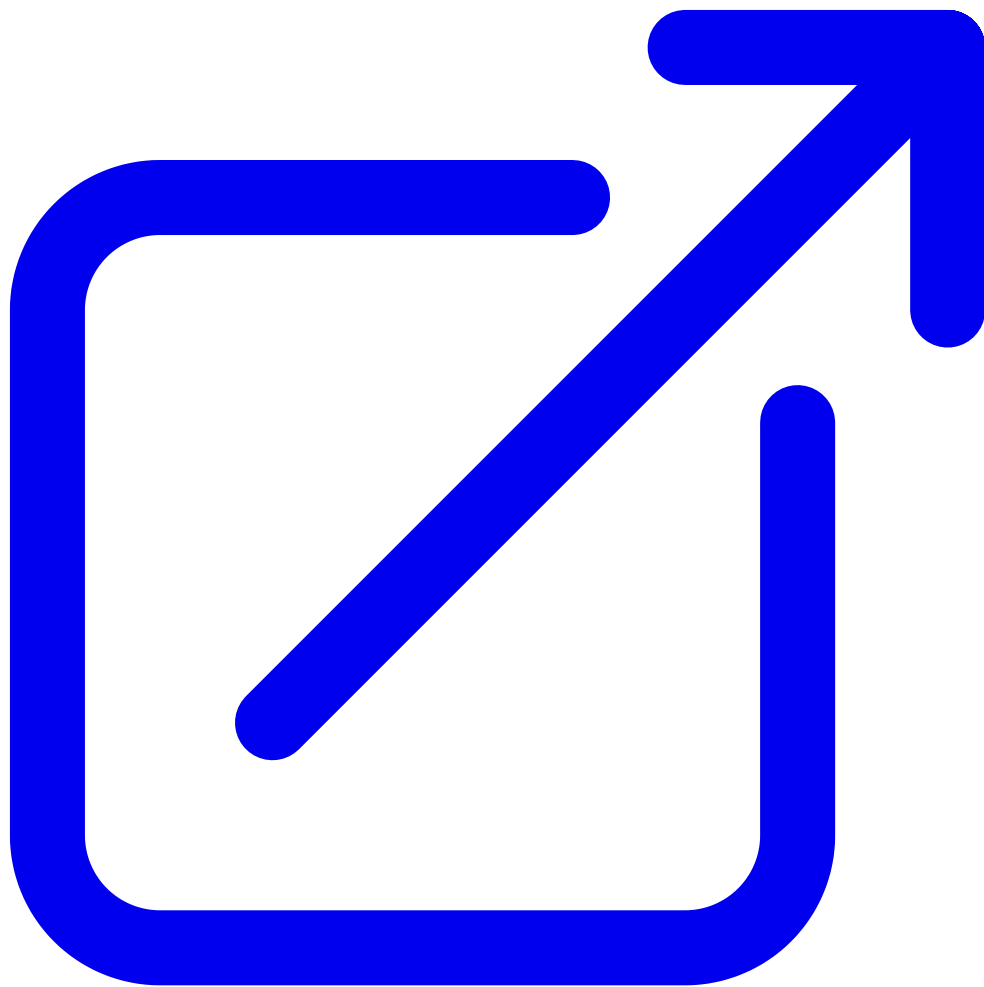
Introduction

Welcome to the 3rd edition of *Composing Programs*, a free online introduction to programming and computer science.

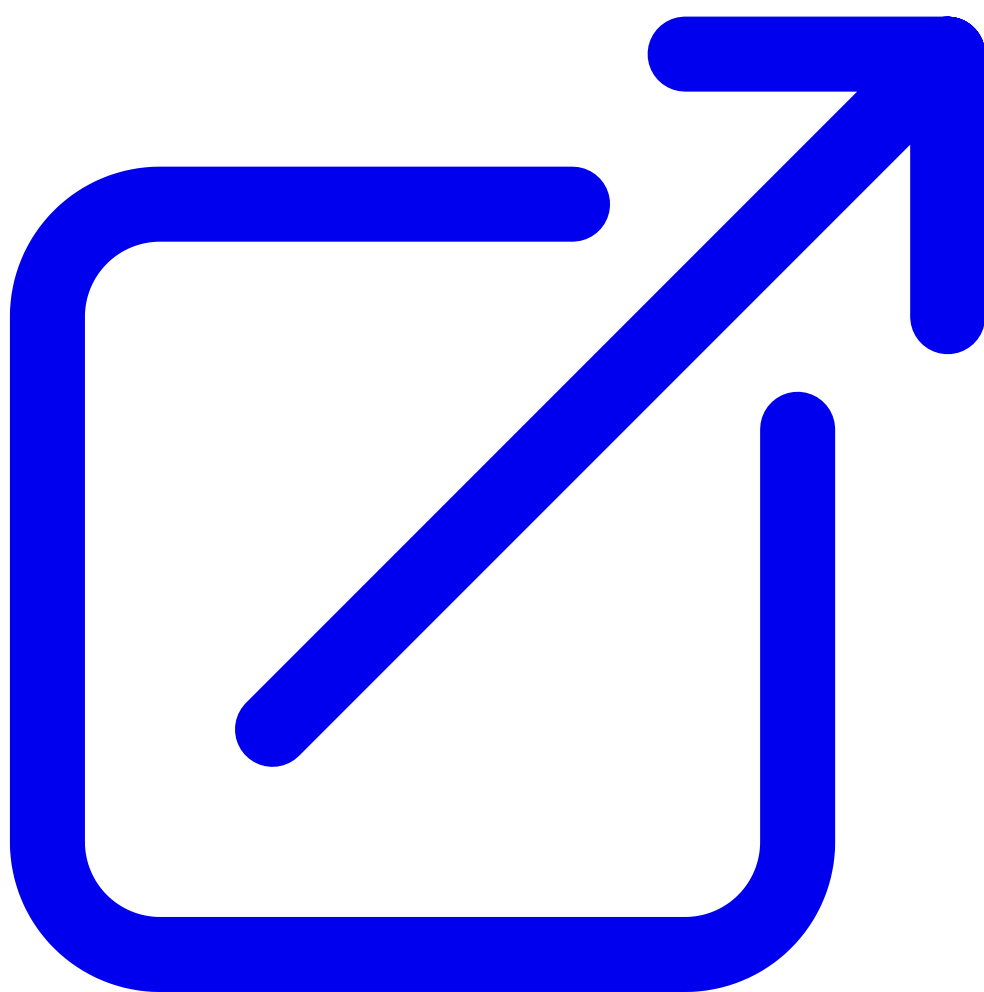
- [Composing Programs](#)
- Chapter 1: Functions
 - [1.1 Getting Started](#)
 - [1.2 Elements of Programming](#)
 - [1.3 Defining New Functions](#)
 - [1.4 Designing Functions](#)
 - [1.5 Control](#)
 - [1.6 Higher-Order Functions](#)
 - [1.7 Recursive Functions](#)
 - [Functions Reference](#)
- Chapter 2: Data
 - [2.1 Introduction](#)
 - [2.2 Sequences](#)
 - [2.3 Compound Data](#)
 - [2.4 Mutable Data](#)
 - [2.5 Object-Oriented Programming](#)
 - [2.6 Implementing Classes and Objects](#)
 - [2.7 Object Abstraction](#)
- [Appendix: Examples](#)

Additional sections will be added this semester.

The [first](#)

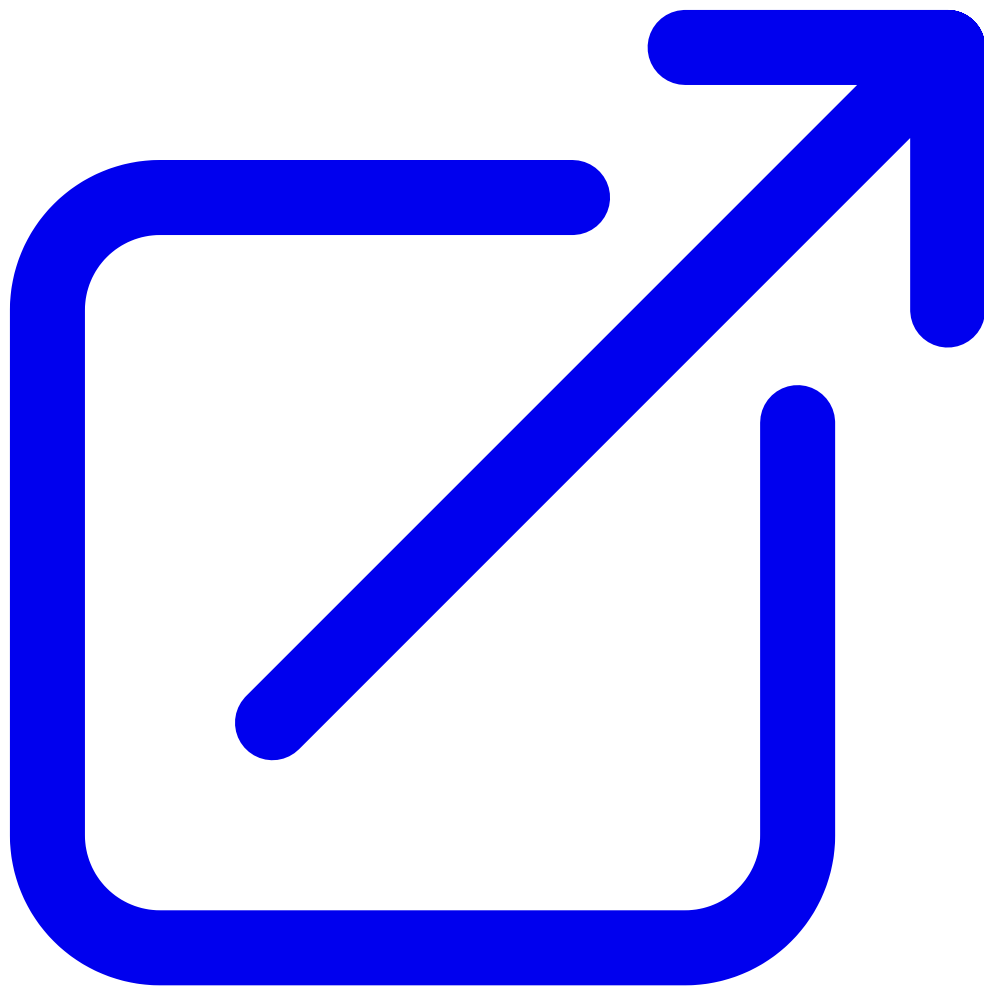


and [second](#)

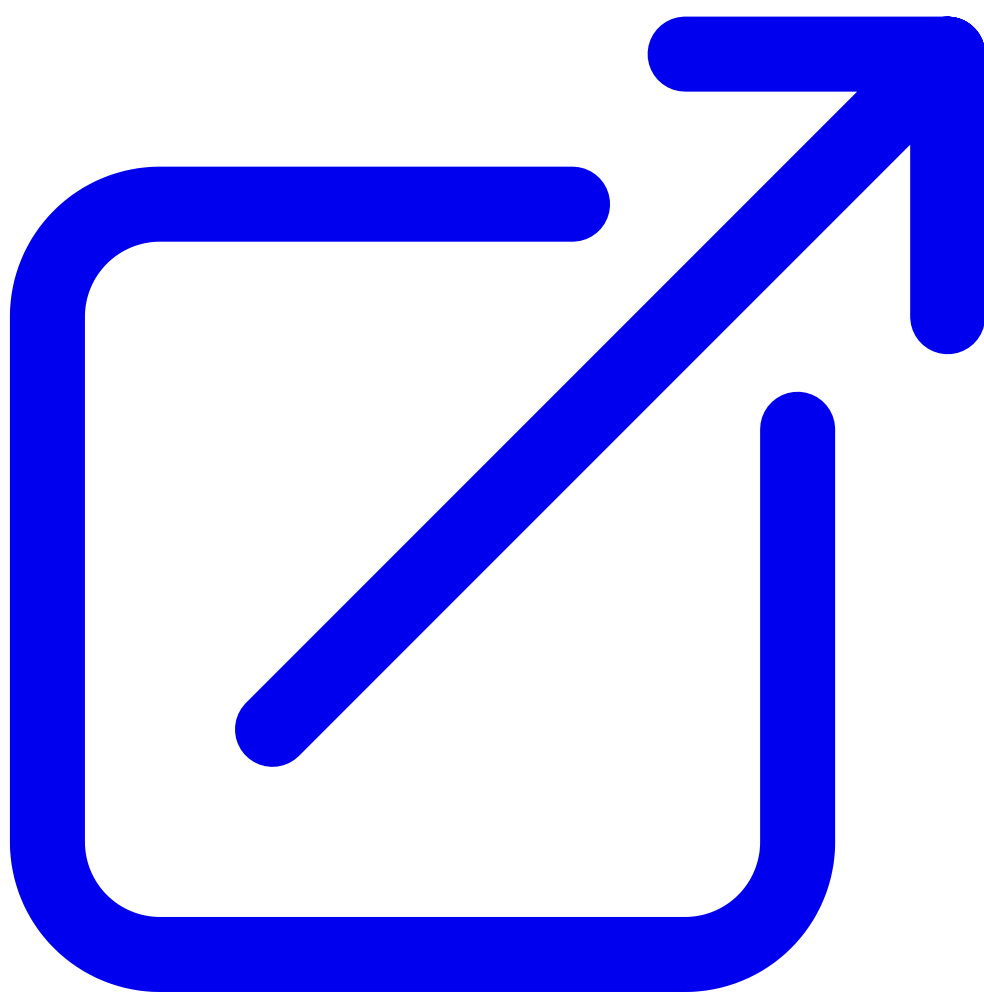


editions are still available.

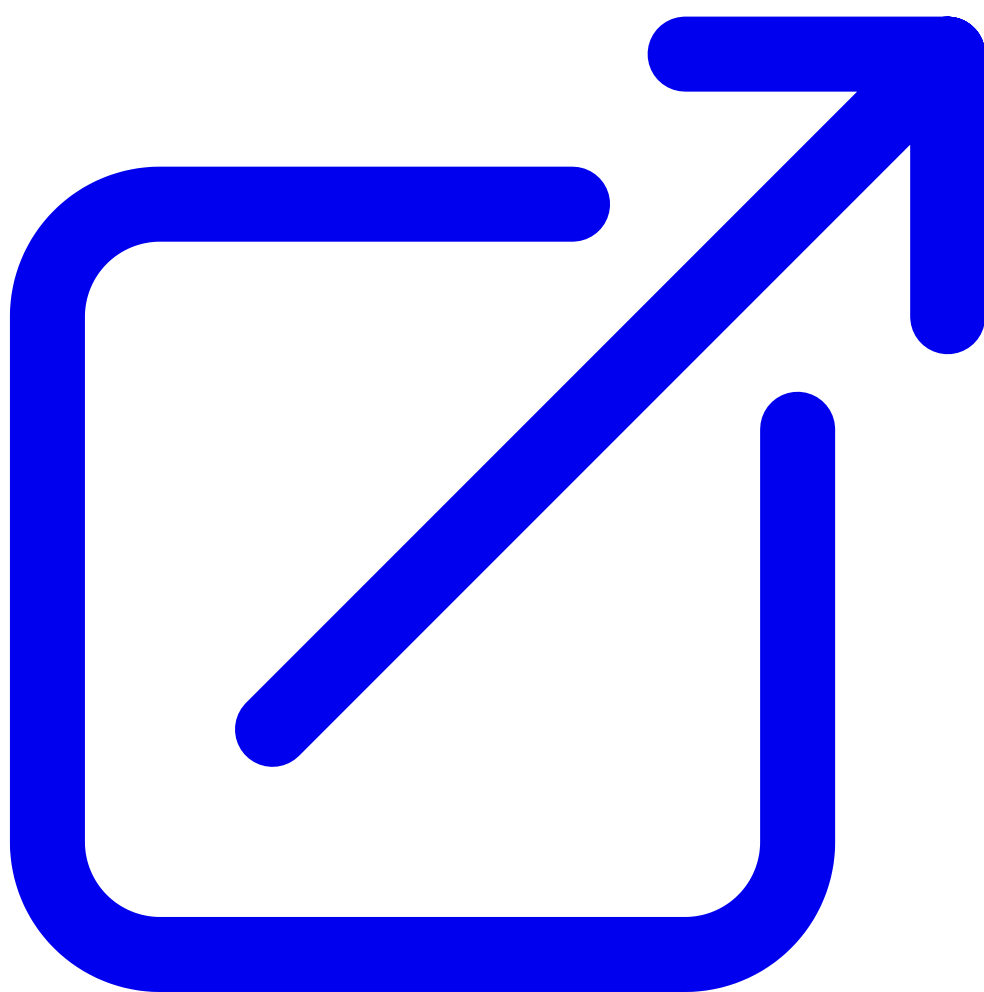
In the tradition of [SICP](#)



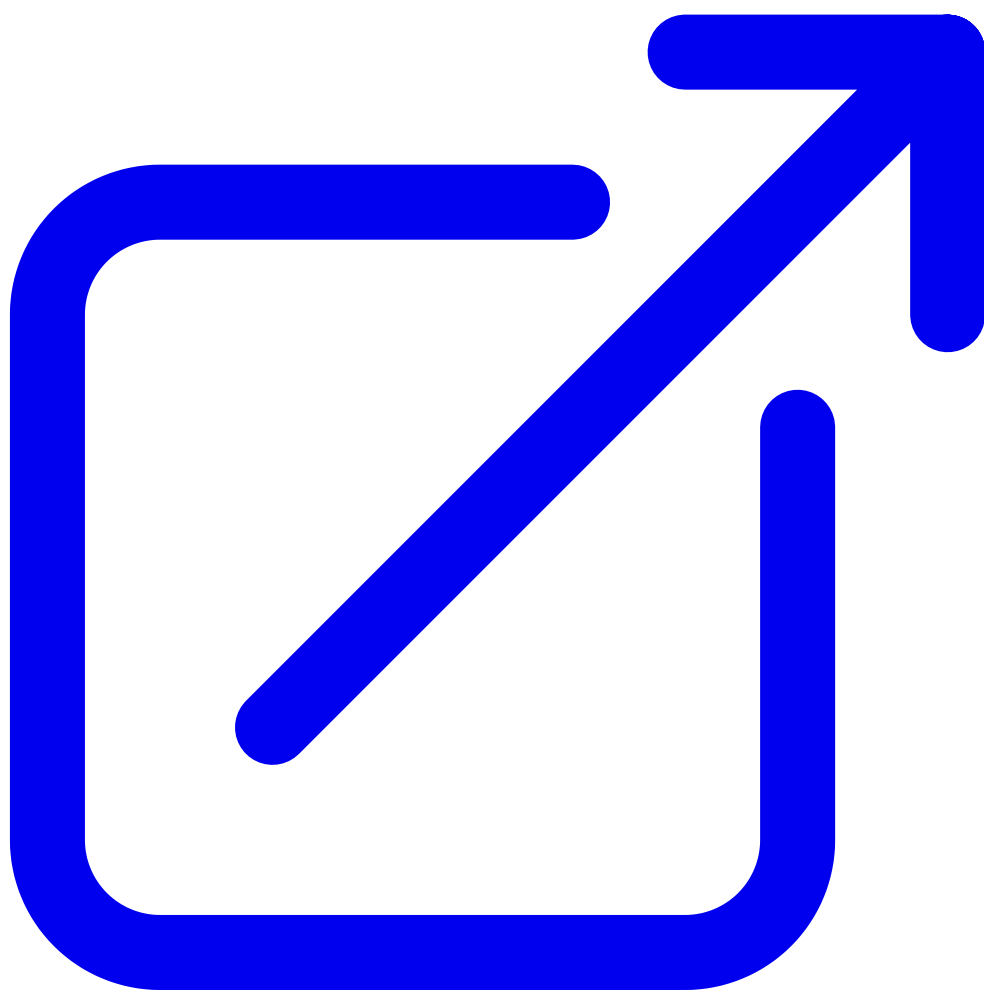
, this text introduces abstraction, programming paradigms, problem solving, and techniques for managing the complexity of large programs. These concepts are illustrated primarily using the [Python](#)



programming language. Some later sections use [SQL](#)

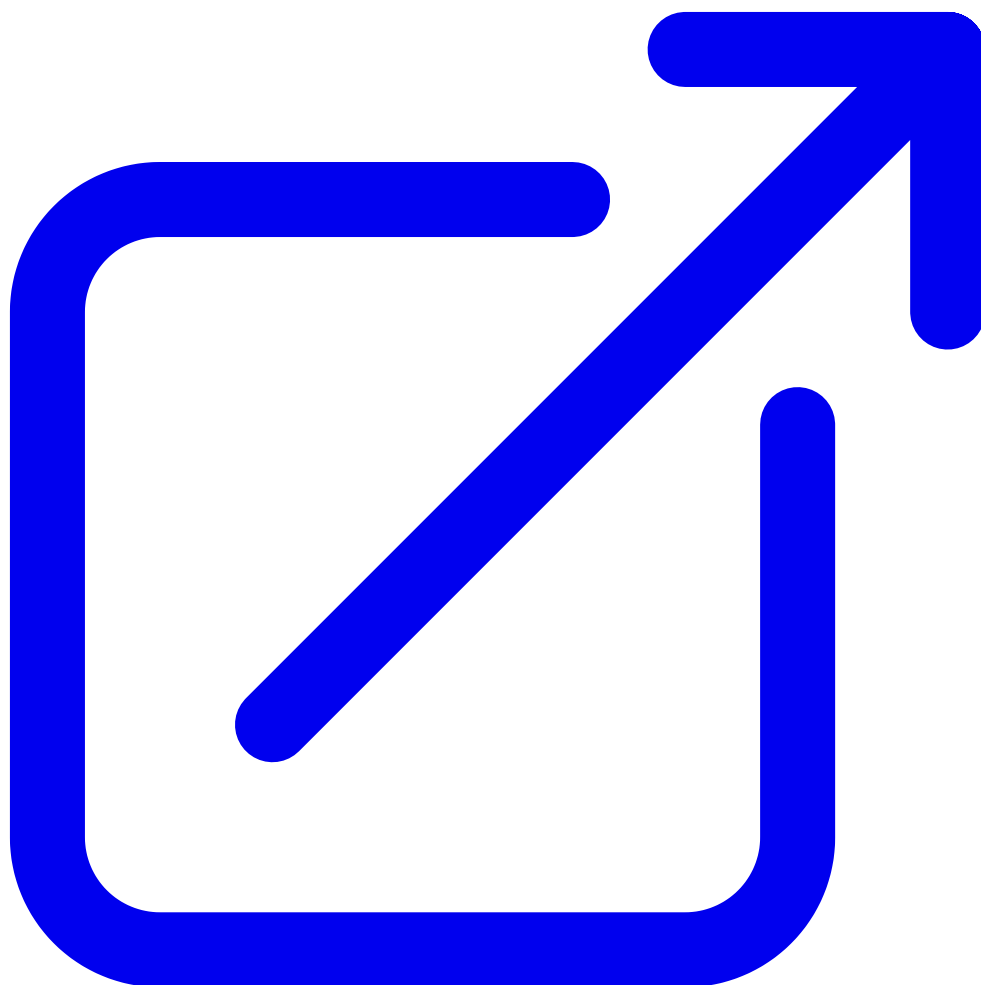


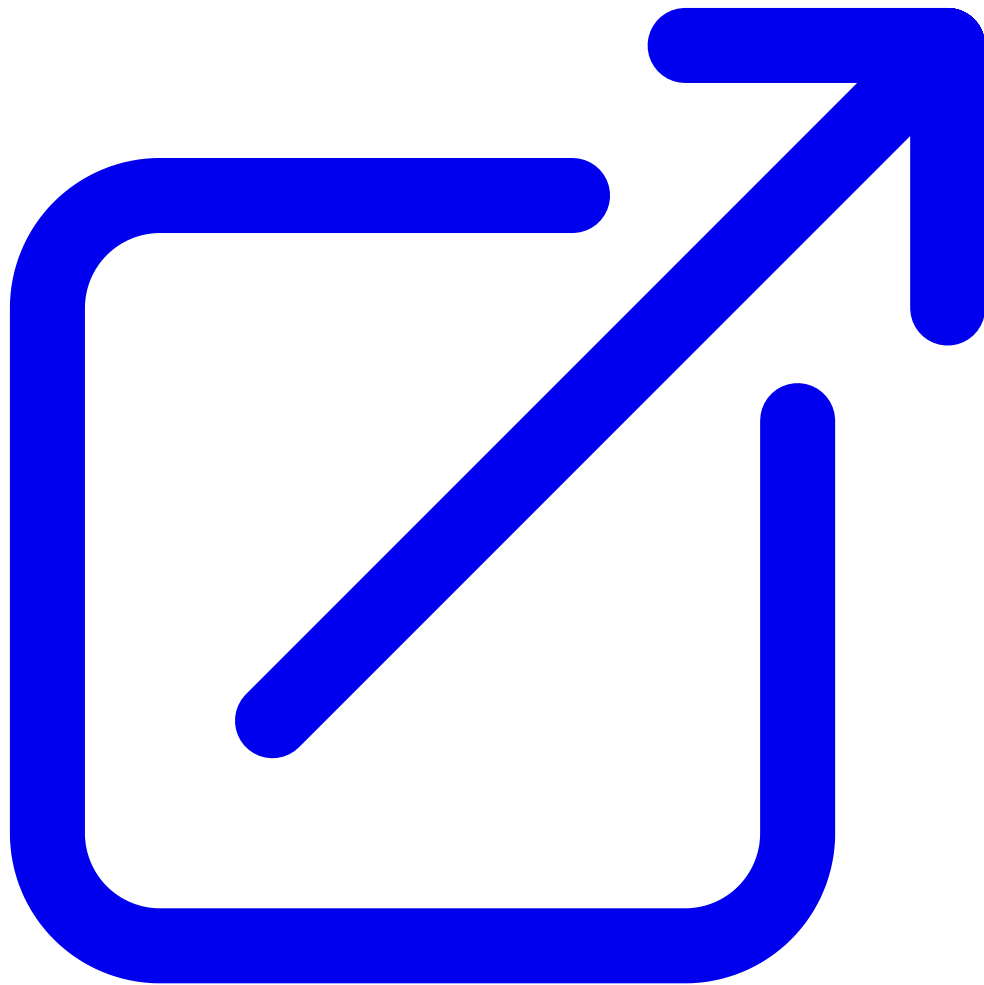
and [Gleam](#)



This third edition focuses on how to use artificial intelligence (AI) tools to write, extend, and understand computer programs. The way to build this ability is first to understand how programming languages represent information, express problem-solving logic, and manage complexity, and only then learn how to use AI tools. Therefore, most of the book is not about AI or its use, but instead about programming foundations.

The text features interactive **environment diagrams** generated by the [Online Python Tutor](#)





This online book is a derivative of Structure and Interpretation of Computer Programs (2nd ed.) by Harold Abelson, Gerald Jay Sussman, and Julie Sussman. Composing Programs was originally authored in 2011 under the CC BY-NC-SA 3.0 Unported license applied to SICP by MIT Press at that time. (SICP was later relicensed under CC BY-SA 4.0 in 2016). As permitted under Section 4(b) of the original 3.0 license, this adaptation has been upgraded to CC BY-NC-SA 4.0, but remains non-commercial.

Copyright John DeNero.

[Chapter 1: Functions](#)

[1.1 Getting Started](#)



1.1 Getting Started

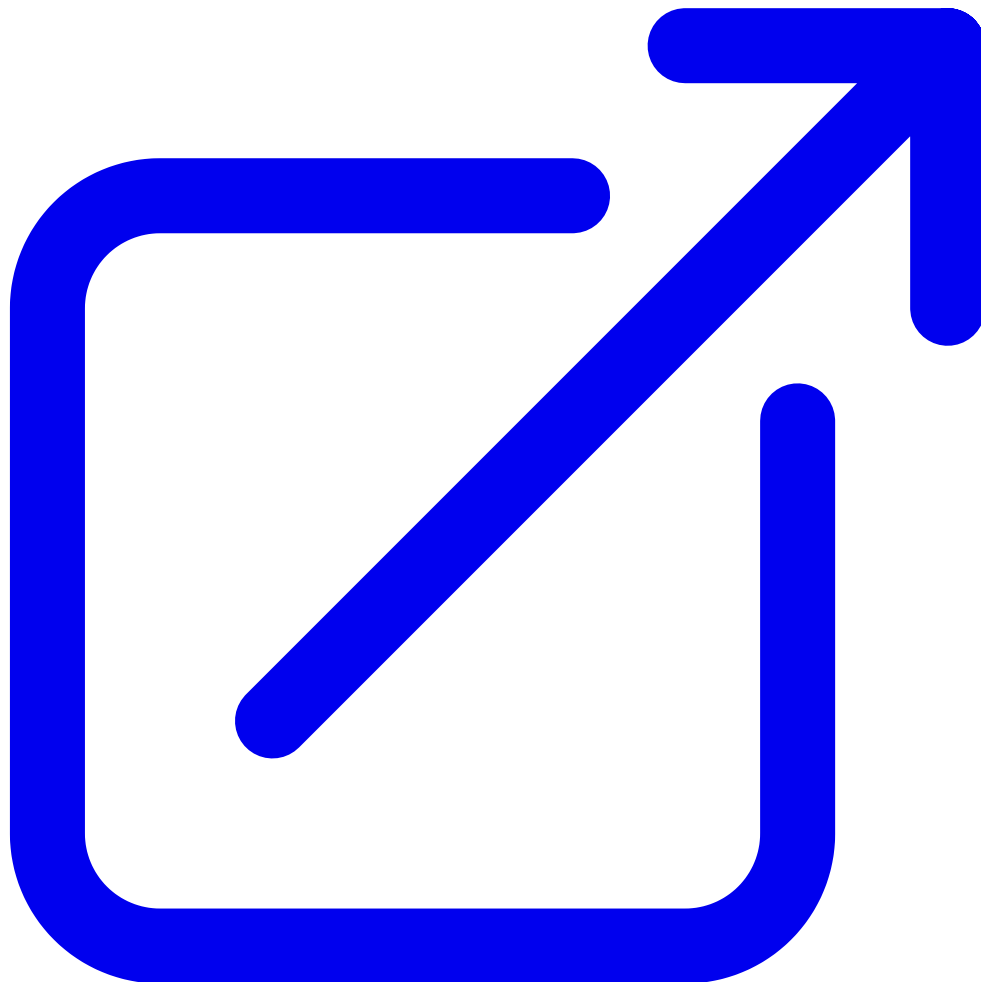


1.1 Getting Started

Computer science is a broad academic discipline. The areas of globally distributed systems, theory, robotics, graphics, security, computer architecture, and dozens of emerging sub-fields all expand with new techniques and discoveries every year. The rapid progress of computer science has left few aspects of human life unaffected, and advances in artificial intelligence are increasing the rate of change. Commerce, communication, science, art, leisure, and politics have all become computational domains.

The vast impact of computer science is due in large part to an elegant and powerful set of fundamental ideas: all computing begins with representing information, specifying problem-solving logic to process it, and creating abstractions that manage the complexity of that logic. A programming language, which is one of humanity's most clever and flexible inventions, is the medium by which these ideas are expressed. Understanding how computer programs are written and organized, as well as how computers run programs to carry out computational processes, provides the background needed to create new programs and expand the impact of computer science even further.

These fundamental ideas have long been taught using the classic textbook *Structure and Interpretation of Computer Programs* ([SICP](#))



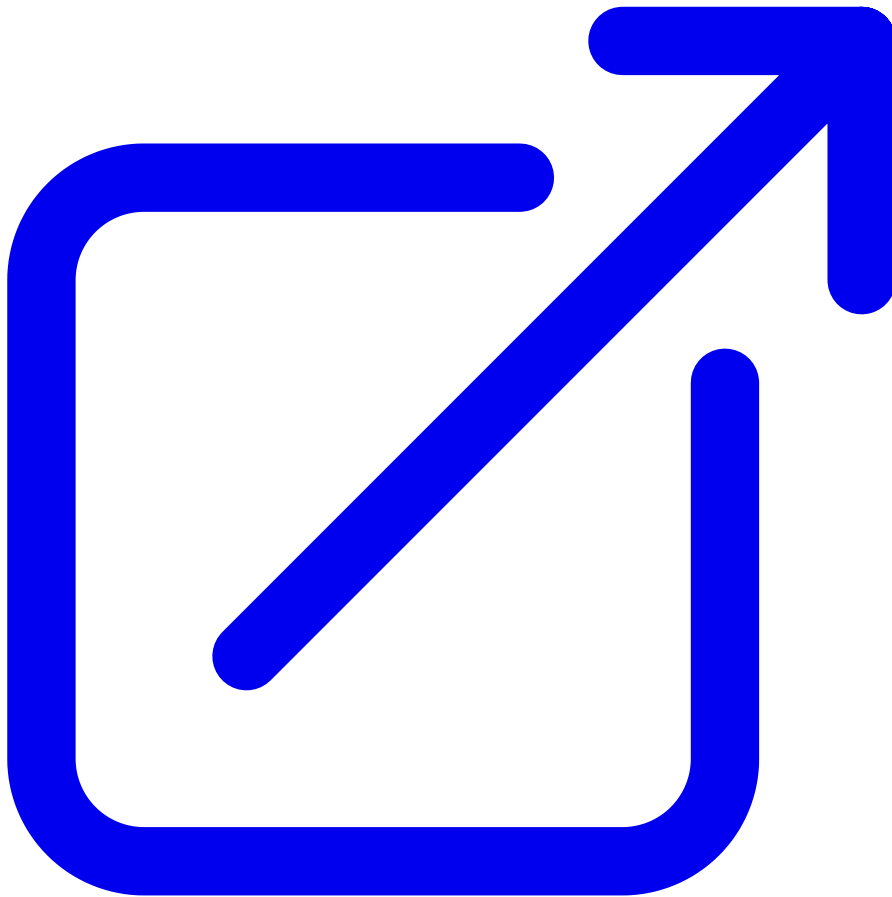
) by Harold Abelson and Gerald Jay Sussman with Julie Sussman, first published in 1984. This text borrows heavily from that textbook, which the original authors have kindly licensed for adaptation and reuse.

This third edition of Composing Programs focuses on how to use artificial intelligence (AI) tools to write, extend, and understand computer programs. It requires no human skill at all to tell AI to write programs that have already been fully and precisely described. However, creating novel and useful programs requires someone to invent and describe them, which currently often requires the same thorough understanding of representation, logic, abstraction, and programming languages that are needed to write programs from scratch. Creating programs can be vastly more efficient now, since AI can generate or edit a large program based on only a short description, but the human expertise required to envisage a program's design, describe it accurately, and improve upon the output of AI tools remains central to the process of creating programs. The right time to learn about effectively using AI to develop programs is just after developing that core understanding, which is why the topic comes at the end of this book.

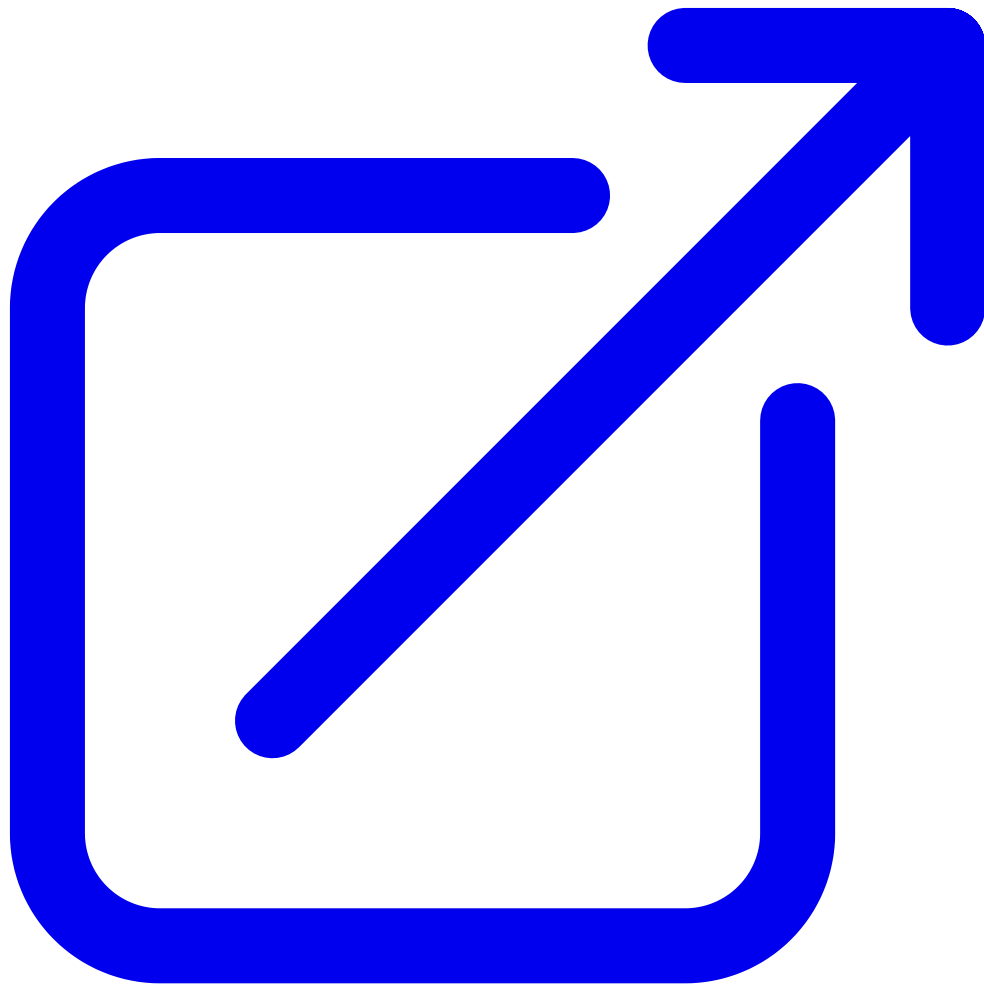
Programming in PythonII

A language isn't something you learn so much as something you join.

[Arika Okrent](#)



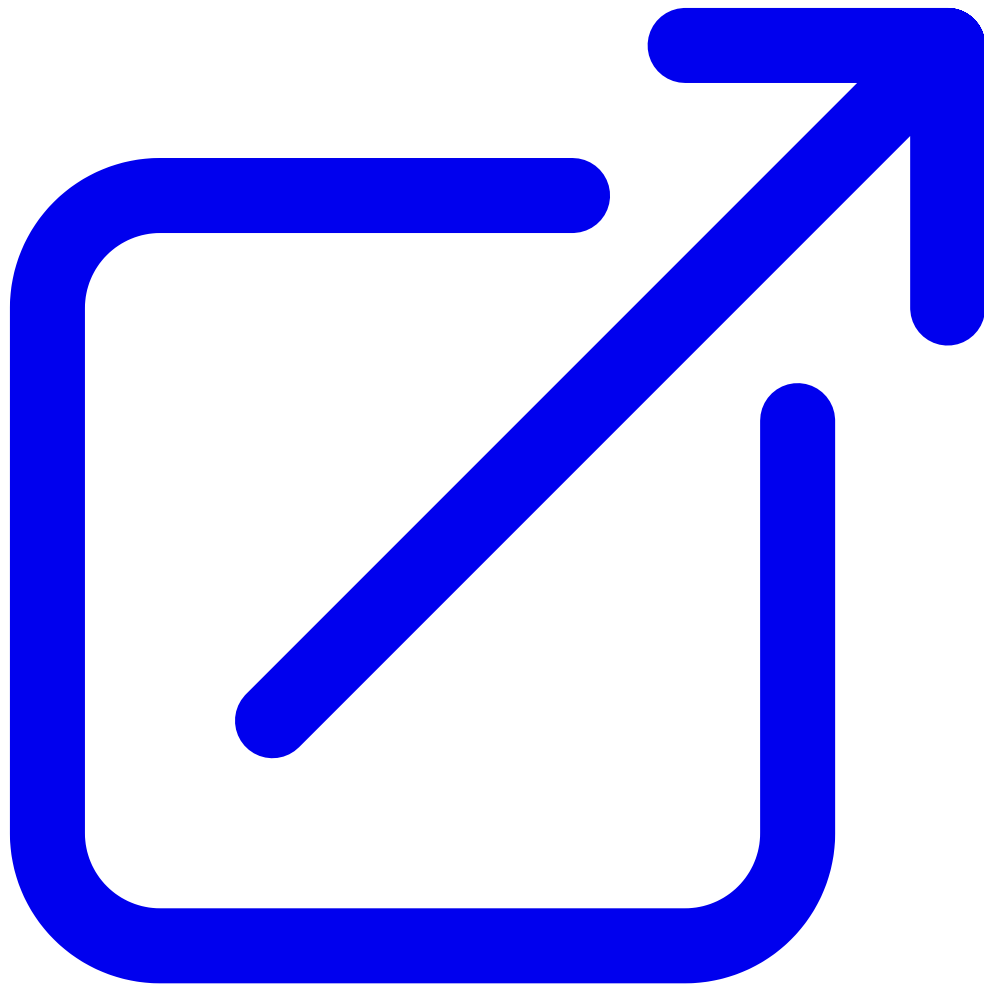
In this text, we will work primarily with the [Python](#)



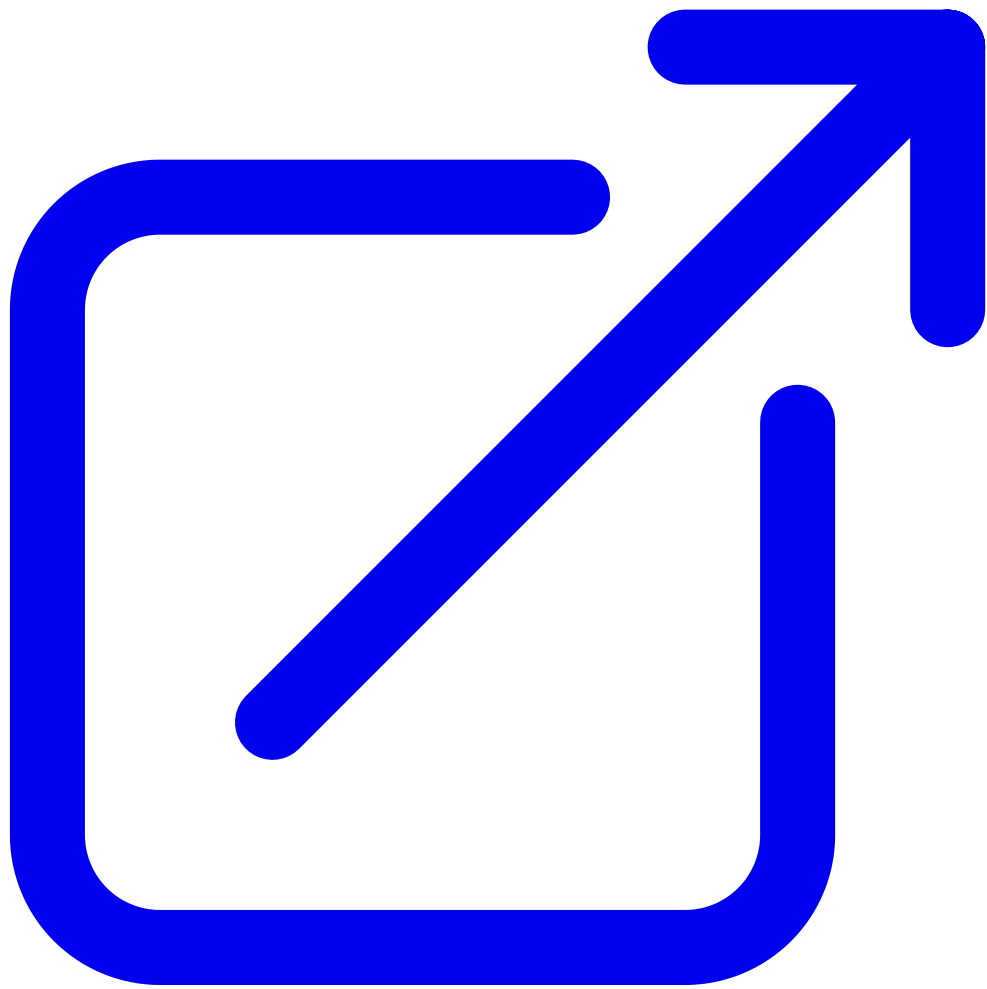
language.

Python is a widely used programming language that has recruited enthusiasts from many professions: web programmers, game engineers, scientists, academics, and even designers of new programming languages. When you learn Python, you join a million-person-strong community of developers. Developer communities are tremendously important institutions: members help each other solve problems, share their projects and experiences, and collectively develop software and tools. Dedicated members often achieve celebrity and widespread esteem for their contributions.

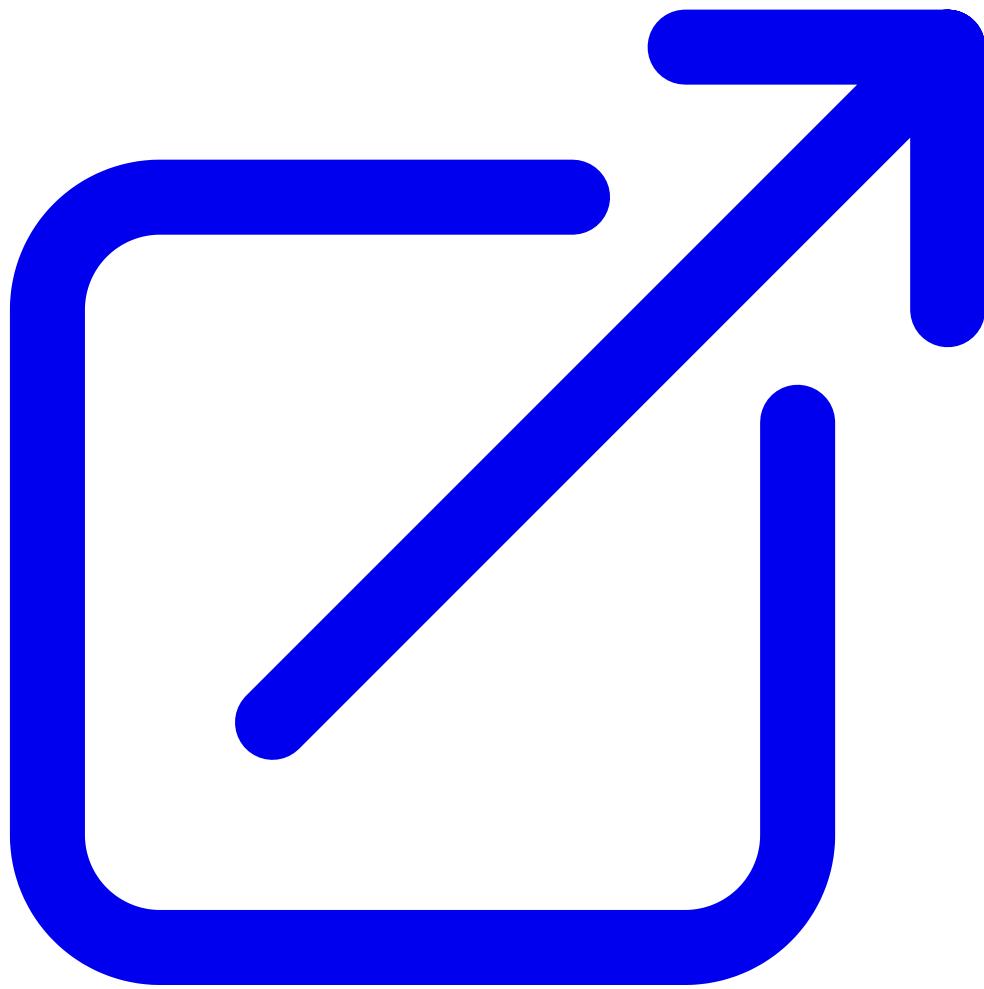
The Python language itself is the product of a [large volunteer community](#).



that prides itself on the [diversity](#).

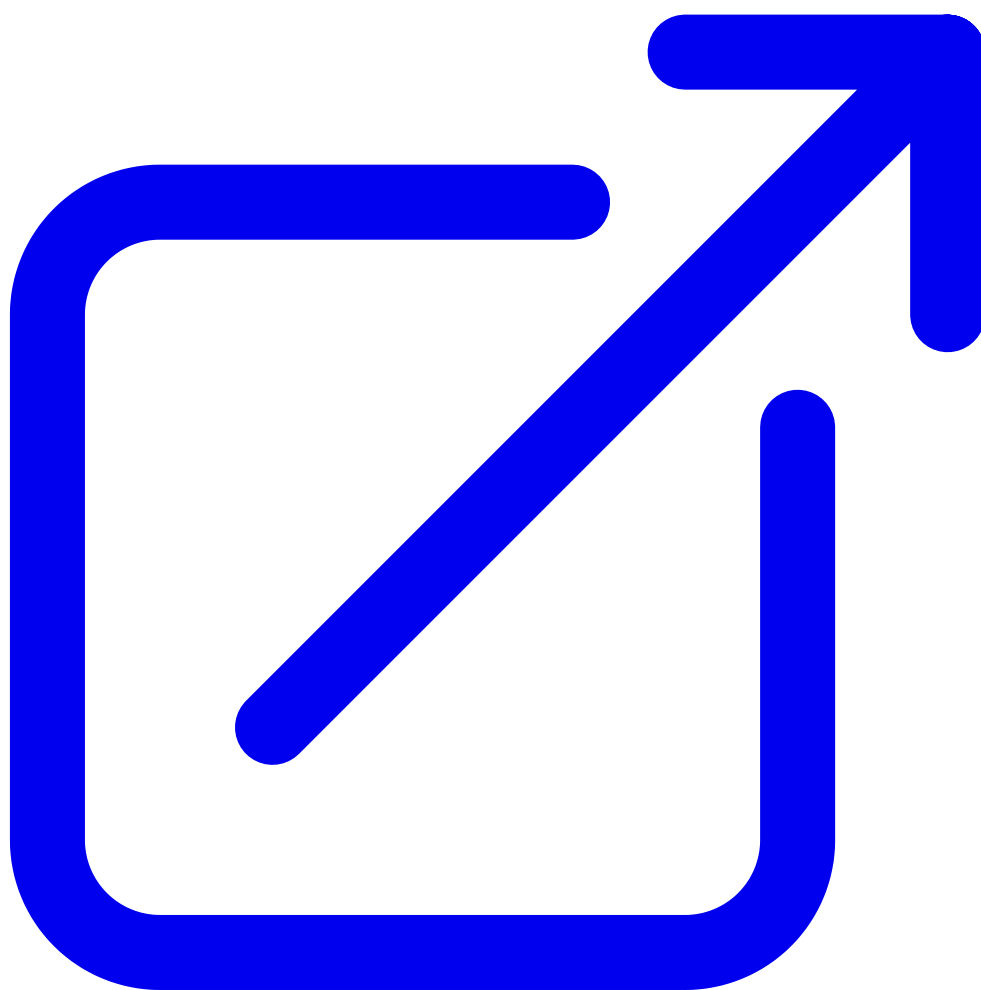


of its contributors. The language was conceived and first implemented by [Guido van Rossum](#) in the late 1980's. The first chapter of his [Python 3 Tutorial](#)



explains why Python is so popular, among the many languages available today.

Python excels as an instructional language because, throughout its history, Python's developers have emphasized the human interpretability of Python code, reinforced by the [Zen of Python](#)



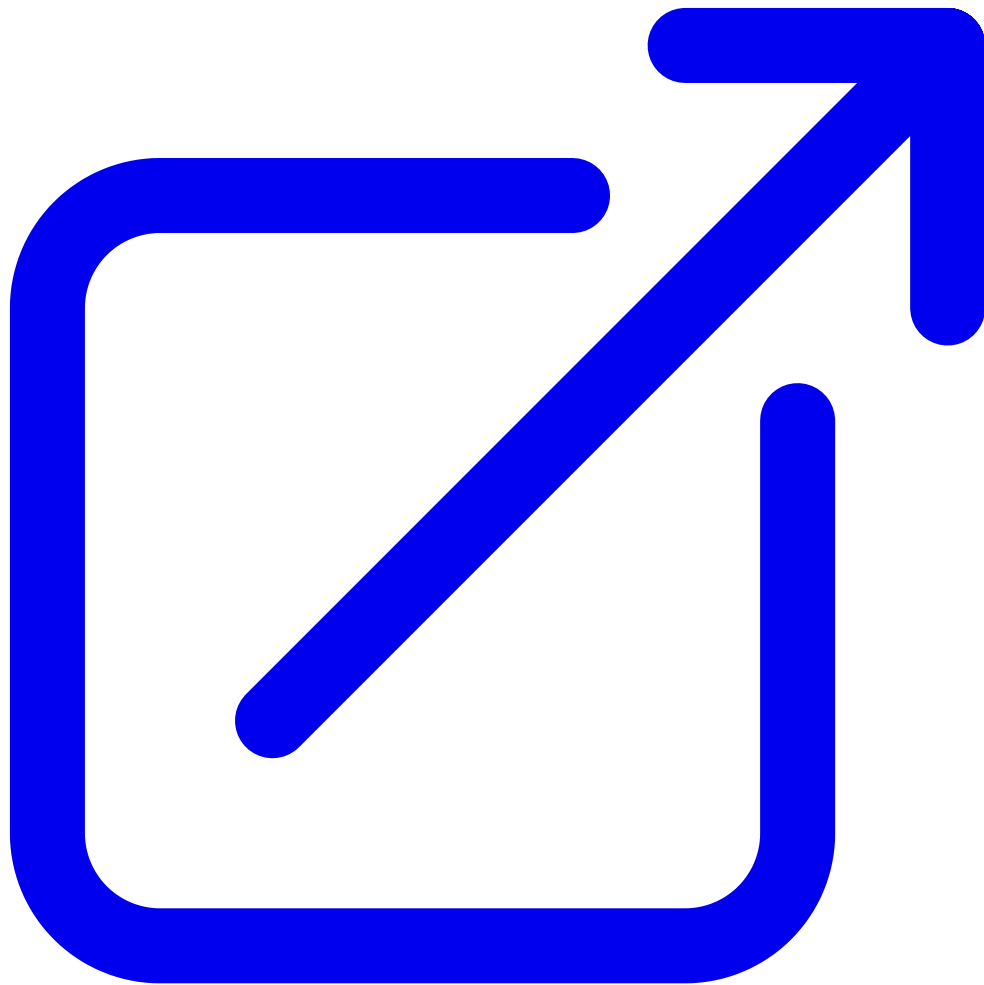
guiding principles of beauty, simplicity, and readability. Python is particularly appropriate for this text because its broad set of features support a variety of different programming styles, which we will explore.

The best way to get started programming in Python is to interact with the interpreter directly. This section describes how to install Python 3 and the Visual Studio Code editor, then initiate an interactive session with the interpreter and start programming. All of this software is free and used regularly by millions of programmers.

Installing Python 3¹

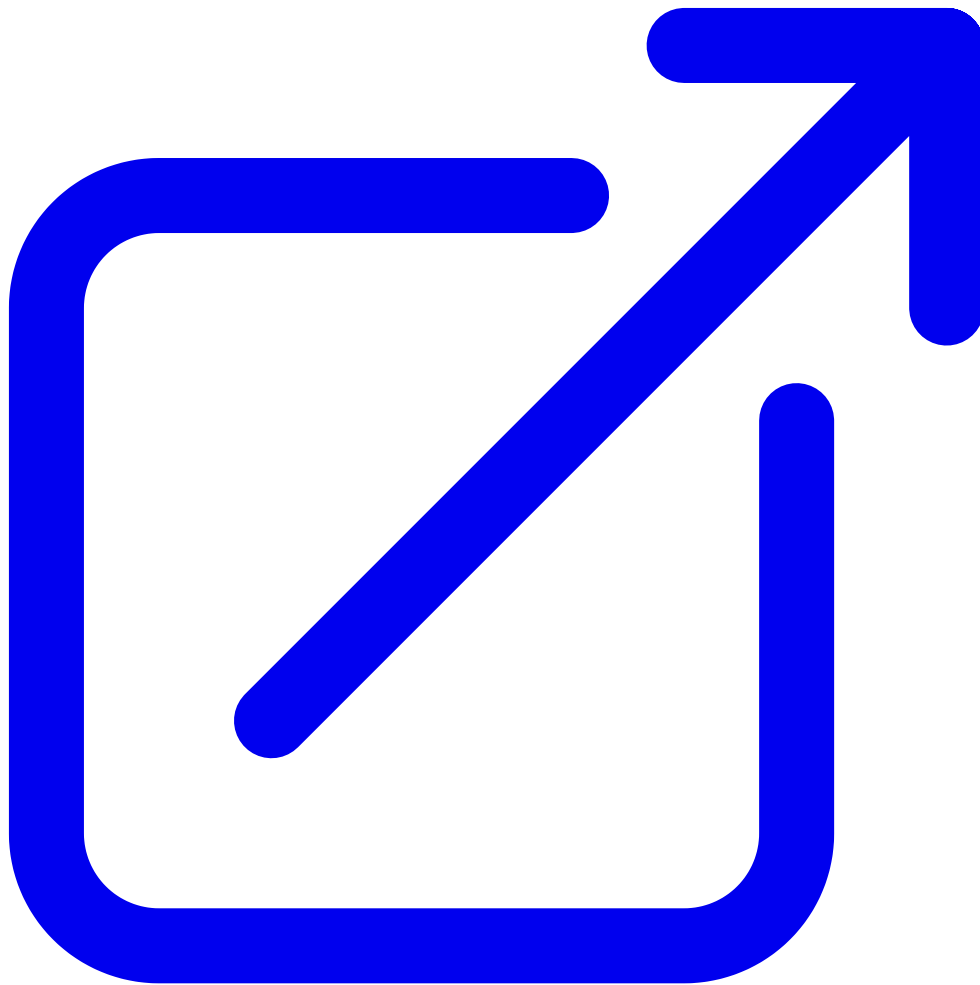
Install Python using the official installer for your operating system. Always choose a version of Python labeled **stable** that begins with a 3, such as 3.14.7. The larger the version number, the better.

Windows. Download the *Windows installer (64-bit)* from the [Windows downloads page](#)



and run it. On the first screen of the installer, check the box labeled **Add python.exe to PATH** before clicking *Install Now*.

Mac. Download the *macOS 64-bit universal2 installer* from the [macOS downloads page](#)

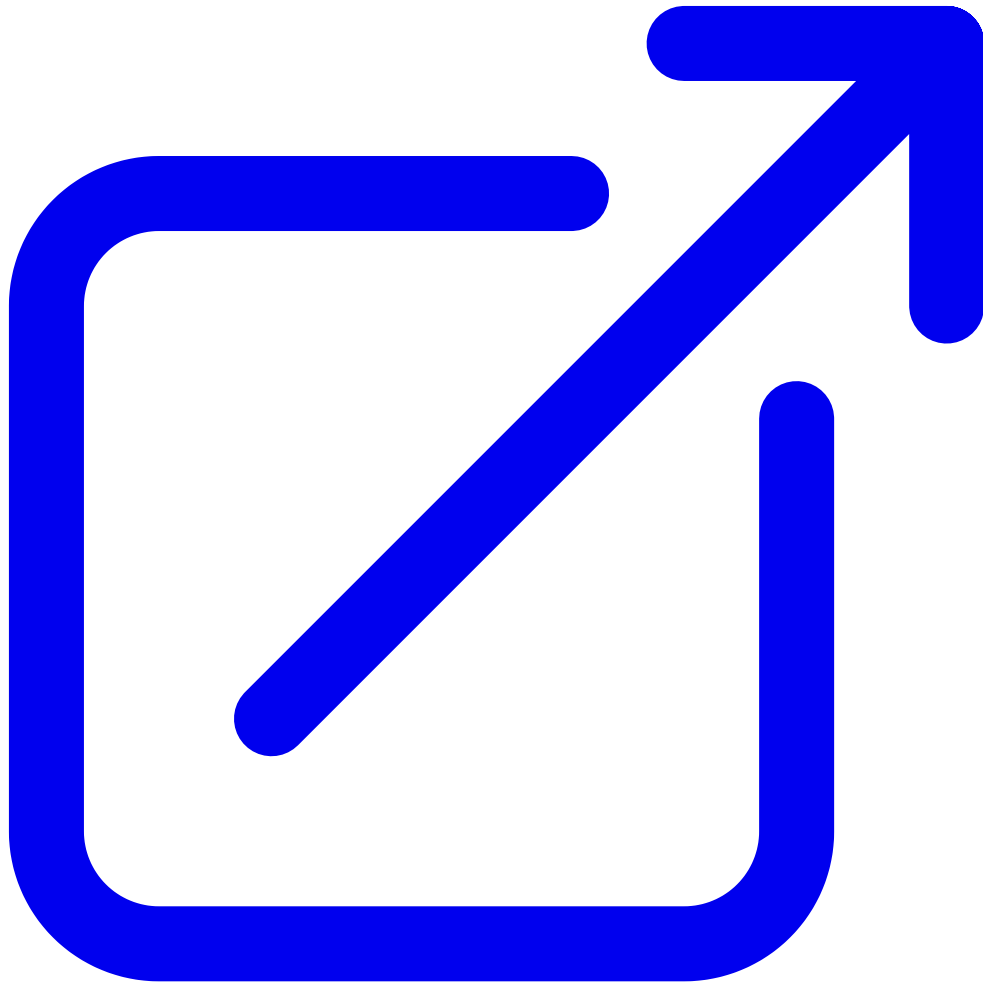


, open the downloaded `.pkg` file, and follow the instructions of the installer.

Installing and Configuring Visual Studio Code¹

Editors are applications used to write and edit programs. Visual Studio Code, commonly called VS Code, is the most popular editor globally.

Install Visual Studio Code using the official installer for your operating system, which is available from the [Visual Studio Code download page](#)

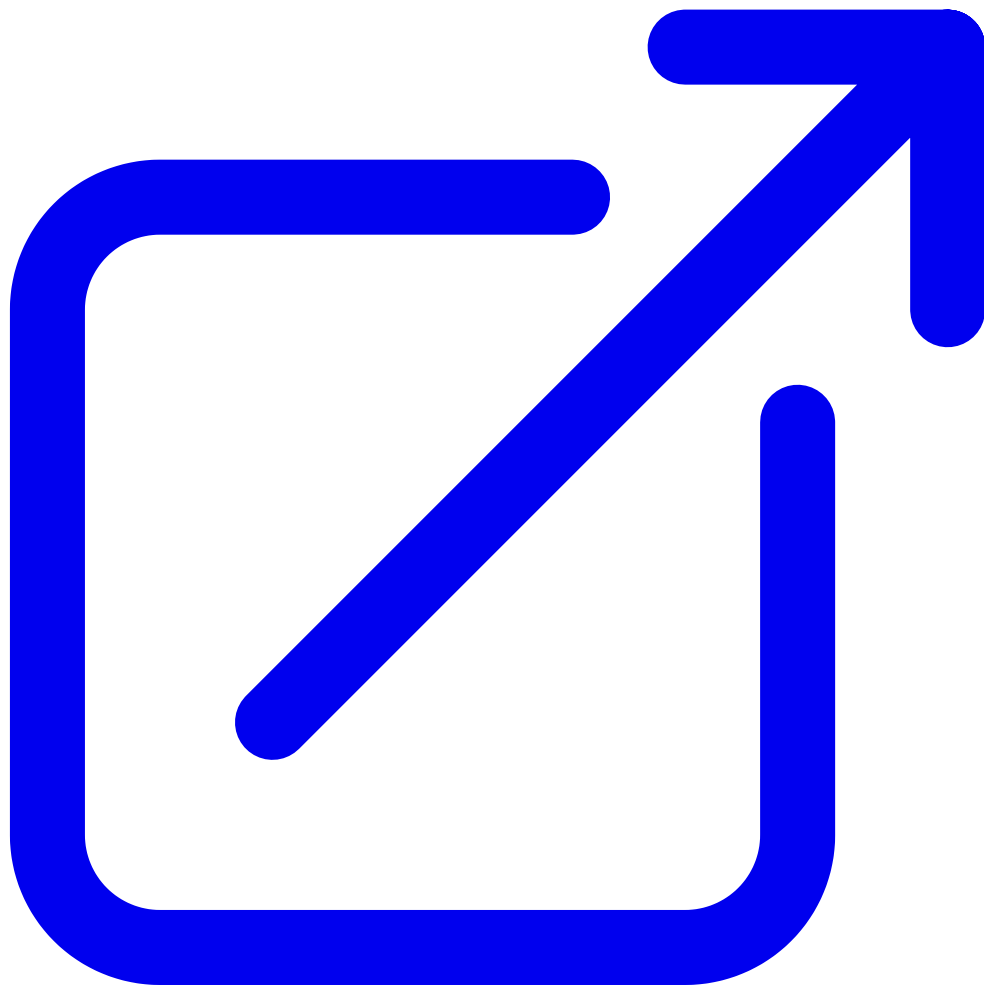


Windows. Download the *Windows x64 User Installer* and run it.

Mac. Download the *Mac Universal* archive, open the downloaded file, and drag the *Visual Studio Code* application into your *Applications* folder.

When you first open VS Code and are asked to log in to use GitHub Copilot AI features or anything else, you can just close the login window. There's no need to log in to VS Code.

There are two more important steps once VS Code is installed and open. If you open any of the companion assignments to this edition of the textbook, such as a lab or homework or project from [cs61a.org](#)

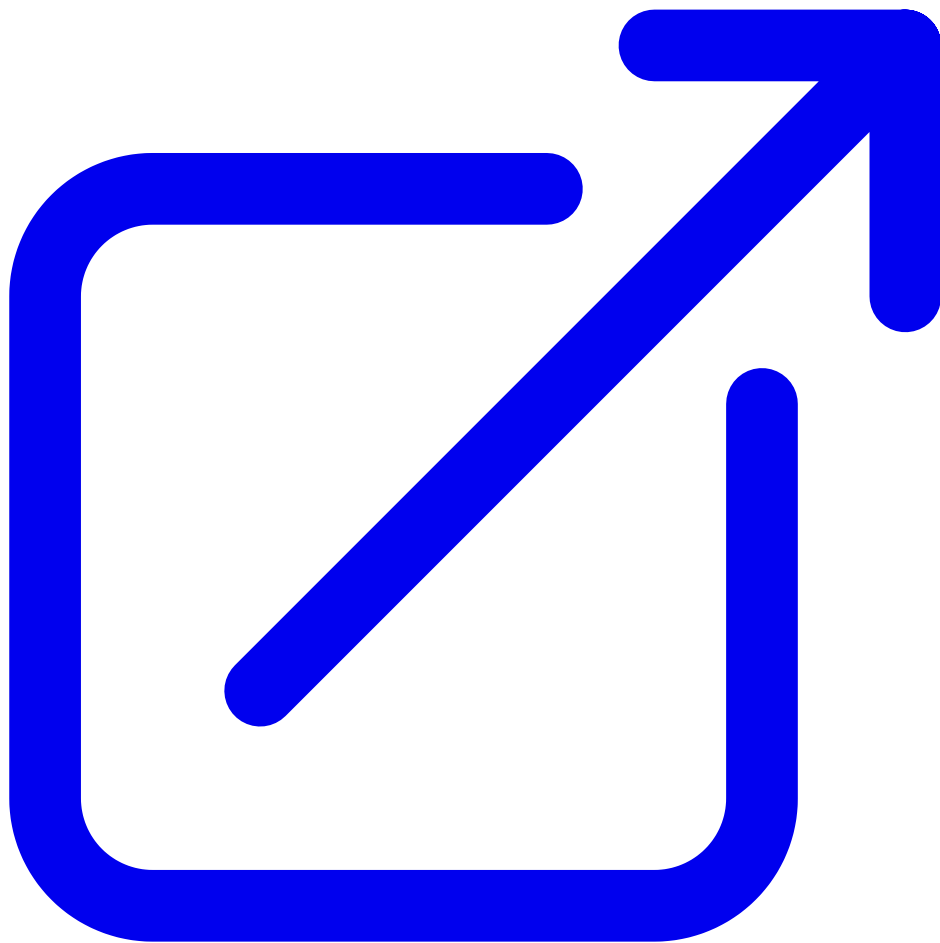


, then these steps should happen automatically. If instead you just want to write your own programs, then follow these two steps:

1. Add support for Python by clicking the *Extensions* icon

 Extensions icon

in the left sidebar, searching for `python` , and installing the extension called [Python](#)



published by Microsoft. It should be the top option and have a description that starts with, "Python language support."

2. Turn off GitHub Copilot by searching for `copilot` and clicking on the pre-installed "GitHub Copilot Chat" extension. Near the top of the page that opens describing the extension, click "Disable AI Features" and then close the extension page.

Interactive Sessions¹

In an interactive Python session, you type some Python *code* after the *prompt*, `>>>`. The Python *interpreter* reads and executes what you type, carrying out your various commands.

To start an interactive session within VS Code, first select `New Terminal` from the `Terminal` menu, then start the Python interpreter by typing either `py` (Windows) or `python3` (Mac/Linux).

If you see the Python prompt, `>>>`, then you have successfully started an interactive session. Try typing `2 + 2` and pressing enter (Windows) or return (Mac/Linux). You should see the answer 4.

```
>>> 2 + 2
4
```

Within VS Code, the terminal running the Python interpreter should look something like this:



Tip. Up and down arrows cycle through the history of Python code you've written.

Errors¹

Python awaits your commands. You are encouraged to experiment with the language. In the rest of this chapter, type in the expressions you read about and change them around to see what happens. However, be prepared for errors. While programming languages are powerful tools, they are also extremely rigid. Even the smallest spelling and formatting changes will cause unexpected output and errors.

Learning to interpret errors and diagnose the cause of unexpected errors is called *debugging*. Some guiding principles of debugging are:

1. **Test incrementally:** Every well-written program is composed of small, modular components that can be tried individually. Instead of starting with a long and complicated expression, first try evaluating its parts to make sure you get the result you expect. This approach will help you identify problems early and gain confidence in your components.
2. **Isolate errors:** An error in the output of a statement can typically be attributed to a particular place in the code. When trying to diagnose a problem, trace the error to the smallest fragment of code you can before trying to correct it.
3. **Check your assumptions:** Interpreters do carry out your instructions to the letter --- no more and no less. Their output is unexpected when the behavior of some code does not match what the programmer believes (or assumes) that behavior to be. Know your assumptions, then focus your debugging effort on verifying that your assumptions actually hold.
4. **Consult others:** You are not alone! If you don't understand an error message, ask a friend, instructor, search engine, or AI tool. A lot of valuable programming knowledge is shared in the process of discussing programs.

Incremental testing, modular design, precise assumptions, and discussion are themes that persist throughout this text. Hopefully, they will also persist throughout your computer science career.



[Composing Programs](#)

[Composing Programs](#)

[Chapter 1: Functions](#)

[1.2 Elements of Programming](#)



1.2 Elements of Programming



1.2 Elements of Programming

A programming language is more than just a means for instructing a computer to perform tasks. The language also provides us with ways to organize our descriptions of computational processes. Programs serve to communicate those ideas among the members of a programming community. The preface to the first edition of the original SICP book emphasized this point: "Programs must be written for people to read, and only incidentally for machines to execute."

When we describe a language, we should pay particular attention to the means that the language provides for composing simple descriptions into more complex ones. Every powerful language has three such mechanisms:

- **primitive expressions and statements**, which represent the simplest building blocks that the language provides,
- **means of combination**, by which compound elements are built from simpler ones, and
- **means of abstraction**, by which compound elements can be named and manipulated as units.

Programming deals primarily with two kinds of things: functions and data. (Soon we will discover that they are really not so distinct.) Informally, data is information that we want to manipulate, and functions describe the steps in manipulating the data. Thus, any powerful programming language should be able to describe data and functions, as well as have some methods for combining and abstracting both functions and data.

Expressions

We'll start simple and build up to more interesting examples soon.

A primitive expression doesn't have any other expressions within it. One kind of primitive expression is a number. More precisely, the expression that you type consists of the digits that represent the number.

```
>>> 42
42
```

Expressions representing numbers may be combined with mathematical operators to form a compound expression, which the interpreter will evaluate:

```
>>> -1 - -1
0
>>> 1/2 + 1/4 + 1/8 + 1/16 + 1/32 + 1/64 + 1/128
0.9921875
```

Python includes many ways to form compound expressions. Rather than attempt to enumerate them all immediately, we will introduce new expression forms as we go, along with the language features that they support.

Call Expressions

The most important kind of compound expression is a *call expression*, which calls a function on some arguments. In mathematics, a function maps from input values to an output value. Similarly, in Python a function is applied to input values that are called its *arguments*, performs some computation, and finally *returns* an output value, often called its *return value*. For example, the `max` function maps its inputs to a single output, which is the largest of the inputs. A call expression has the same form as a function call in mathematics.

```
>>> max(7.5, 9.5)
9.5
```

This call expression has subexpressions: the *operator* is an expression that precedes parentheses, which enclose a comma-delimited list of *operand* expressions. [\[1\]](#)



The operator evaluates to the *function* to be called. Each operand evaluates to one of the *arguments*. Then, the function is *applied* to its arguments.

The order of the arguments in a call expression matters. For instance, the function `pow` raises its first argument to the power of its second argument.

```
>>> pow(100, 2)
10000
>>> pow(2, 100)
1267650600228229401496703205376
```

Function notation has three principal advantages over the mathematical convention of placing an operator such as `+` between two expressions, which is called *infix notation*. First, functions may take an arbitrary number of arguments, not just two:

```
>>> max(1, -2, 3, -4)
3
```

Second, function notation supports *nested* expressions, where the elements are themselves compound expressions. Unlike operators such as `+` and `*` that have order-of-operations rules that must be memorized, the order of a nested call expression is explicit in its nested parentheses.

```
>>> max(min(1, -2), min(pow(3, 5), -4))
-2
```

There is no limit (in principle) to the depth of such nesting and to the overall complexity of the expressions that the Python interpreter can evaluate. However, humans quickly get confused by deep nesting. An important role for you as a programmer is to structure expressions so that they remain interpretable by yourself, your programming partners, and other people who may read your expressions in the future.

Third, mathematical notation has a great variety of forms: multiplication appears between terms, exponents appear as superscripts, division as a horizontal bar, and a square root as a roof with slanted siding. Some of this notation is very hard to type! A convenient way to simplify all of this notation in a way that is easy to type is to express all operations as call expressions.

Importing Library Functions

Python defines a very large number of functions but does not make all of their names available by default. Instead, it organizes built-in functions into modules, which together comprise the Python Standard Library. To use these elements, one imports them. For example, the `math` module provides a variety of familiar mathematical functions:

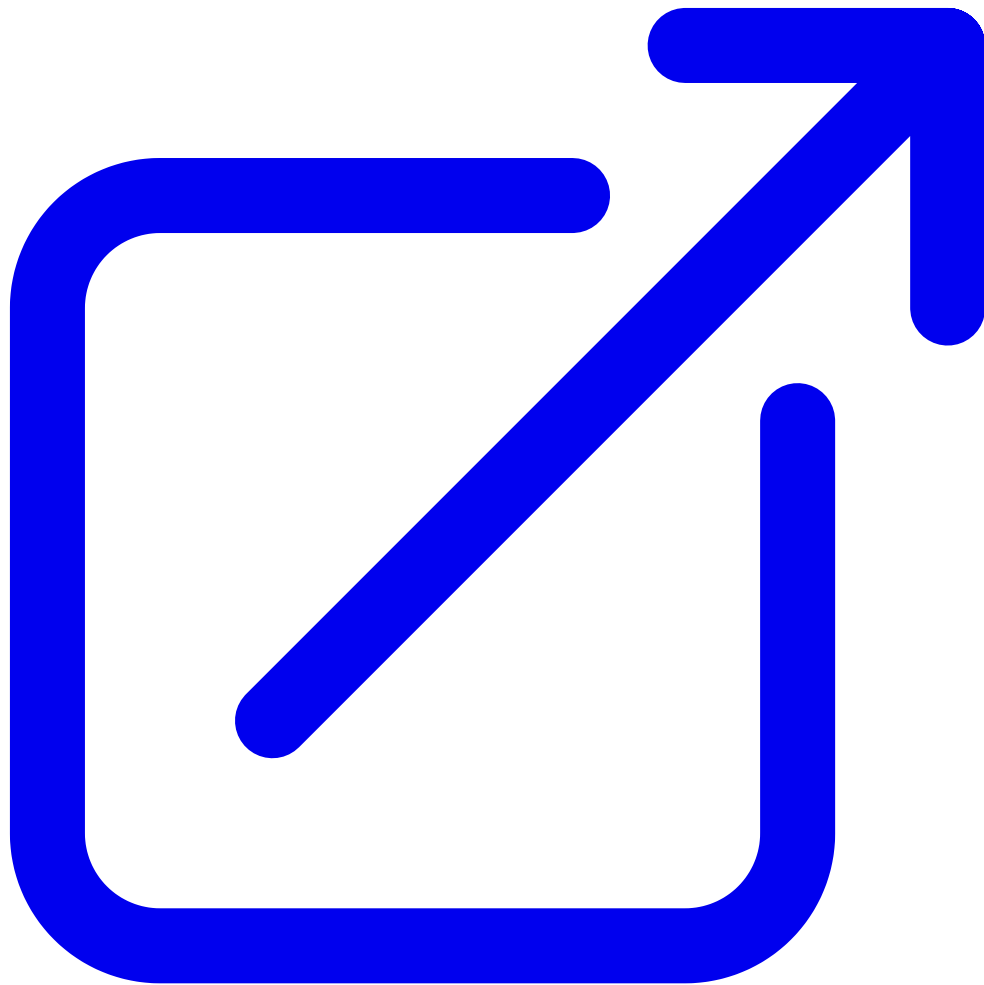
```
>>> from math import sqrt
>>> sqrt(256)
16.0
```

and the `operator` module provides access to functions corresponding to infix operators:

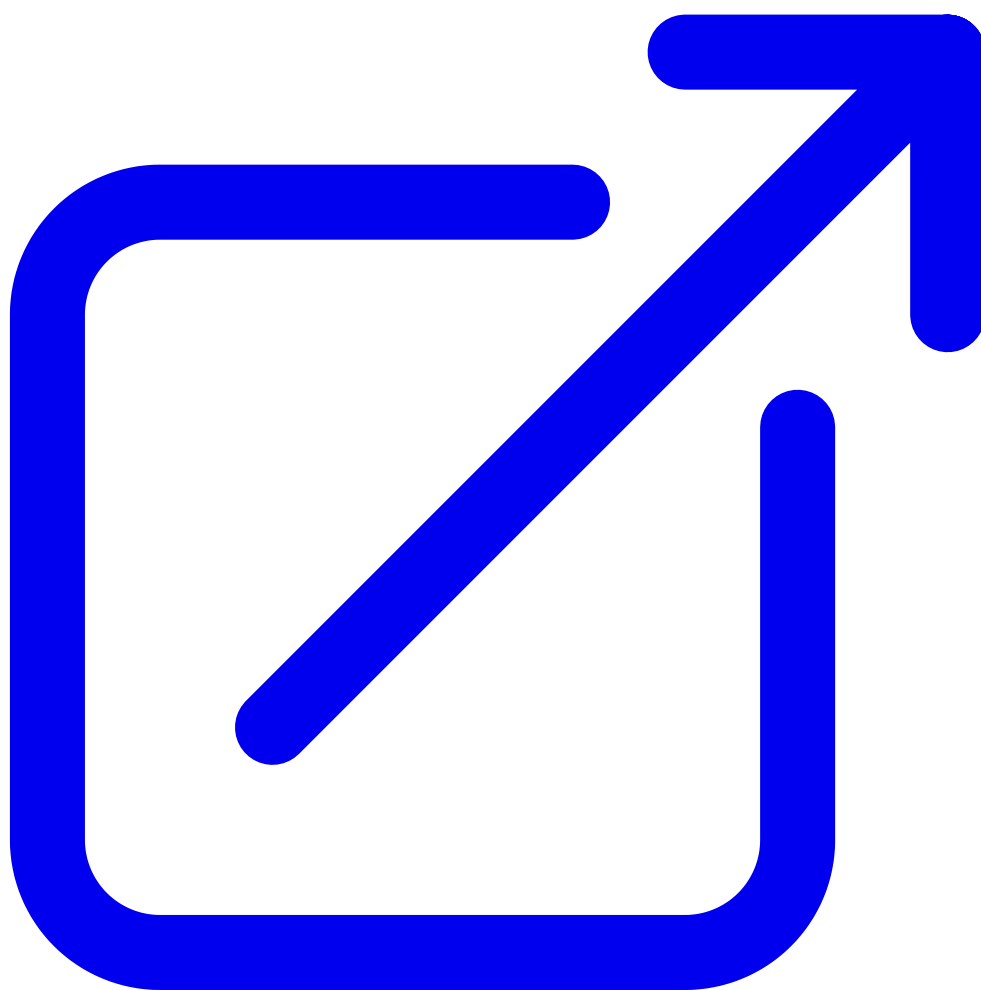
```
>>> from operator import add, sub, mul
>>> add(14, 28)
42
>>> sub(100, mul(7, add(8, 4)))
16
```

An `import` statement designates a module name (e.g., `operator` or `math`), and then lists the named attributes of that module to import (e.g., `sqrt`). Once a function is imported, it can be called multiple times.

There is no difference between using these `operator` functions (e.g., `add`) and the operator symbols themselves (e.g., `+`).



list the functions defined by each module, such as the [math module](#)



. However, this documentation is written for developers who know the whole language well. For now, you may find that experimenting with a function tells you more about its behavior than reading the documentation. As you become familiar with the Python language and vocabulary, this documentation will become a valuable reference source.

Names and the Environment[1](#)

A core feature of a programming language is its ways of assigning names to values. If some value has been given a name, we say that the name *binds* to the value.

In Python, we can establish new bindings using an assignment statement, which contains a name to the left of `=` and a value to the right:

```
>>> radius = 10
>>> radius
10
>>> 2 * radius
20
>>> radius
10
>>> diameter = 2 * radius
>>> diameter
20
```

Names are also bound via `import` statements.

```
>>> from math import pi
>>> pi * 71 / 223
1.0002380197528042
```

The `=` symbol is called the *assignment* operator in Python (and many other languages). Assignment is our simplest means of *abstraction*, for it allows us to use simple names to refer to the results of compound expressions, such as the `diameter` computed above. In this way, complex programs are constructed by building, step by step, data of increasing complexity.

The possibility of binding names to values and later retrieving those values by name means that the interpreter must maintain some sort of memory that keeps track of the names, values, and bindings. This memory is called an *environment*.

Names can also be bound to functions. For instance, the name `max` is bound to the `max` function we have been using. Functions, unlike numbers, are tricky to render as text, so Python prints an identifying description instead, when asked to describe a function:

```
>>> max
<built-in function max>
```

We can use assignment statements to give new names to existing functions.

```
>>> f = max
>>> f
<built-in function max>
>>> f(2, 3, 4)
4
```

And successive assignment statements can rebind a name to a new value.

```
>>> f = 2
>>> f
2
```

In Python, names are often called *variable names* or *variables* because they can be bound to different values in the course of executing a program. When a name is bound to a new value through assignment, it is no longer bound to any previous value. One can even bind built-in names to new values.

```
>>> max = 5
>>> max
5
```

After assigning `max` to 5, the name `max` is no longer bound to a function, and so attempting to call `max(2, 3, 4)` will cause an error.

When executing an assignment statement, Python evaluates the expression to the right of `=` before changing the binding to the name on the left. Therefore, one can refer to a name in the right-side expression, even if it is the name to be bound by the assignment statement.

```
>>> x = 2
>>> x = x + 1
>>> x
3
```

We can also assign multiple values to multiple names in a single statement, where names on the left of `=` and expressions on the right of `=` are separated by commas.

```
>>> area, circumference = pi * radius * radius, 2 * pi * radius
>>> area
314.1592653589793
>>> circumference
62.83185307179586
```

Changing the value of one name does not affect other names. Below, even though the name `area` was bound to a value defined originally in terms of `radius`, the value of `area` has not changed. Updating the value of `area` requires another assignment statement.

```
>>> radius = 11
>>> area
314.1592653589793
>>> area = pi * radius * radius
>>> area
380.1327110843649
```

With multiple assignment, *all* expressions to the right of `=` are evaluated before *any* names to the left are bound to those values. As a result of this rule, swapping the values bound to two names can be performed in a single statement.

```
>>> x, y = 3, 4.5
>>> y, x = x, y
>>> x
4.5
>>> y
3
```

Evaluating Nested Expressions

The process of evaluating expressions is itself a computational procedure that we can describe as a sequence of steps. For example, in evaluating nested call expressions, the interpreter is following this procedure:

1. Evaluate the operator and operand subexpressions, then
2. Apply the function that is the value of the operator subexpression to the arguments that are the values of the operand subexpressions.

Even this simple procedure illustrates some important points about processes in general. The first step dictates that in order to accomplish the evaluation process for a call expression we must first evaluate other expressions. Thus, the evaluation procedure is *recursive* in nature; that is, it includes, as one of its steps, the need to invoke the rule itself.

For example, evaluating

```
>>> mul(add(2, mul(4, 6)), add(3, 5))
208
```

requires that this evaluation procedure be applied four times. If we draw each expression that we evaluate, we can visualize the hierarchical structure of this process.



This illustration is called an *expression tree*. Each position within the tree is an expression paired with its value.

Evaluating the full expression at the top requires first evaluating the subexpressions below it. The bottom expressions are primitive expressions that evaluate to either functions or numbers. Viewing evaluation in terms of this tree, we can imagine that the values of the operands percolate upward, starting from the primitive expressions at the bottom and then combining at higher and higher levels.

Next, observe that the repeated application of the first step brings us to the point where we need to evaluate, not call expressions, but primitive expressions such as numerals (e.g., 2) and names (e.g., `add`). We take care of the primitive cases by stipulating that

- A numeral evaluates^[2] to the number it names,
- A (variable) name evaluates to the value bound to that name in the current environment.

Notice the important role of an environment in determining the values of the symbols in expressions. In Python, it is meaningless to speak of the value of an expression such as

```
>>> add(x, 1)
```

without specifying any information about the environment that would provide a value for the name `x` (or even for the name `add`). Environments provide the context in which evaluation takes place, which plays an important role in our understanding of program execution.

This evaluation procedure does not suffice to evaluate all Python code, only call expressions, numerals, and names. For instance, it does not handle assignment statements. Executing

```
>>> x = 3
```

does not return a value nor apply a function to some arguments, since the purpose of assignment is instead to bind a name to a value. Statements are not evaluated but *executed*; they do not produce a value but instead make some change. Each type of expression or statement has its own evaluation or execution procedure.

Pure and Non-Pure Functions¹

Throughout this text, we will distinguish between two types of functions.

Pure Functions¹

Pure functions have some input (their arguments) and return some output (the result of applying them), and that's it. Pure functions may perform computation, but what makes them pure is that they don't have any other impact on the program or computer besides providing their return value and always return the same value each time they are called on the same arguments.

The built-in function `abs`, which computes and returns the absolute value of its argument, is an example.

```
>>> abs(-2)
2
```

It can be depicted as a small machine that takes input and produces output.



Non-Pure Functions

In addition to returning a value, applying a non-pure function can generate *side effects*, which make some change to the state of the interpreter or computer. A common side effect is to generate output on the screen. The `print` function does this.

```
>>> print(1, 2, 3)
1 2 3
```

While `print` and `abs` may appear to be similar in these examples, they work in fundamentally different ways. The value that `print` returns is always `None`, a special Python value that represents nothing. The interactive Python interpreter does not automatically print the value `None`. In the case of `print`, the function itself is printing output as a side effect of being called.



Assigning the return value to a name shows the difference.

```
>>> x = abs(-3)
>>> print(x)
3
>>> y = print(-3)
-3
>>> print(y)
None
```

Because `print` does not return its argument, the result of calling `print` can't be used in some more complicated expression. For example, while `abs(-3) * 2` is 6 because `abs(-3)` returns 3, `print(-3) * 2` causes an error because `print(-3)` returns `None`, which can't be multiplied by 2.

A nested expression of calls to `print` is an interesting case to study. Each call to `print` displays one line, and calling `print` on the return value of a call to `print` displays `None`.

```
>>> print(print(1), print(2))
1
2
None None
```

If you find this output to be unexpected, draw an expression tree to clarify why evaluating this expression produces this peculiar output.

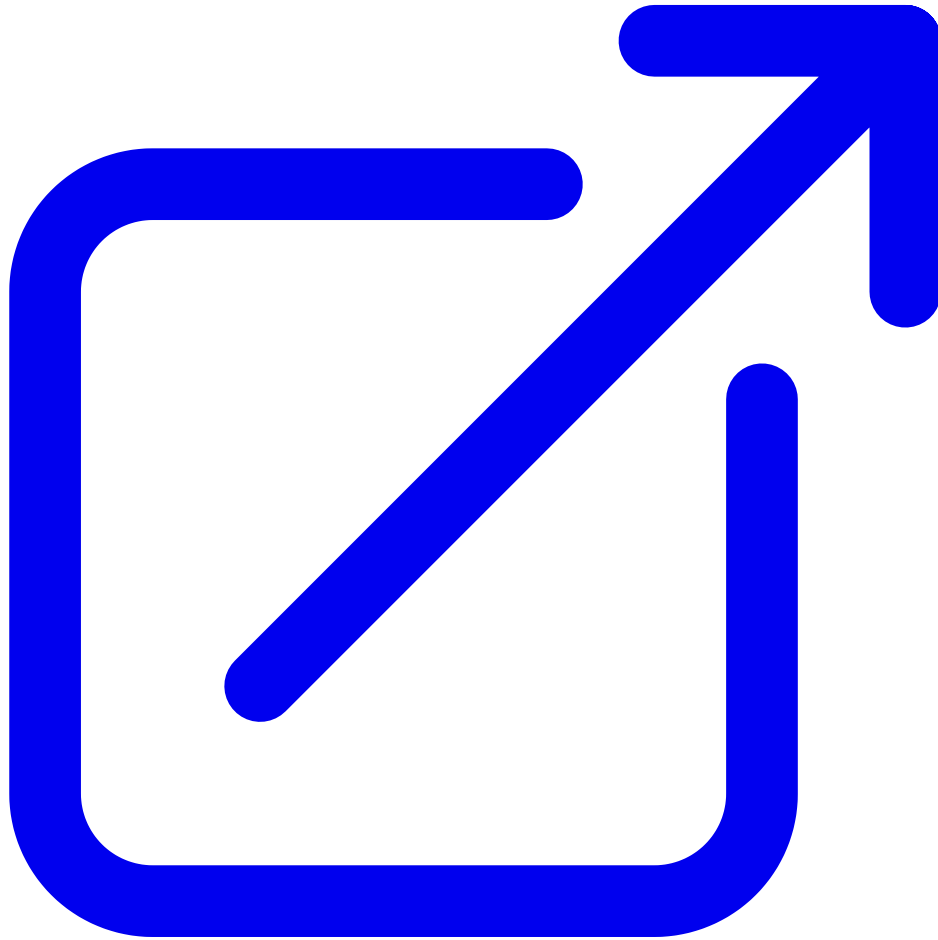
Pure functions are restricted in that they cannot have side effects or change behavior over time. Imposing these restrictions yields substantial benefits. First, pure functions can be composed reliably into compound call expressions. We have seen that functions such as `max`, `pow` and `sqrt` can be used effectively in nested expressions.

Pure functions have other advantages, such as making complex programs easier to understand and more predictable, as well as allowing multiple functions to be applied concurrently.

For these reasons, we concentrate heavily on creating and using pure functions in the remainder of this chapter. Non-pure functions will be discussed at length in Chapter 2.

Footnotes

1. The [official Python language reference](#)



uses different terminology to describe the parts of call expressions. Rather than *operator*, the expression before parentheses is called the *primary* (or sometimes just the *function*). Rather than *operand*, the expressions within parentheses are called *argument expressions*. This text uses the more classic *operator* and *operand* terms because they can help distinguish between expressions (which are code) and values (which are data).



2. A pedantic note: when we say that some expression evaluates to a value, we actually mean that the Python interpreter evaluates that expression to that value. It is the interpreter that endows meaning to the programming language by running programs. Since the interpreter is a fixed program that always behaves consistently, it's natural to talk about what expressions evaluate to what values in the context of Python programs.



[Chapter 1: Functions](#)

[1.1 Getting Started](#)

[Chapter 1: Functions](#)

[1.3 Defining New Functions](#)



1.3 Defining New Functions



1.3 Defining New Functions

```
from pytutor import env_diagram
```

We have identified in Python some of the elements that must appear in any powerful programming language:

1. Numbers and arithmetic are *primitive* built-in data values and functions or operators.
2. Nested function application provides a means of *combining* operations.
3. Binding names to values provides a limited means of *abstraction*.

Now we will learn about *function definitions*, a much more powerful abstraction technique by which a name can be bound to a compound operation, which can then be referred to as a unit.

We begin by examining how to express the idea of *squaring*. We might say, "To square something, multiply it by itself." This is expressed in Python as

```
>>> def square(x):  
    return mul(x, x)
```

which defines a new function that has been given the name `square`. This user-defined function is not built into the interpreter. It represents the compound operation of multiplying something by itself. The `x` in this definition is called a *formal parameter*, which provides a name for the thing to be multiplied. The definition creates this user-defined function and binds it to the name `square`.

Def Statements¶

Function definitions consist of a `def` statement that indicates a `<name>` and a comma-separated list of named `<formal parameters>`, then a `return` statement, called the function body, that specifies the `<return expression>` of the function, which is an expression to be evaluated whenever the function is applied:

```
def <name>(<formal parameters>):  
    return <return expression>
```

The second line *must* be indented --- most programmers use four spaces to indent. The return expression is not evaluated right away; it is stored as part of the newly defined function and evaluated only when the function is eventually applied.

Having defined `square`, we can apply it with a call expression:

```
>>> square(21)
441
>>> square(add(2, 5))
49
>>> square(square(3))
81
```

We can also use `square` as a building block in defining other functions. For example, we can easily define a function `sum_squares` that, given any two numbers as arguments, returns the sum of their squares:

```
>>> def sum_squares(x, y):
    return add(square(x), square(y))

>>> sum_squares(3, 4)
25
```

User-defined functions are used in exactly the same way as built-in functions. Indeed, one cannot tell from the definition of `sum_squares` whether `square` is built into the interpreter, imported from a module, or defined by the user.

Both `def` statements and assignment statements bind names to values, and any existing bindings are lost. For example, `g` below first refers to a function of no arguments, then a number, and then a different function of two arguments.

```
>>> def g():
    return 1
>>> g()
1
>>> g = 2
>>> g
2
>>> def g(h, i):
    return h + i
>>> g(1, 2)
3
```

Environments

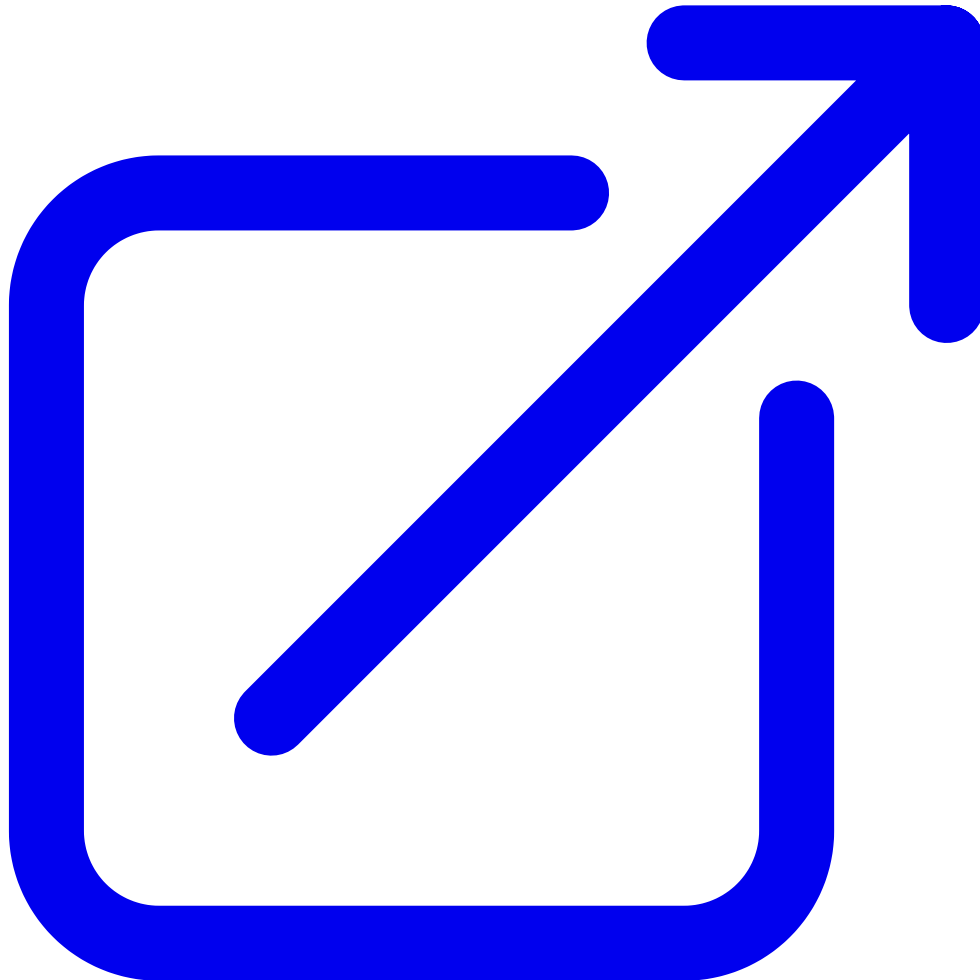
Our subset of Python is now complex enough that the meaning of programs is non-obvious. What if a formal parameter has the same name as a built-in function? Can two functions share names without confusion? To resolve such questions, we must describe environments in more detail.

An environment in which an expression is evaluated is a sequence of *frames*, depicted as boxes. Each frame contains *bindings*, each of which associates a name with its corresponding value. Within a frame, a name can only appear once. There is a single *global* frame. Assignment and import statements add entries to the first frame of the current environment. So far, our environment consists only of the global frame.

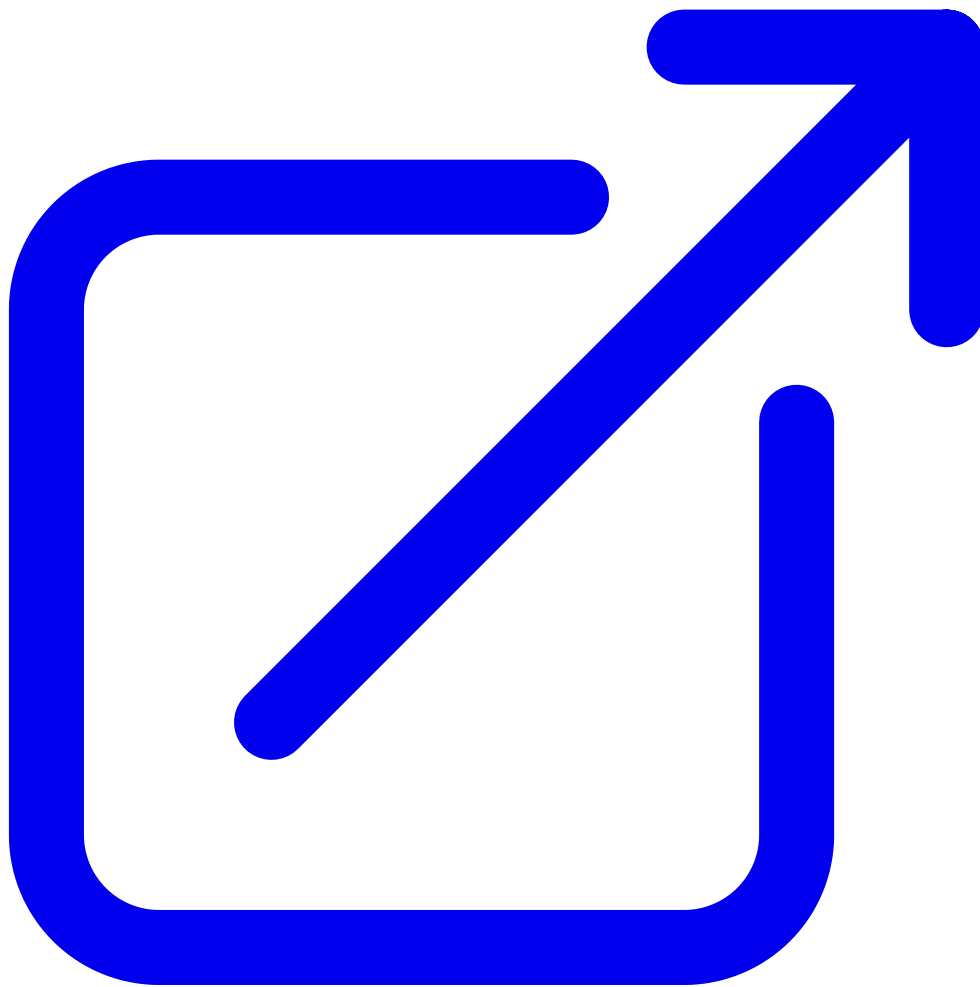
```
env_diagram(r'''  
from math import pi  
tau = 2 * pi  
''', cumulative=False)
```

Loading...

This *environment diagram* shows the bindings of the current environment, along with the values to which names are bound. The environment diagrams in this text are interactive: you can step through the lines of the small program on the left to see the state of the environment evolve on the right. You can also click on the “Edit code in Online Python Tutor” link to load the example into the [Online Python Tutor](#).



, a tool created by [Philip Guo](#)



for generating these environment diagrams. You are encouraged to create examples yourself and study the resulting environment diagrams.

Functions appear in environment diagrams as well. An `import` statement binds a name to a built-in function. A `def` statement binds a name to a user-defined function created by the definition. The resulting environment after importing `mul` and defining `square` appears below:

```
env_diagram(r'''
from operator import mul
def square(x):
    return mul(x, x)
''', cumulative=False, labels=False)
```

Loading...

Each function is a line that starts with `func`, followed by the function name and formal parameters. Built-in functions such as `mul` do not have formal parameter names, and so `...` is always used instead.

The name of a function is repeated twice, once in the frame and again as part of the function itself. The name appearing in the function is called the *intrinsic name*. The name in a frame is a *bound name*. Two or more different bound names may refer to the same function, but that function itself has only one intrinsic name that never changes.

The names bound to functions in frames are the ones used during evaluation. The intrinsic name of a function does not play a role in evaluation. Step through the example below using the *Forward* button to see that once the name `max` is bound to the value 3, it can no longer be used as a function.

```
env_diagram(r'''
f = max
max = 3
result = f(2, 3, 4)
max(1, 2) # Causes an error
''', step=2, cumulative=False, labels=False)
```

Loading...

The error message `TypeError: 'int' object is not callable` is reporting that the name `max` (currently bound to the number 3) is an integer and not a function. Therefore, it cannot be used as the operator in a call expression.

Function signatures. Functions differ in the number of arguments that they are allowed to take. To track these requirements, we draw each function in a way that shows the function name and its formal parameters. The user-defined function `square` takes only `x`; providing more or fewer arguments will result in an error. A description of the formal parameters of a function is called the function's signature.

The function `max` can take an arbitrary number of arguments. It is rendered as `max(...)`. Regardless of the number of arguments taken, all built-in functions will be rendered as `<name>(...)`, because these primitive functions were never explicitly defined.

Calling User-Defined Functions¹

To evaluate a call expression whose operator names a user-defined function, the Python interpreter follows a computational process. As with any call expression, the interpreter evaluates the operator and operand expressions, and then applies the named function to the resulting arguments.

Applying a user-defined function introduces a second *local* frame, which is only accessible to that function. To apply a user-defined function to some arguments:

1. Bind the arguments to the names of the function's formal parameters in a new *local* frame.
2. Execute the body of the function in the environment that starts with this frame.

The environment in which the body is evaluated consists of two frames: first the local frame that contains formal parameter bindings, then the global frame that contains everything else. Each instance of a function application has its own independent local frame.

To illustrate an example in detail, several steps of the environment diagram for the same example are depicted below. After executing the first import statement, only the name `mul` is bound in the global frame.

```
env_diagram(r'''
from operator import mul
def square(x):
    return mul(x, x)
square(-2)
''', step=1, cumulative=True, labels=False)
```

Loading...

First, the definition statement for the function `square` is executed. Notice that the entire `def` statement is processed in a single step. The body of a function is not executed until the function is called (not when it is defined).

```
env_diagram(r'''
from operator import mul
def square(x):
    return mul(x, x)
square(-2)
''', step=2, cumulative=True, labels=False)
```

Loading...

Next, the `square` function is called with the argument `-2`, and so a new frame is created with the formal parameter `x` bound to the value `-2`.

```
env_diagram(r'''
from operator import mul
def square(x):
    return mul(x, x)
square(-2)
''', step=3, cumulative=True, labels=False)
```

Loading...

Then, the name `x` is looked up in the current environment, which consists of the two frames shown. In both occurrences, `x` evaluates to `-2`, and so the `square` function returns 4.

```
env_diagram(r'''
from operator import mul
def square(x):
    return mul(x, x)
square(-2)
''', step=4, cumulative=True, labels=False)
```

Loading...

The “Return value” in the `square()` frame is not a name binding; instead it indicates the value returned by the function call that created the frame.

Even in this simple example, two different environments are used. The top-level expression `square(-2)` is evaluated in the global environment, while the return expression `mul(x, x)` is evaluated in the environment created by calling `square`. Both `x` and `mul` are bound in this environment, but in different frames.

The order of frames in an environment affects the value returned by looking up a name in an expression. We stated previously that a name is evaluated to the value associated with that name in the current environment. We can now be more precise:

Name evaluation. A name evaluates to the value bound to that name in the earliest frame of the current environment in which that name is found. (Here, “earliest” refers to the order in which frames are inspected, starting with the local frame for the function being called.)

The purpose of having multiple frames is to allow the same name to have different meanings in different contexts. Natural languages such as English also have this property: the word “president” refers to different people when used to refer to the President of the United States on an American news broadcast, which is a broad global context shared by millions of viewers and when used to refer to the president of a student group during a meeting of the members of that group, which is a much more local context. Within the student group meeting, the more locally relevant meaning of “president” is used rather than the more global context.

This approach to name evaluation means that a name used within the body of a function (the indented code of the `def` statement) will prioritize formal parameters and assignment statements within the function over global names. If the same name is used in both the local and global frame, we say that the local name *shadows* the global name.

Our conceptual framework of environments, names, and functions constitutes a *model of evaluation*, which precisely and correctly describes how the interpreter evaluates call expressions. It is called a model because it leaves out some details, such as how the computer’s memory is organized. Those details turn out to be important for speed optimization, but not for understanding the behavior of a program.

Example: Calling a User-Defined Function¶

Let us again consider our two simple function definitions and illustrate the process that evaluates a call expression for a user-defined function.

```
env_diagram(r'''
from operator import add, mul
def square(x):
    return mul(x, x)

def sum_squares(x, y):
    return add(square(x), square(y))

result = sum_squares(5, 12)
''', step=3, cumulative=True, labels=False)
```

Loading...

Python first evaluates the name `sum_squares`, which is bound to a user-defined function in the global frame. The primitive numeric expressions 5 and 12 evaluate to the numbers they represent.

Next, Python applies `sum_squares`, which introduces a local frame that binds `x` to 5 and `y` to 12.

```
env_diagram(r'''
from operator import add, mul
def square(x):
    return mul(x, x)

def sum_squares(x, y):
    return add(square(x), square(y))

result = sum_squares(5, 12)
''', step=4, cumulative=True, labels=False)
```

Loading...

The body of `sum_squares` contains this call expression:

```

add      (  square(x)  ,  square(y)  )
-----
operator  operand 0    operand 1

```

All three subexpressions are evaluated in the current environment, which begins with the frame labeled `sum_squares()`. The operator subexpression `add` is a name found in the global frame, bound to the built-in function for addition. The two operand subexpressions must be evaluated in turn, before addition is applied. Both operands are evaluated in the current environment beginning with the frame labeled `sum_squares`.

In operand 0, `square` names a user-defined function in the global frame, while `x` names the number 5 in the local frame. Python applies `square` to 5 by introducing yet another local frame that binds `x` to 5.

```

env_diagram(r'''
from operator import add, mul
def square(x):
    return mul(x, x)

def sum_squares(x, y):
    return add(square(x), square(y))

result = sum_squares(5, 12)
''' , step=5, cumulative=True, labels=False)

```

Loading...

Using this environment, the expression `mul(x, x)` evaluates to 25.

Our evaluation procedure now turns to operand 1, for which `y` names the number 12. Python evaluates the body of `square` again, this time introducing yet another local frame that binds `x` to 12. Hence, operand 1 evaluates to 144.

```

env_diagram(r'''
from operator import add, mul
def square(x):
    return mul(x, x)

def sum_squares(x, y):
    return add(square(x), square(y))

result = sum_squares(5, 12)
''' , step=8, cumulative=True, labels=False)

```

Loading...

Finally, applying addition to the arguments 25 and 144 yields a final return value for `sum_squares`: 169.

```

env_diagram(r'''
from operator import add, mul
def square(x):
    return mul(x, x)

def sum_squares(x, y):
    return add(square(x), square(y))

result = sum_squares(5, 12)
''' , cumulative=True, labels=False)

```

Loading...

This example illustrates many of the fundamental ideas we have developed so far. Names are bound to values, which are distributed across many independent local frames, along with a single global frame that contains shared names. A new local frame is introduced every time a function is called, even if the same function is called twice.

All of this machinery exists to ensure that names resolve to the correct values at the correct times during program execution. This example illustrates why our model requires the complexity that we have introduced. All three local frames contain a binding for the name `x`, but that name is bound to different values in different frames. Local frames keep these names separate.

Local Names

One detail of a function's implementation that should not affect the function's behavior is the implementer's choice of names for the function's formal parameters. Thus, the following functions should provide the same behavior:

```

>>> def square(x):
    return mul(x, x)
>>> def square(y):
    return mul(y, y)

```

This principle -- that the meaning of a function should be independent of the parameter names chosen by its author -- has important consequences, such as the fact that the parameter names of a function only refer to their arguments within the body of the function. They are *local* to the function.

If the parameters were not local to the bodies of their respective functions, then the parameter `x` in `square` could be confused with the parameter `x` in `sum_squares`. Critically, this is not the case: the bindings for `x` in different local frames are unrelated. The language is designed to ensure this independence.

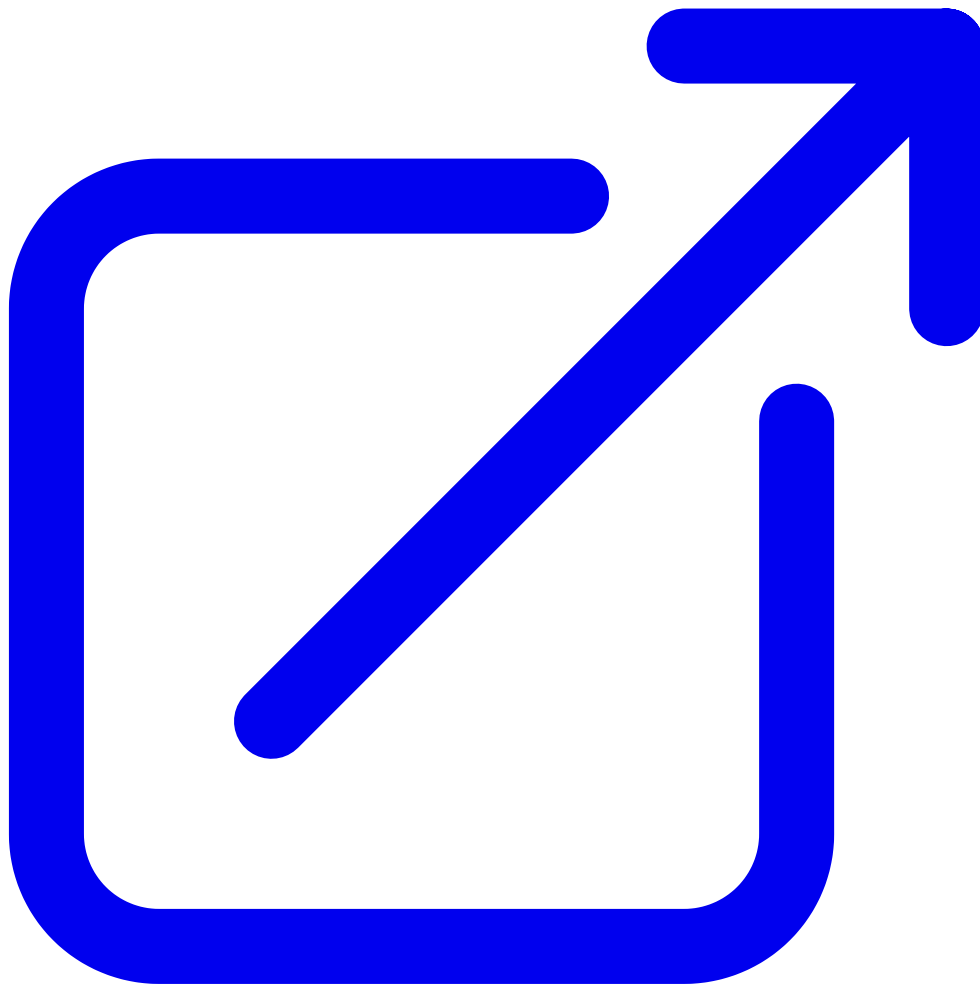
We say that the *scope* of a local name is limited to the body of the user-defined function that defines it. When a name is no longer accessible, it is out of scope. This scoping behavior isn't a new fact about our model; it is a consequence of the way environments work.

How do scope and environments relate? Scope is about the code: it describes where in the code a name can be used. For example, a function's parameter names can be used in the function's body, but not outside of it. Environments are about evaluating code: they keep track of the bindings between names and values as a program runs. Scope and environments are related because the rules by which different environments are created and used ensure that the names which are in scope are always in the current environment, and the names which are out of scope are not.

Choosing Names¹

The interchangeability of names does not imply that formal parameter names do not matter at all. On the contrary, well-chosen function and parameter names are essential for the human interpretability of function definitions!

The following guidelines are adapted from the [style guide for Python code](#)



, which serves as a guide for all (non-rebellious) Python programmers. A shared set of conventions smooths communication among members of a developer community. As a side effect of following these conventions, you will find that your code becomes more internally consistent.

1. Function names are lowercase, with words separated by underscores. Descriptive names are encouraged.
2. Function names typically evoke operations applied to arguments by the interpreter (e.g., `print`, `add`, `square`) or the name of the quantity that results (e.g., `max`, `abs`, `sum`).
3. Parameter names are lowercase, with words separated by underscores. Single-word names are preferred.
4. Parameter names should evoke the role of the parameter in the function, not just the kind of argument that is allowed.
5. Single letter parameter names are acceptable when their role is obvious, but avoid "l" (lowercase ell), "O" (capital oh), or "I" (capital i) to avoid confusion with numerals.

There are many exceptions to these guidelines, even in the Python standard library. Like the vocabulary of the English language, Python has inherited words from a variety of contributors, and the result is not always consistent.

Functions as Abstractions

Though it is very simple, `sum_squares` exemplifies the most powerful property of user-defined functions. The function `sum_squares` is defined in terms of the function `square`, but relies only on the relationship that `square` defines between its input arguments and its output values, not how it works.

We can write `sum_squares` without concerning ourselves with *how* to square a number. The details of how the square is computed can be ignored when reading the body of `sum_squares`. Indeed, as far as `sum_squares` is concerned, `square` is not a particular function body, but rather an abstraction of a function, a so-called functional abstraction. It could square by multiplying or by repeatedly adding or first multiplying then adding. Any function that computes the square is equally good.

Thus, considering only the values they return, the following two functions for squaring a number should be indistinguishable. Each takes a numerical argument and produces the square of that number as the value.

```
>>> def square(x):  
    return mul(x, x)  
>>> def square(x):  
    return mul(x, x-1) + x
```

In other words, a function definition gives a name to some relationship between inputs and outputs, but abstracts away the details. The users of the function may not have written the function themselves and might not even know how it works. A programmer should not need to know how the function is implemented in order to use it. The Python Library has this property. Many developers use the functions defined there, but few ever inspect their implementation. That's abstraction at work.

Aspects of a functional abstraction. To master the use of a functional abstraction, it is often useful to consider its three core attributes. The *domain* of a function is the set of arguments it can take. The *range* of a function is the set of values it can return. The *intent* or *behavior* of a function is the relationship it computes between inputs and output (as well as any side effects it might generate). Understanding functional abstractions via their domain, range, and intent is critical to using them correctly in a complex program.

For example, any `square` function that we use to implement `sum_squares` should have these attributes:

- The *domain* is any single real number.
- The *range* is any non-negative real number.
- The *intent* is that the output is the square of the input.

These attributes do not specify how the intent is carried out; that detail is abstracted away.

Operators

Mathematical operators (such as `+` and `-`) provided our first example of a method of combination, but we have yet to define an evaluation procedure for expressions that contain these operators.

Python expressions with infix operators each have their own evaluation procedures, but you can often think of them as short-hand for call expressions. When you see

```
>>> 2 + 3
5
```

simply consider it to be short-hand for

```
>>> add(2, 3)
5
```

Infix notation can be nested, just like call expressions. Python applies the normal mathematical rules of operator precedence, which dictate how to interpret a compound expression with multiple operators.

```
>>> 2 + 3 * 4 + 5
19
```

evaluates to the same result as

```
>>> add(add(2, mul(3, 4)), 5)
19
```

The nesting in the call expression is more explicit than the operator version, but also harder to read. Python also allows subexpression grouping with parentheses, to override the normal precedence rules or make the nested structure of an expression more explicit.

```
>>> (2 + 3) * (4 + 5)
45
```

evaluates to the same result as

```
>>> mul(add(2, 3), add(4, 5))
45
```

When it comes to division, Python provides two infix operators: `/` and `//`. The former is normal division, so that it results in a *floating point*, or decimal value, even if the divisor evenly divides the dividend:

```
>>> 5 / 4
1.25
>>> 8 / 4
2.0
```

The `//` operator, on the other hand, rounds the result down to an integer:

```
>>> 5 // 4
1
>>> -5 // 4
-2
```

These two operators are shorthand for the `truediv` and `floordiv` functions.

```
>>> from operator import truediv, floordiv
>>> truediv(5, 4)
1.25
>>> floordiv(5, 4)
1
```

You should feel free to use infix operators and parentheses in your programs. Idiomatic Python prefers operators over call expressions for simple mathematical operations.



[Chapter 1: Functions](#)

[1.2 Elements of Programming](#)

[Chapter 1: Functions](#)

[1.4 Designing Functions](#)



1.4 Designing Functions



1.4 Designing Functions

Functions are an essential ingredient of all programs, large and small, and serve as our primary medium to express computational processes in a programming language. So far, we have discussed the formal properties of functions and how they are applied. We now turn to the topic of what makes a good function. Fundamentally, the qualities of good functions all reinforce the idea that functions are abstractions.

- Each function should have exactly one job. That job should be identifiable with a short name and characterizable in a single line of text. Functions that perform multiple jobs in sequence should be divided into multiple functions.
- *Don't repeat yourself* is a central tenet of software engineering. The so-called DRY principle states that multiple fragments of code should not describe redundant logic. Instead, that logic should be implemented once, given a name, and applied multiple times. If you find yourself copying and pasting a block of code, you have probably found an opportunity for functional abstraction.
- Functions should be defined generally. Squaring is not in the Python Library precisely because it is a special case of the `pow` function, which raises numbers to arbitrary powers.

These guidelines improve the readability of code, reduce the number of errors, and often minimize the total amount of code written. Decomposing a complex task into concise functions is a skill that takes experience to master. Fortunately, Python provides several features to support your efforts.

Documentation¹

A function definition will often include documentation describing the function, called a *docstring*, which must be indented along with the function body. Docstrings are conventionally triple quoted. The first line describes the job of the function in one line. The following lines can describe arguments and clarify the behavior of the function:

```
>>> def pressure(v, t, n):
    """Compute the pressure in pascals of an ideal gas.

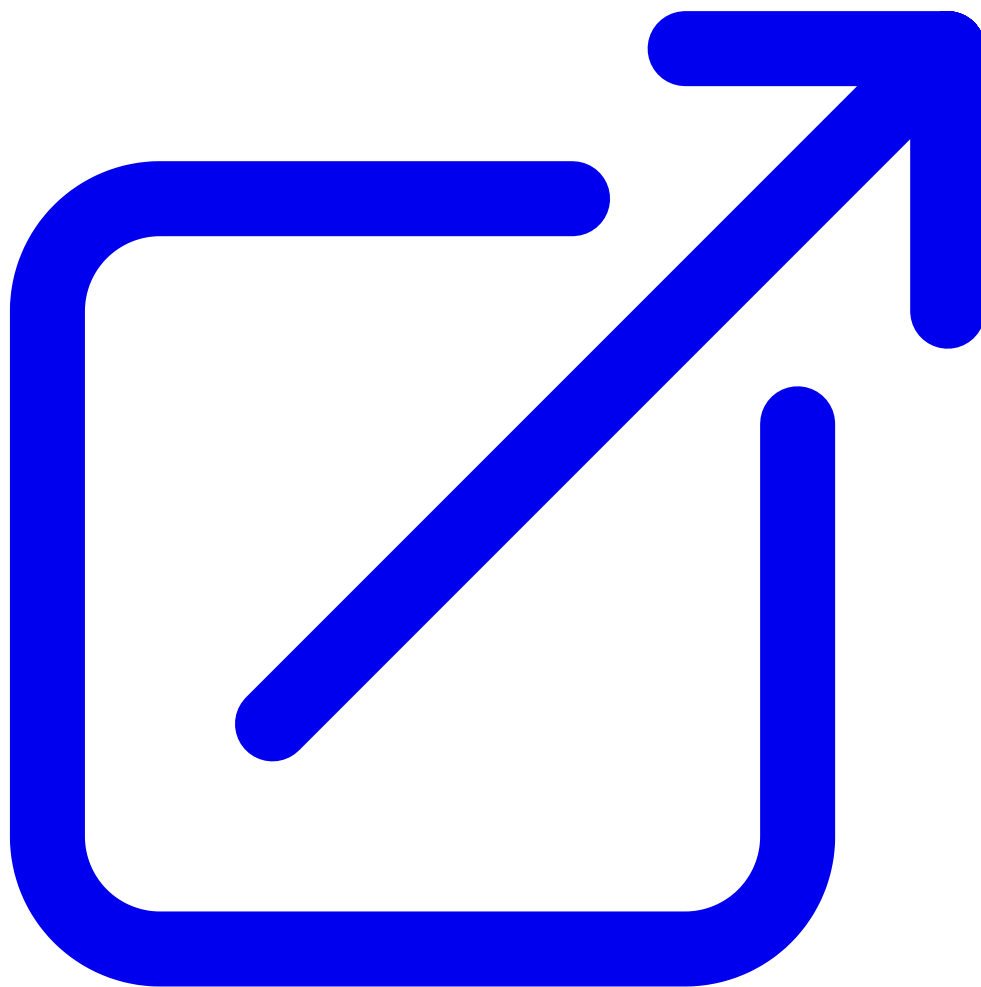
    Applies the ideal gas law: https://en.wikipedia.org/wiki/Ideal_gas_law

    v -- volume of gas, in cubic meters
    t -- absolute temperature in degrees kelvin
    n -- particles of gas
    """
    k = 1.38e-23 # Boltzmann's constant
    return n * k * t / v
```

When you call `help` with the name of a function as an argument, you see its docstring (type `q` to quit Python help).

```
>>> help(pressure)
```

When writing Python programs, include docstrings for all but the simplest functions. Remember, code is written only once, but often read many times. The Python docs include [docstring guidelines](#)



that maintain consistency across different Python projects.

Comments. Comments in Python can be attached to the end of a line following the `#` symbol. For example, the comment `Boltzmann's constant` above describes `k`. These comments don't ever appear in Python's `help`, and they are ignored by the interpreter. They exist for humans alone.

Default Argument Values¹

A consequence of defining general functions is the introduction of additional arguments. Functions with many arguments can be awkward to call and difficult to read.

In Python, we can provide default values for the arguments of a function. When calling that function, arguments with default values are optional. If they are not provided, then the default value is bound to the formal parameter name instead. For instance, if an application commonly computes pressure for one mole of particles, this value can be provided as a default:

```
>>> def pressure(v, t, n=6.022e23):  
    """Compute the pressure in pascals of an ideal gas.  
  
    v -- volume of gas, in cubic meters  
    t -- absolute temperature in degrees kelvin  
    n -- particles of gas (default: one mole)  
    """  
    k = 1.38e-23 # Boltzmann's constant  
    return n * k * t / v
```

The `=` symbol means two different things in this example, depending on the context in which it is used. In the `def` statement header, `=` does not perform assignment, but instead indicates a default value to use when the `pressure` function is called. By contrast, the assignment statement to `k` in the body of the function binds the name `k` to an approximation of Boltzmann's constant.

```
>>> pressure(1, 273.15)  
2269.974834  
>>> pressure(1, 273.15, 3 * 6.022e23)  
6809.924502
```

The `pressure` function is defined to take three arguments, but only two are provided in the first call expression above. In this case, the value for `n` is taken from the `def` statement default. If a third argument is provided, the default is ignored.

As a guideline, most data values used in a function's body should be expressed as default values to named arguments, so that they are easy to inspect and can be changed by the function caller. Some values that never change, such as the fundamental constant `k`, can be bound in the function body or in the global frame.



[Chapter 1: Functions](#)

[1.3 Defining New Functions](#)

[Chapter 1: Functions](#)

[1.5 Control](#)



1.5 Control



1.5 Control

```
from pytz import tzinfo
from operator import mul
```

The expressive power of the functions that we can define at this point is very limited because we have not introduced a way to make comparisons and to perform different operations depending on the result of a comparison. *Control statements* give us this ability. They are statements that control the flow of a program's execution based on the results of logical comparisons.

Control statements differ from the expressions that we have studied so far. They don't have a value like an expression does. Instead, executing a control statement determines what the interpreter should do next.

Statements

We have seen three kinds of statements already: assignment, `def`, and `return` statements. These lines of Python code are not themselves expressions, although they all contain expressions as components.

Rather than being evaluated, statements are *executed*. Each statement describes some change to the interpreter state, and executing a statement applies that change. As we have seen for `return` and assignment statements, executing statements can involve evaluating subexpressions contained within them.

The body of a function is a sequence of statements. In Python, an expression can be written on a line by itself, in which case it is treated as a statement and called an *expression statement*. The value of such an expression is discarded.

For example, leaving out the `return` keyword in the example below makes `mul(x, x)` an expression statement, and so `x` is multiplied by itself, but the result is discarded rather than returned.

```
>>> def square(x):
    mul(x, x) # Watch out! This call doesn't return a value.

>>> print(square(3))
None
>>> y = square(4)
>>> print(y)
None
```

In order to do something with the value of an expression, it must be part of some larger statement, such as an assignment or return statement.

```
>>> def square(x):
    return mul(x, x)

>>> print(square(3))
9
>>> y = square(4)
>>> print(y)
16
```

Sometimes it does make sense to have a function whose body is an expression, when a non-pure function like `print` is called. However, displaying a value with `print` does not return it, and so that value is not available to be assigned or otherwise combined further.

```
>>> def print_square(x):
    print(mul(x, x))

>>> print_square(3)
9
>>> y = print_square(4)
16
>>> print(y)
None
```

The Python interpreter's job is to execute programs, which are sequences of statements. However, much of the interesting work of computation comes from evaluating expressions. Statements control which expressions are evaluated in a program, what environments they are evaluated in, and what happens to their results.

Compound Statements

A simple statement is a single line that doesn't end in a colon. A compound statement is so called because it is composed of other statements (both simple and compound). Compound statements typically span multiple lines and start with a one-line header ending in a colon, which identifies the type of statement. Together, a header and an indented suite of statements is called a clause. A compound statement consists of one or more clauses:

```
<header>:
    <statement>
    <statement>
    ...
<separating header>:
    <statement>
    <statement>
    ...
...
```

We can understand the statements we have already introduced in these terms.

- Expression statements, return statements, and assignment statements are simple statements.
- A `def` statement is a compound statement. The suite that follows the `def` header defines the function body.

Specialized evaluation rules for each kind of header dictate when and if the statements in its suite are executed. We say that the header controls its suite. For example, in the case of `def` statements, we saw that the return expression is not evaluated immediately, but instead stored for later use when the defined function is eventually called.

We can also understand multi-line programs now.

- To execute a sequence of statements, execute the first statement. If that statement does not redirect control, then proceed to execute the rest of the sequence of statements, if any remain.

This definition exposes the essential structure of a recursively defined *sequence*: a sequence can be decomposed into its first item and the rest of its items. The *rest* of a sequence of statements is itself a sequence of statements! Thus, we can recursively apply this execution rule.

The important consequence of this rule is that statements are executed in order, but later statements may never be reached if control is redirected.

Practical guidance. When indenting a suite, all lines must be indented the same amount and in the same way (use spaces, not tabs). Any variation in indentation will cause an error.

Local Assignment

Originally, we stated that the body of a user-defined function consisted only of a `return` statement with a single return expression. In fact, functions can define a sequence of operations that extends beyond a single expression.

Whenever a user-defined function is applied, the sequence of clauses in the suite of its definition is executed in an environment starting with a local frame that is created by calling that function. A `return` statement redirects control: the process of function application ends whenever the first `return` statement is executed, and the value of the `return` expression is the returned value of the function being applied.

Assignment statements can appear within a function body. For instance, this function returns the absolute difference between two quantities as a percentage of the first, using a two-step calculation:

```
env_diagram(r'''
def percent_difference(x, y):
    difference = abs(x-y)
    return 100 * difference / x
result = percent_difference(40, 50)
''', cumulative=True, labels=False)
```

Loading...

The effect of an assignment statement is to bind a name to a value in the first frame of the current environment. As a consequence, assignment statements within a function body cannot affect the global frame. The fact that functions can only manipulate their local frame greatly simplifies programming. Pure functions interact only via the values they take and return, rather than affecting the behavior of other functions.

Of course, the `percent_difference` function could be written as a single expression, as shown below, but the return expression is more complex.

```
>>> def percent_difference(x, y):
    return 100 * abs(x-y) / x
>>> percent_difference(40, 50)
25.0
```

So far, local assignment hasn't increased the expressive power of our function definitions. It will do so when combined with other control statements. It can still be useful on its own to clarify the meaning of complex expressions by assigning names to intermediate quantities.

Conditional Statements

Python has a built-in function for computing absolute values.

```
>>> abs(-2)
2
```

We would like to be able to implement such a function ourselves, but doing so requires a comparison and a choice. We would like to express that if `x` is negative, `abs(x)` returns `-x`. Furthermore, if `x` is 0, `abs(x)` returns 0. Otherwise, `x` is positive and so `abs(x)` returns `x`. In Python, we can express this choice with a conditional statement.

```
env_diagram(r'''
def absolute_value(x):
    """Compute abs(x)."""
    if x < 0:
        return -x
    elif x == 0:
        return 0
    else:
        return x
''')
```

```
result = absolute_value(-2)
'''', cumulative=True, labels=False)
```

Loading...

A conditional statement in Python consists of a series of headers and suites: a required `if` clause, an optional sequence of `elif` clauses, and finally an optional `else` clause:

```
if <expression>:
    <suite>
elif <expression>:
    <suite>
else:
    <suite>
```

When executing a conditional statement, each clause is considered in order. The computational process of executing a conditional clause follows.

1. Evaluate the header's expression.
2. If it is a true value, execute the suite. Then, skip over all subsequent clauses in the conditional statement.

If the `else` clause is reached (which only happens if all `if` and `elif` expressions evaluate to false values), its suite is executed.

Boolean Values and Operators

Boolean contexts. Above, the execution procedures mention “a false value” and “a true value.” The expressions inside the header statements of conditional blocks are said to be in *boolean contexts*: their truth values matter to control flow, but otherwise their values are not assigned or returned. Python includes several false values, including 0, `None`, and the *boolean* value `False`. All other numbers are true values. In Chapter 2, we will see that every built-in kind of data in Python has both true and false values.

Boolean values. Python has two boolean values, called `True` and `False`. Boolean values represent truth values in logical expressions. The built-in comparison operations, `>`, `<`, `>=`, `<=`, `==`, `!=`, return these values.

```
>>> 4 < 2
False
>>> 5 >= 5
True
```

This second example reads “5 is greater than or equal to 5”, and corresponds to the function `ge` in the `operator` module.

```
>>> 0 == -0
True
```

This final example reads “0 equals -0”, and corresponds to `eq` in the `operator` module. Notice that Python distinguishes assignment (`=`) from equality comparison (`==`), a convention shared across many programming languages.

Boolean operators. Three basic logical operators are also built into Python:

```
>>> True and False
False
>>> True or False
True
>>> not False
True
```

Logical expressions have corresponding evaluation procedures. These procedures exploit the fact that the truth value of a logical expression can sometimes be determined without evaluating all of its subexpressions, a feature called *short-circuiting*.

To evaluate the expression `<left> and <right>`:

1. Evaluate the subexpression `<left>`.
2. If the result is a false value `v`, then the expression evaluates to `v`.
3. Otherwise, the expression evaluates to the value of the subexpression `<right>`.

To evaluate the expression `<left> or <right>`:

1. Evaluate the subexpression `<left>`.
2. If the result is a true value `v`, then the expression evaluates to `v`.
3. Otherwise, the expression evaluates to the value of the subexpression `<right>`.

To evaluate the expression `not <exp>`:

1. Evaluate `<exp>`; the value is `True` if the result is a false value, and `False` otherwise.

These values, rules, and operators provide us with a way to combine the results of comparisons. Functions that perform comparisons and return boolean values typically begin with `is`, not followed by an underscore (e.g., `isfinite`, `isdigit`, `isinstance`, etc.).

Iteration

In addition to selecting which statements to execute, control statements are used to express repetition. If each line of code we wrote were only executed once, programming would be a very unproductive exercise. Only through repeated execution of statements do we unlock the full potential of computers. We have already seen one form of repetition: a function can be applied many times, although it is only defined once. Iterative control structures are another mechanism for executing the same statements many times.

Consider the sequence of Fibonacci numbers, in which each number is the sum of the preceding two:

```
0, 1, 1, 2, 3, 5, 8, 13, 21, ...
```

Each value is constructed by repeatedly applying the sum-previous-two rule. The zeroth and first are fixed to 0 and 1, so that 1 is the second item. For instance, the eighth Fibonacci number is 21.

We can use a `while` statement to compute the `n`th Fibonacci number. We need to track which value we've reached (`k`), along with the `k`th value (`curr`) and its predecessor (`pred`). Step through this function and observe how the Fibonacci numbers evolve one by one, bound to `curr`.

```
env_diagram(r'''
def fib(n):
    """Compute the nth Fibonacci number, for n >= 1."""
    pred, curr = 0, 1 # Fibonacci numbers 0 and 1
    k = 1 # Which Fib number is curr?
    while k < n:
        pred, curr = curr, pred + curr
        k = k + 1
    return curr

result = fib(8)
''' , step=1, cumulative=False, labels=False)
```

Loading...

Remember that commas separate multiple names and values in an assignment statement. The line:

```
pred, curr = curr, pred + curr
```

has the effect of rebinding the name `pred` to the value of `curr`, and simultaneously rebinding `curr` to the value of `pred + curr`. All of the expressions to the right of `=` are evaluated before any rebinding takes place.

This order of events -- evaluating everything on the right of `=` before updating any bindings on the left -- is essential for correctness of this function.

A `while` clause contains a header expression followed by a suite:

```
while <expression>:
    <suite>
```

To execute a `while` clause:

1. Evaluate the header's expression.
2. If it is a true value, execute the suite, then return to step 1.

In step 2, the entire suite of the `while` clause is executed before the header expression is evaluated again.

In order to prevent the suite of a `while` clause from being executed indefinitely, the suite should always change some binding in each pass.

A `while` statement that does not terminate is called an infinite loop. Press `<Control>-C` to force Python to stop looping.

Testing

Testing a function is the act of verifying that the function's behavior matches expectations. Our language of functions is now sufficiently complex that we need to start testing our implementations.

A *test* is a mechanism for systematically performing this verification. Tests typically take the form of another function that contains one or more sample calls to the function being tested. The returned value is then verified against an expected result. Unlike most functions, which are meant to be general, tests involve selecting and validating calls with specific argument values. Tests also serve as documentation: they demonstrate how to call a function and what argument values are appropriate.

Assertions

Programmers use `assert` statements to verify expectations, such as the output of a function being tested. An `assert` statement has an expression in a boolean context, followed by a quoted line of text (single or double quotes are both fine, but be consistent) that will be displayed if the expression evaluates to a false value.

```
>>> assert fib(8) == 21, 'The 8th Fibonacci number should be 21'
```

When the expression being asserted evaluates to a true value, executing an `assert` statement has no effect. When it is a false value, `assert` causes an error that halts execution.

A test function for `fib` should test several arguments, including extreme values of `n`.

```
>>> def fib_test():
    assert fib(1) == 1, 'The 1st Fibonacci number should be 1'
    assert fib(3) == 2, 'The 3rd Fibonacci number should be 2'
    assert fib(50) == 12586269025, 'Error at the 50th Fibonacci number'
```

When writing Python in files, rather than directly into the interpreter, tests are typically written in the same file or a neighboring file with the suffix `_test.py`.

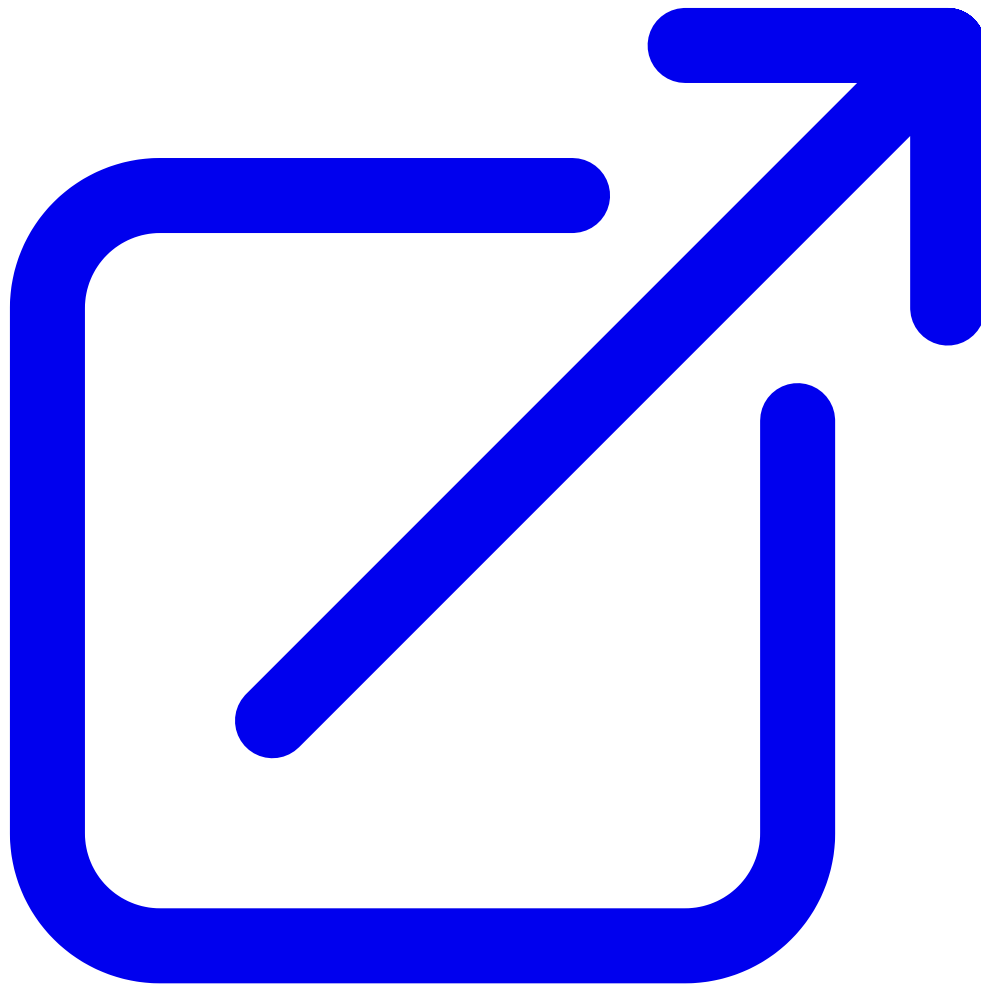
Doctests

Python provides a convenient method for placing simple tests directly in the docstring of a function. The first line of a docstring should contain a one-line description of the function, followed by a blank line. A detailed description of arguments and behavior may follow. In addition, the docstring may include a sample interactive session that calls the function:

```
>>> def sum_naturals(n):
    """Return the sum of the first n natural numbers.

    >>> sum_naturals(10)
    55
    >>> sum_naturals(100)
    5050
    """
    total, k = 0, 1
    while k <= n:
        total = total + k
        k = k + 1
    return total
```

Then, the interaction can be verified via the [doctest module](#)



```
>>> from doctest import testmod
>>> testmod()
TestResults(failed=0, attempted=2)
```

To verify the doctest interactions for only a single function, we use a `doctest` function called `run_docstring_examples`. This function is (unfortunately) a bit complicated to call. Its first argument is the function to test. The second should always be the result of the expression `globals()`, a built-in function that returns the global environment. The third argument is `True` to indicate that we would like “verbose” output: a catalog of all tests run.

```
>>> from doctest import run_docstring_examples
>>> run_docstring_examples(sum_naturals, globals(), True)
Finding tests in NoName
Trying:
    sum_naturals(10)
Expecting:
    55
ok
Trying:
    sum_naturals(100)
Expecting:
    5050
ok
```

When the return value of a function does not match the expected result, the `run_docstring_examples` function will report this problem as a test failure.

When writing Python in files, all doctests in a file can be run by starting Python with the doctest command line option:

```
python3 -m doctest <python_source_file>
```

The key to effective testing is to write (and run) tests immediately after implementing new functions. It is even good practice to write some tests before you implement, in order to have some example inputs and outputs in your mind. A test that applies a single function is called a *unit test*. Exhaustive unit testing is a hallmark of good program design.



[Chapter 1: Functions](#)

[1.4 Designing Functions](#)

[Chapter 1: Functions](#)

[1.6 Higher-Order Functions](#)



1.6 Higher-Order Functions



1.6 Higher-Order Functions

```
from pytutor import env_diagram
```

We have seen that functions describe compound operations independent of the particular values of their arguments. That is, in `square`,

```
>>> def square(x):  
    return x * x
```

we are not talking about the square of a particular number, but rather about a method for obtaining the square of any number. Of course, we could get along without ever defining this function, by always writing expressions such as

```
>>> 3 * 3  
9  
>>> 5 * 5  
25
```

and never mentioning `square` explicitly. This practice would suffice for simple computations such as `square`, but would become arduous for more complex examples such as `abs` or `fib`. In general, lacking function definition would put us at the disadvantage of forcing us to work always at the level of the particular operations that happen to be primitives in the language (multiplication, in this case) rather than in terms of functional abstractions. Our programs would be able to compute squares, but our language would lack the ability to express the concept of squaring.

One of the things we should demand from a powerful programming language is the ability to build abstractions by assigning names to common patterns and then to work in terms of the names directly. Functions provide this ability. As we will see in the following examples, there are common programming patterns that recur in code, but are used with a number of different functions. These patterns can also be abstracted, by giving them names.

To express certain general patterns as named concepts, we will need to construct functions that can accept other functions as arguments or return functions as values. Functions that manipulate functions are called higher-order functions. This section shows how higher-order functions can serve as powerful abstraction mechanisms, vastly increasing the expressive power of our language.

Functions as Arguments

Consider the following three functions, which all compute summations. The first, `sum_naturals`, computes the sum of natural numbers up to `n`:

```
>>> def sum_naturals(n):
    total, k = 0, 1
    while k <= n:
        total = total + k
        k = k + 1
    return total

>>> sum_naturals(100)
5050
```

The second, `sum_cubes`, computes the sum of the cubes of natural numbers up to `n`.

```
>>> def sum_cubes(n):
    total, k = 0, 1
    while k <= n:
        total = total + k * k * k
        k = k + 1
    return total

>>> sum_cubes(100)
25502500
```

The third, `pi_sum`, computes the sum of terms in the series

$$\frac{8}{1 \cdot 3} + \frac{8}{5 \cdot 7} + \frac{8}{9 \cdot 11} + \dots$$

$1 \cdot 38 + 5 \cdot 78 + 9 \cdot 118 + \dots$

(1)

which converges to π (very slowly).

```
>>> def pi_sum(n):
    total, k = 0, 1
    while k <= n:
        total = total + 8 / ((4*k-3) * (4*k-1))
        k = k + 1
    return total

>>> pi_sum(100)
3.1365926848388144
```

These three functions clearly share a common underlying pattern. They are for the most part identical, differing only in name and the expression involving `k` used to compute the term to be added. We could generate each of the functions by filling in slots in the same template:

```
def <name>(n):
    total, k = 0, 1
    while k <= n:
        total = total + <term>(k)
        k = k + 1
    return total
```

The presence of such a common pattern is strong evidence that there is a useful abstraction waiting to be brought to the surface. Each of these functions is a summation of terms. As program designers, we would like our language to be powerful enough so that we can write a function that expresses the concept of summation itself rather than only functions that compute particular sums. We can do so readily in Python by taking the common template shown above and transforming the “slots” into formal parameters:

In the example below, `summation` takes as its two arguments the upper bound `n` together with the function `term` that computes the k th term. We can use `summation` just as we would any function, and it expresses summations succinctly. Take the time to step through this example, and notice how binding `cube` to the local name `term` ensures that the result $1*1*1 + 2*2*2 + 3*3*3 = 36$ is computed correctly. In this example, frames which are no longer needed are removed to save space.

```

env_diagram(r'''
def summation(n, term):
    total, k = 0, 1
    while k <= n:
        total = total + term(k)
        k = k + 1
    return total

def cube(x):
    return x*x*x

def sum_cubes(n):
    return summation(n, cube)

result = sum_cubes(3)
''' , step=3, cumulative=False, labels=False)

```

Loading...

Using an `identity` function that returns its argument, we can also sum natural numbers using exactly the same `summation` function.

```

>>> def summation(n, term):
    total, k = 0, 1
    while k <= n:
        total = total + term(k)
        k = k + 1
    return total

>>> def identity(x):
    return x

>>> def sum_naturals(n):
    return summation(n, identity)

>>> sum_naturals(10)
55

```

The `summation` function can also be called directly, without defining another function for a specific sequence.

```

>>> summation(10, square)
385

```

We can define `pi_sum` using our `summation` abstraction by defining a function `pi_term` to compute each term. We pass the argument `1e6`, a shorthand for `1 * 106 = 1000000`, to generate a close approximation to pi.

```

>>> def pi_term(x):
    return 8 / ((4*x-3) * (4*x-1))

>>> def pi_sum(n):
    return summation(n, pi_term)

>>> pi_sum(1e6)
3.141592153589902

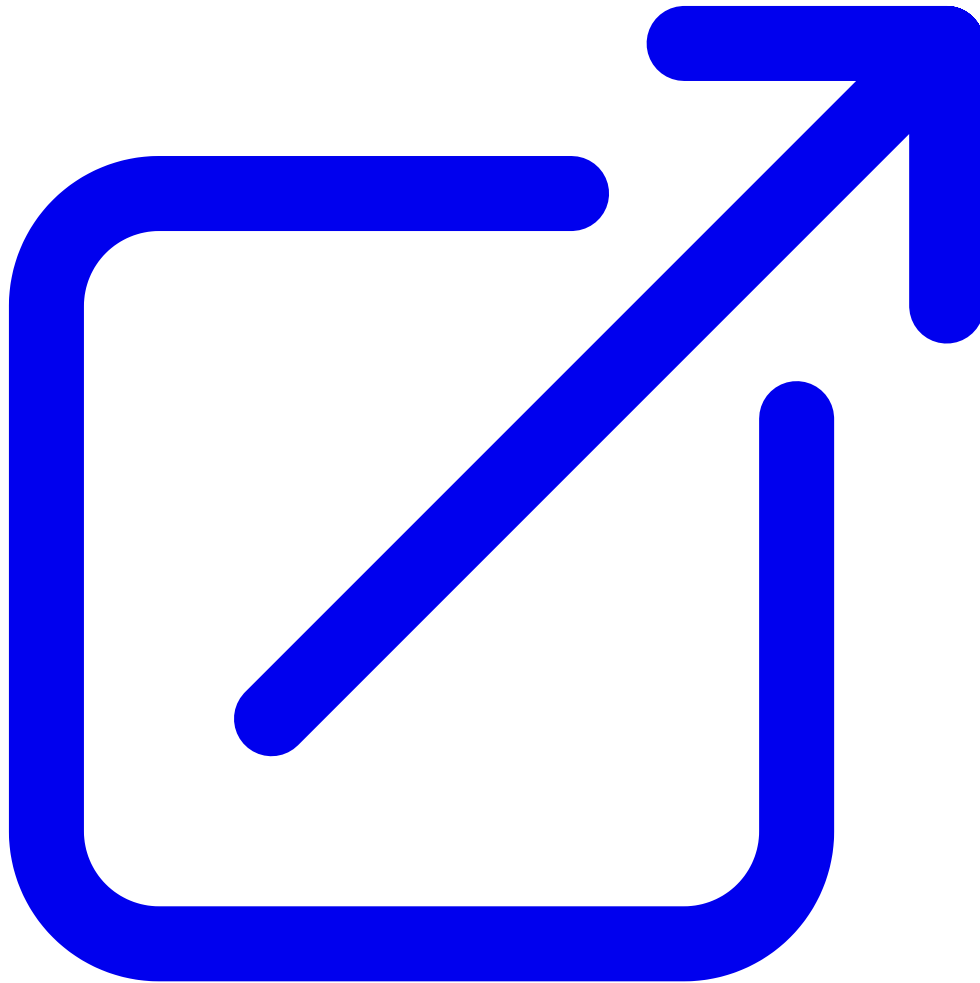
```

Functions as General Methods

We introduced user-defined functions as a mechanism for abstracting patterns of numerical operations so as to make them independent of the particular numbers involved. With higher-order functions, we begin to see a more powerful kind of abstraction: some functions express general methods of computation, independent of the particular functions they call.

Despite this conceptual extension of what a function means, our environment model of how to evaluate a call expression extends gracefully to the case of higher-order functions, without change. When a user-defined function is applied to some arguments, the formal parameters are bound to the values of those arguments (which may be functions) in a new local frame.

Consider the following example, which implements a general method for iterative improvement and uses it to compute the [golden ratio](#)



. The golden ratio, often called “phi” or ϕ , is a number near 1.6 that appears frequently in nature, art, and architecture. It’s the number that’s exactly one more than its reciprocal: $\phi = 1 + \frac{1}{\phi} = 1 + \phi^{-1}$.

An iterative improvement algorithm begins with a `guess` of a solution to an equation. It repeatedly applies an `update` function to improve that guess, and applies a `close` comparison to check whether the current `guess` is “close enough” to be considered correct.

```
>>> def improve(update, close, guess=1):
    while not close(guess):
        guess = update(guess)
    return guess
```

This `improve` function is a general expression of repetitive refinement. It doesn’t specify what problem is being solved: those details are left to the `update` and `close` functions passed in as arguments.

Among the well-known properties of the golden ratio are that it can be computed by repeatedly summing the inverse of any positive number with 1, and that it is one less than its square. We can express these properties as functions to be used with `improve`.

```
>>> def golden_update(guess):
    return 1/guess + 1

>>> def square_close_to_successor(guess):
    return approx_eq(guess * guess, guess + 1)
```

Above, we introduce a call to `approx_eq` that is meant to return `True` if its arguments are approximately equal to each other. To implement `approx_eq`, we can compare the absolute value of the difference between two numbers to a small tolerance value.

```
>>> def approx_eq(x, y, tolerance=1e-15):
    return abs(x - y) < tolerance
```

Calling `improve` with the arguments `golden_update` and `square_close_to_successor` will compute a finite approximation to the golden ratio.

```
>>> improve(golden_update, square_close_to_successor)
1.6180339887498951
```

By tracing through the steps of evaluation, we can see how this result is computed. First, a local frame for `improve` is constructed with bindings for `update`, `close`, and `guess`. In the body of `improve`, the name `close` is bound to `square_close_to_successor`, which is called on the initial value of `guess`. Trace through the rest of the steps to see the computational process that evolves to compute the golden ratio.

```
env_diagram(r'''
def improve(update, close, guess=1):
    while not close(guess):
        guess = update(guess)
    return guess

def golden_update(guess):
    return 1/guess + 1

def square_close_to_successor(guess):
    return approx_eq(guess * guess,
                      guess + 1)

def approx_eq(x, y, tolerance=1e-3):
    return abs(x - y) < tolerance

phi = improve(golden_update,
              square_close_to_successor)
''', step=6, cumulative=False, labels=False)
```

Loading...

This example illustrates two related big ideas in computer science. First, naming and functions allow us to abstract away a vast amount of complexity. While each function definition has been trivial, the computational process set in motion by our evaluation procedure is quite intricate. Second, it is only by virtue of the fact that we have an extremely general evaluation procedure for the Python language that small components can be composed into complex processes. Understanding the procedure of interpreting programs allows us to validate and inspect the process we have created.

As always, our new general method `improve` needs a test to check its correctness. The golden ratio can provide such a test, because it also has an exact closed-form solution, which we can compare to this iterative result.

```
>>> from math import sqrt
>>> phi = 1/2 + sqrt(5)/2
>>> def improve_test():
    approx_phi = improve(golden_update, square_close_to_successor)
    assert approx_eq(phi, approx_phi), 'phi differs from its approximation'

>>> improve_test()
```

For this test, no news is good news: `improve_test` returns `None` after its `assert` statement is executed successfully.

Defining Functions III: Nested Definitions

The above examples demonstrate how the ability to pass functions as arguments significantly enhances the expressive power of our programming language. Each general concept or equation maps onto its own short function. One negative consequence of this approach is that the global frame becomes cluttered with names of small functions, which must all be unique. Another problem is that we are constrained by particular function signatures: the `update` argument to `improve` must take exactly one argument. Nested function definitions address both of these problems, but require us to enrich our environment model.

Let's consider a new problem: computing the square root of a number. In programming languages, "square root" is often abbreviated as `sqrt`. Repeated application of the following update converges to the square root of `a`:

```
>>> def average(x, y):
    return (x + y)/2

>>> def sqrt_update(x, a):
    return average(x, a/x)
```

This two-argument update function is incompatible with `improve` (it takes two arguments, not one), and it provides only a single update, while we really care about taking square roots by repeated updates. The solution to both of these issues is to place function definitions inside the body of other definitions.

```
>>> def sqrt(a):
    def sqrt_update(x):
        return average(x, a/x)
    def sqrt_close(x):
        return approx_eq(x * x, a)
    return improve(sqrt_update, sqrt_close)
```

Like local assignment, local `def` statements only affect the current local frame. These functions are only in scope while `sqrt` is being evaluated. Consistent with our evaluation procedure, these local `def` statements don't even get evaluated until `sqrt` is called.

Lexical Scope

Locally defined functions also have access to the name bindings in the scope in which they are defined. In this example, `sqrt_update` refers to the name `a`, which is a formal parameter of its enclosing function `sqrt`. This discipline of sharing names among nested definitions is called *lexical scoping*. Critically, the inner functions have access to the names in the environment where they are defined (not where they are called).

We require two extensions to our environment model to enable lexical scoping.

1. Each user-defined function has a parent environment: the environment in which it was defined.
2. When a user-defined function is called, its local frame's parent is that parent environment.

Previous to `sqrt`, all functions were defined in the global environment, and so they all had the same parent: global. By contrast, when Python evaluates the first two clauses of `sqrt`, it creates functions that are associated with a local environment. In the call

```
>>> sqrt(256)
16.0
```

the environment first adds a local frame for `sqrt` and evaluates the `def` statements for `sqrt_update` and `sqrt_close`.

```
env_diagram(r'''
def average(x, y):
    return (x + y)/2

def improve(update, close, guess=1):
    while not close(guess):
        guess = update(guess)
    return guess

def approx_eq(x, y, tolerance=1e-3):
    return abs(x - y) < tolerance

def sqrt(a):
    def sqrt_update(x):
        return average(x, a/x)
    def sqrt_close(x):
        return approx_eq(x * x, a)
    return improve(sqrt_update, sqrt_close)

result = sqrt(256)
''' , step=7, cumulative=False)
```

Loading...

Function values each have a new annotation that we will include in environment diagrams from now on, a *parent*. The parent of a function value is the first frame of the environment in which that function was defined. Functions without parent annotations were defined in the global environment. When a user-defined function is called, the frame created has the same parent as that function.

Subsequently, the name `sqrt_update` resolves to this newly defined function, which is passed as an argument to `improve`. Within the body of `improve`, we must apply our `update` function (bound to `sqrt_update`) to the initial guess `x` of 1. This final application creates an environment for `sqrt_update` that begins with a local frame containing only `x`, but with the parent frame `sqrt` still containing a binding for `a`.

```

env_diagram(r'''
def average(x, y):
    return (x + y)/2

def improve(update, close, guess=1):
    while not close(guess):
        guess = update(guess)
    return guess

def approx_eq(x, y, tolerance=1e-3):
    return abs(x - y) < tolerance

def sqrt(a):
    def sqrt_update(x):
        return average(x, a/x)
    def sqrt_close(x):
        return approx_eq(x * x, a)
    return improve(sqrt_update, sqrt_close)

result = sqrt(256)
''' , step=14, cumulative=False)

```

Loading...

The most critical part of this evaluation procedure is the transfer of the parent for `sqrt_update` to the frame created by calling `sqrt_update`. This frame is also annotated with `[parent=f1]`.

Extended environments. An environment can consist of an arbitrarily long chain of frames, which always concludes with the global frame. Previous to this `sqrt` example, environments had at most two frames: a local frame and the global frame. By calling functions that were defined within other functions, via nested `def` statements, we can create longer chains. The environment for this call to `sqrt_update` consists of three frames: the local `sqrt_update` frame, the `sqrt` frame in which `sqrt_update` was defined (labeled `f1`), and the global frame.

The return expression in the body of `sqrt_update` can resolve a value for `a` by following this chain of frames. Looking up a name finds the first value bound to that name in the current environment. Python checks first in the `sqrt_update` frame -- no `a` exists. Python checks next in the parent frame, `f1`, and finds a binding for `a` to 256.

Hence, we realize two key advantages of lexical scoping in Python.

- The names of a local function do not interfere with names external to the function in which it is defined, because the local function name will be bound in the current local environment in which it was defined, rather than the global environment.
- A local function can access the environment of the enclosing function, because the body of the local function is evaluated in an environment that extends the evaluation environment in which it was defined.

The `sqrt_update` function carries with it some data: the value for `a` referenced in the environment in which it was defined. Because they “enclose” information in this way, locally defined functions are often called *closures*.

Functions as Returned Values

We can achieve even more expressive power in our programs by creating functions whose returned values are themselves functions. An important feature of lexically scoped programming languages is that locally defined functions maintain their parent environment when they are returned. The following example illustrates the utility of this feature.

Once many simple functions are defined, function *composition* is a natural method of combination to include in our programming language. That is, given two functions $f(x)$ and $g(x)$, we might want to define $h(x) = f(g(x))$. We can define function composition using our existing tools:

```
>>> def compose1(f, g):
    def h(x):
        return f(g(x))
    return h
```

The environment diagram for this example shows how the names `f` and `g` are resolved correctly, even in the presence of conflicting names.

```
env_diagram(r'''
def square(x):
    return x * x

def successor(x):
    return x + 1

def compose1(f, g):
    def h(x):
        return f(g(x))
    return h

def f(x):
    """Never called."""
    return -x

square_successor = compose1(square, successor)
result = square_successor(12)
''', cumulative=True)
```

Loading...

The 1 in `compose1` is meant to signify that the composed functions all take a single argument. This naming convention is not enforced by the interpreter; the 1 is just part of the function name.

At this point, we begin to observe the benefits of our effort to define precisely the environment model of computation. No modification to our environment model is required to explain our ability to return functions in this way.

Example: Newton's Method¹

This extended example shows how function return values and local definitions can work together to express general ideas concisely. We will implement an algorithm that is used broadly in machine learning, scientific computing, hardware design, and optimization.

Newton's method is a classic iterative approach to finding the arguments of a mathematical function that yield a return value of 0. These values are called the *zeros* of the function. Finding a zero of a function is often equivalent to solving some other problem of interest, such as computing a square root.

A motivating comment before we proceed: it is easy to take for granted the fact that we know how to compute square roots. Not just Python, but your phone, web browser, or pocket calculator can do so for you. However, part of learning computer science is understanding how quantities like these can be computed, and the general approach presented here is applicable to solving a large class of equations beyond those built into Python.

Newton's method is an iterative improvement algorithm: it improves a guess of the zero for any function that is *differentiable*, which means that it can be approximated by a straight line at any point. Newton's method follows these linear approximations to find function zeros.

Imagine a line through the point $(x, f(x))$ that has the same slope as the curve for function $f(x)$ at that point. Such a line is called the *tangent*, and its slope is called the *derivative* of f at x .

This line's slope is the ratio of the change in function value to the change in function argument. Hence, translating x by $f(x)$ divided by the slope will give the argument value at which this tangent line touches 0.



A `newton_update` expresses the computational process of following this tangent line to 0, for a function `f` and its derivative `df`.

```
>>> def newton_update(f, df):
    def update(x):
        return x - f(x) / df(x)
    return update
```

Finally, we can define the `find_zero` function in terms of `newton_update`, our `improve` algorithm, and a comparison to see if $f(x)$ is near 0.

```
>>> def find_zero(f, df):
    def near_zero(x):
        return approx_eq(f(x), 0)
    return improve(newton_update(f, df), near_zero)
```

Computing Roots

Using Newton's method, we can compute roots of arbitrary degree n . The degree n root of a is x such that $x \cdot x \cdot \dots \cdot x = a$ with x repeated n times. For example,

- The square (second) root of 64 is 8, because $8 \cdot 8 = 64$.
- The cube (third) root of 64 is 4, because $4 \cdot 4 \cdot 4 = 64$.

- The sixth root of 64 is 2, because $2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 64$.

We can compute roots using Newton's method with the following observations:

- The square root of 64 (written $\sqrt{64}$)

$\sqrt{\quad}$

) is the value x such that $x^2 - 64 = 0$

- More generally, the degree n root of a (written $\sqrt[n]{a}$)

$\sqrt{\quad}$

) is the value x such that $x^n - a = 0$

If we can find a zero of this last equation, then we can compute degree n roots. By plotting the curves for n equal to 2, 3, and 6 and a equal to 64, we can visualize this relationship.



We first implement `square_root_newton` by defining `f` and its derivative `df`. We use from calculus the fact that the derivative of $f(x) = x^2 - a$ is $f'(x) = 2x$.

```
>>> def square_root_newton(a):
    def f(x):
        return x * x - a
    def df(x):
        return 2 * x
    return find_zero(f, df)

>>> square_root_newton(64)
8.0
```

Generalizing to roots of arbitrary degree n , we compute $f(x) = x^n - a$ and its derivative $df(x) = n \cdot x^{n-1}$.

```
>>> def power(x, n):
    """Return x * x * x * ... * x for x repeated n times."""
    product, k = 1, 0
    while k < n:
        product, k = product * x, k + 1
    return product

>>> def nth_root_of_a(n, a):
    def f(x):
        return power(x, n) - a
    def df(x):
        return n * power(x, n-1)
    return find_zero(f, df)

>>> nth_root_of_a(2, 64)
8.0
>>> nth_root_of_a(3, 64)
4.0
>>> nth_root_of_a(6, 64)
2.0
```

The approximation error in all of these computations can be reduced by changing the `tolerance` in `approx_eq` to a smaller number.

As you experiment with Newton's method, be aware that it will not always converge. The initial guess of `improve` must be sufficiently close to the zero, and various conditions about the function must be met. Despite this shortcoming, Newton's method is a powerful general computational method for solving differentiable equations. Very fast algorithms for logarithms and large integer division employ variants of the technique in modern computers.

Currying

We can use higher-order functions to convert a function that takes multiple arguments into a chain of functions that each take a single argument. More specifically, given a function `f(x, y)`, we can define a function `g` such that `g(x)(y)` is equivalent to `f(x, y)`. Here, `g` is a higher-order function that takes in a single argument `x` and returns another function that takes in a single argument `y`. This transformation is called *currying*.

As an example, we can define a curried version of the `pow` function:

```
>>> def curried_pow(x):
    def h(y):
        return pow(x, y)
    return h

>>> curried_pow(2)(3)
8
```

Some programming languages, such as Haskell, only allow functions that take a single argument, so the programmer must curry all multi-argument procedures. In more general languages such as Python, currying is useful when we require a function that takes in only a single argument. For example, the *map* pattern applies a single-argument function to a sequence of values. In later chapters, we will see more general examples of the map pattern, but for now, we can implement the pattern in a function:

```
>>> def map_to_range(start, end, f):
    while start < end:
        print(f(start))
        start = start + 1
```

We can use `map_to_range` and `curried_pow` to compute the first ten powers of two, rather than specifically writing a function to do so:

```
>>> map_to_range(0, 10, curried_pow(2))
1
2
4
8
16
32
64
128
256
512
```

We can similarly use the same two functions to compute powers of other numbers. Currying allows us to do so without writing a specific function for each number whose powers we wish to compute.

In the above examples, we manually performed the currying transformation on the `pow` function to obtain `curried_pow`. Instead, we can define functions to automate currying, as well as the inverse *uncurrying* transformation:

```
>>> def curry2(f):
    """Return a curried version of the given two-argument function."""
    def g(x):
        def h(y):
            return f(x, y)
        return h
    return g

>>> def uncurry2(g):
    """Return a two-argument version of the given curried function."""
    def f(x, y):
        return g(x)(y)
    return f

>>> pow_curried = curry2(pow)
>>> pow_curried(2)(5)
32

>>> map_to_range(0, 10, pow_curried(2))
1
2
4
8
16
32
64
128
256
512
```

The `curry2` function takes in a two-argument function `f` and returns a single-argument function `g`. When `g` is applied to an argument `x`, it returns a single-argument function `h`. When `h` is applied to `y`, it calls `f(x, y)`. Thus, `curry2(f)(x)(y)` is equivalent to `f(x, y)`. The `uncurry2` function reverses the currying transformation, so that `uncurry2(curry2(f))` is equivalent to `f`.

```
>>> uncurry2(pow_curried)(2, 5)
32
```

Lambda Expressions¹

So far, each time we have wanted to define a new function, we needed to give it a name. But for other types of expressions, we don't need to associate intermediate values with a name. That is, we can compute `a*b + c*d` without having to name the subexpressions `a*b` or `c*d`, or the full expression. In Python, we can create function values on the fly using `lambda` expressions, which evaluate to unnamed functions. A `lambda` expression evaluates to a function that has a single return expression as its body. Assignment and control statements are not allowed.

```
>>> def compose1(f, g):
    return lambda x: f(g(x))
```

We can understand the structure of a `lambda` expression by constructing a corresponding English sentence:

<code>lambda</code>	<code>x</code>	:	<code>f(g(x))</code>
"A function that	takes <code>x</code>	and returns	<code>f(g(x))</code> "

The result of a `lambda` expression is called a `lambda` function. It has no intrinsic name (and so Python prints `<lambda>` for the name), but otherwise it behaves like any other function.

```
>>> s = lambda x: x * x
>>> s
<function <lambda> at 0xf3f490>
>>> s(12)
144
```

In an environment diagram, the result of a `lambda` expression is a function as well, named with the Greek letter λ (`lambda`). Our `compose` example can be expressed quite compactly with `lambda` expressions.

```
env_diagram(r'''
def compose1(f, g):
    return lambda x: f(g(x))

f = compose1(lambda x: x * x,
              lambda y: y + 1)
result = f(12)
''', cumulative=True)
```

Loading...

Some programmers find using unnamed functions from lambda expressions to be shorter and more direct. However, compound `lambda` expressions are notoriously illegible, despite their brevity. The following definition is correct, but many programmers have trouble understanding it quickly.

```
>>> compose1 = lambda f,g: lambda x: f(g(x))
```

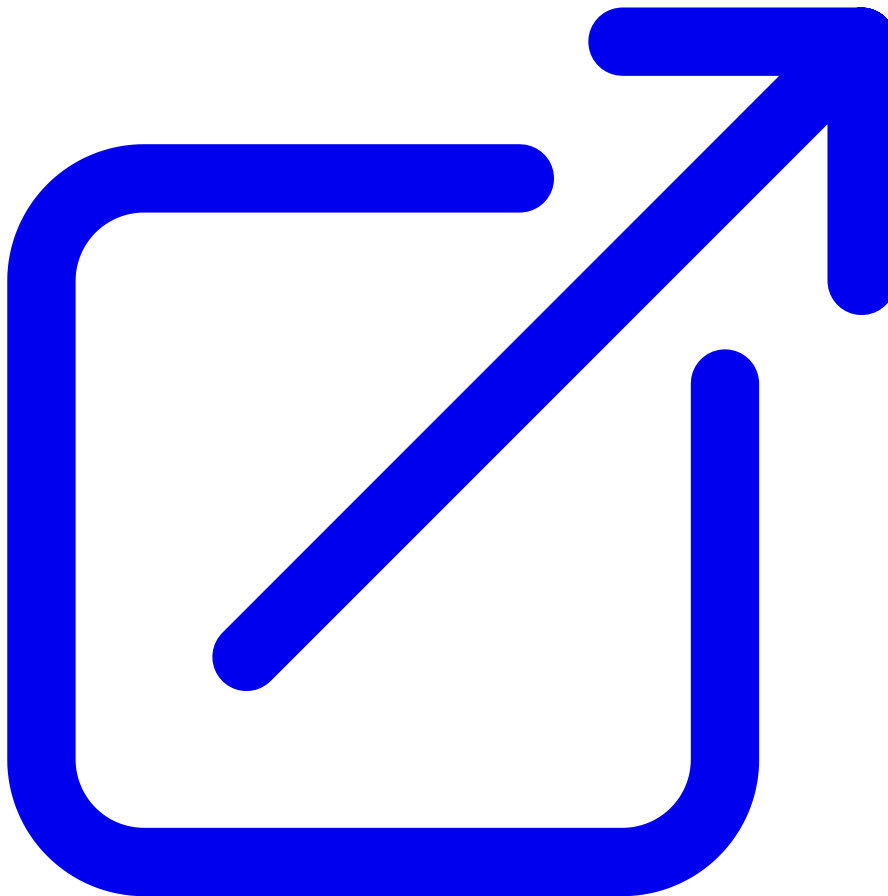
In general, Python style prefers explicit `def` statements to lambda expressions, but allows them in cases where a simple function is needed as an argument or return value.

Such stylistic rules are merely guidelines; you can program any way you wish. However, as you write programs, think about the audience of people who might read your program one day. When you can make your program easier to understand, you do those people a favor.

The term *lambda* is a historical accident resulting from the incompatibility of written mathematical notation and the constraints of early type-setting systems.

It may seem perverse to use lambda to introduce a procedure/function. The notation goes back to Alonzo Church, who in the 1930's started with a "hat" symbol; he wrote the square function as " $\hat{y} . y \times y$ ". But frustrated typographers moved the hat to the left of the parameter and changed it to a capital lambda: " $\Lambda y . y \times y$ "; from there the capital lambda was changed to lowercase, and now we see " $\lambda y . y \times y$ " in math books and `(lambda (y) (* y y))` in Lisp.

Peter Norvig (norvig.com/lispy2.html)



)

Despite their unusual etymology, `lambda` expressions and the corresponding formal language for function application, the *lambda calculus*, are fundamental computer science concepts shared far beyond the Python programming community. We will revisit this topic when we study the design of interpreters in Chapter 3.

Abstractions and First-Class Functions [1](#)

We began this section with the observation that user-defined functions are a crucial abstraction mechanism, because they permit us to express general methods of computing as explicit elements in our programming language. Now we've seen how higher-order functions permit us to manipulate these general methods to create further abstractions.

As programmers, we should be alert to opportunities to identify the underlying abstractions in our programs, build upon them, and generalize them to create more powerful abstractions. This is not to say that one should always write programs in the most abstract way possible; expert programmers know how to choose the level of abstraction appropriate to their task. But it is important to be able to think in terms of these abstractions, so that we can be ready to apply them in new contexts. The significance of higher-order functions is that they enable us to represent these abstractions explicitly as elements in our programming language, so that they can be handled just like other computational elements.

In general, programming languages impose restrictions on the ways in which computational elements can be manipulated. Elements with the fewest restrictions are said to have first-class status. Some of the "rights and privileges" of first-class elements are:

1. They may be bound to names.
2. They may be passed as arguments to functions.
3. They may be returned as the results of functions.
4. They may be included in data structures.

Python awards functions full first-class status, and the resulting gain in expressive power is enormous.

Function Decorators¹

Python provides special syntax to apply higher-order functions as part of executing a `def` statement, called a decorator. Perhaps the most common example is a trace.

```
>>> def trace(fn):
    def wrapped(x):
        print('-> ', fn, '(', x, ')')
        return fn(x)
    return wrapped

>>> @trace
def triple(x):
    return 3 * x

>>> triple(12)
-> <function triple at 0x102a39848> ( 12 )
36
```

In this example, a higher-order function `trace` is defined, which returns a function that precedes a call to its argument with a `print` statement that outputs the argument. The `def` statement for `triple` has an annotation, `@trace`, which affects the execution rule for `def`. As usual, the function `triple` is created. However, the name `triple` is not bound to this function. Instead, the name `triple` is bound to the returned function value of calling `trace` on the newly defined `triple` function. In code, this decorator is equivalent to:

```
>>> def triple(x):
    return 3 * x

>>> triple = trace(triple)
```

In the projects associated with this text, decorators are used for tracing, as well as selecting which functions to call when a program is run from the command line.

Extra for experts. The decorator symbol `@` may also be followed by a call expression. The expression following `@` is evaluated first (just as the name `trace` was evaluated above), the `def` statement second, and finally the result of evaluating the decorator expression is applied to the newly defined function, and the result is bound to the name in the `def` statement.



[Chapter 1: Functions](#)

[1.5 Control](#)

[Chapter 1: Functions](#)

[1.7 Recursive Functions](#)



1.7 Recursive Functions



1.7 Recursive Functions

```
from pytutor import env_diagram
```

A function is called *recursive* if the body of the function calls the function itself, either directly or indirectly. That is, the process of executing the body of a recursive function may in turn require applying that function again. Recursive functions do not use any special syntax in Python, but they do require some effort to understand and create.

We'll begin with an example problem: write a function that sums the digits of a natural number. When designing recursive functions, we look for ways in which a problem can be broken down into simpler problems. In this case, the operators `%` and `//` can be used to separate a number into two parts: its last digit and all but its last digit.

```
>>> 18117 % 10
7
>>> 18117 // 10
1811
```

The sum of the digits of 18117 is $1+8+1+1+7 = 18$. Just as we can separate the number, we can separate this sum into the last digit, 7, and the sum of all but the last digit, $1+8+1+1 = 11$. This separation gives us an algorithm: to sum the digits of a number `n`, add its last digit `n % 10` to the sum of the digits of `n // 10`. There's one special case: if a number has only one digit, then the sum of its digits is itself. This algorithm can be implemented as a recursive function.

```
>>> def sum_digits(n):
    """Return the sum of the digits of positive integer n."""
    if n < 10:
        return n
    else:
        all_but_last, last = n // 10, n % 10
        return sum_digits(all_but_last) + last
```

This definition of `sum_digits` is both complete and correct, even though the `sum_digits` function is called within its own body. The problem of summing the digits of a number is broken down into two steps: summing all but the last digit, then adding the last digit. Both of these steps are simpler than the original problem. The function is recursive because the first step is the same kind of problem as the original problem. That is, `sum_digits` is exactly the function we need in order to implement `sum_digits`.

```
>>> sum_digits(9)
9
>>> sum_digits(18117)
18
>>> sum_digits(9437184)
36
>>> sum_digits(11408855402054064613470328848384)
126
```

We can understand precisely how this recursive function applies successfully using our environment model of computation. No new rules are required.

```
env_diagram(r'''
def sum_digits(n):
    if n < 10:
        return n
    else:
        all_but_last, last = n // 10, n % 10
        return sum_digits(all_but_last) + last

sum_digits(738)
''', cumulative=True)
```

Loading...

When the `def` statement is executed, the name `sum_digits` is bound to a new function, but the body of that function is not yet executed. Therefore, the circular nature of `sum_digits` is not a problem yet. Then, `sum_digits` is called on 738:

1. A local frame for `sum_digits` with `n` bound to 738 is created, and the body of `sum_digits` is executed in the environment that starts with that frame.
2. Since 738 is not less than 10, the assignment statement on line 5 is executed, splitting 738 into 73 and 8.
3. In the following return statement, `sum_digits` is called on 73, the value of `all_but_last` in the current environment.
4. Another local frame for `sum_digits` is created, this time with `n` bound to 73. The body of `sum_digits` is again executed in the new environment that starts with this frame.
5. Since 73 is also not less than 10, 73 is split into 7 and 3 and `sum_digits` is called on 7, the value of `all_but_last` evaluated in this frame.
6. A third local frame for `sum_digits` is created, with `n` bound to 7.
7. In the environment starting with this frame, it is true that `n < 10`, and therefore 7 is returned.
8. In the second local frame, this return value 7 is summed with 3, the value of `last`, to return 10.
9. In the first local frame, this return value 10 is summed with 8, the value of `last`, to return 18.

This recursive function applies correctly, despite its circular character, because it is applied twice, but with a different argument each time. Moreover, the second application was a simpler instance of the digit summing problem than the first. Generate the environment diagram for the call `sum_digits(18117)` to see that each successive call to `sum_digits` takes a smaller argument than the last, until eventually a single-digit input is reached.

This example also illustrates how functions with simple bodies can evolve complex computational processes by using recursion.

The Anatomy of Recursive Functions

A common pattern can be found in the body of many recursive functions. The body begins with a *base case*, a conditional statement that defines the behavior of the function for the inputs that are simplest to process. In the case of `sum_digits`, the base case is any single-digit argument, and we simply return that argument. Some recursive functions will have multiple base cases.

The base cases are then followed by one or more *recursive calls*. Recursive calls always have a certain character: they simplify the original problem. Recursive functions express computation by simplifying problems incrementally. For example, summing the digits of 7 is simpler than summing the digits of 73, which in turn is simpler than summing the digits of 738. For each subsequent call, there is less work left to be done.

Recursive functions often solve problems in a different way than the iterative approaches that we have used previously. Consider a function `fact` to compute n factorial, where for example `fact(4)` computes $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$.

A natural implementation using a `while` statement accumulates the total by multiplying together each positive integer up to n .

```
>>> def fact_iter(n):
    total, k = 1, 1
    while k <= n:
        total, k = total * k, k + 1
    return total

>>> fact_iter(4)
24
```

On the other hand, a recursive implementation of factorial can express `fact(n)` in terms of `fact(n-1)`, a simpler problem. The base case of the recursion is the simplest form of the problem: `fact(1)` is 1.

```
env_diagram(n'''
def fact(n):
    if n == 1:
        return 1
    else:
        return n * fact(n-1)

fact(4)
''', cumulative=True)
```

Loading...

These two factorial functions differ conceptually. The iterative function constructs the result from the base case of 1 to the final total by successively multiplying in each term. The recursive function, on the other hand, constructs the result directly from the final term, n , and the result of the simpler problem, `fact(n-1)`.

As the recursion “unwinds” through successive applications of the `fact` function to simpler and simpler problem instances, the result is eventually built starting from the base case. The recursion ends by passing the argument 1 to `fact`; the result of each call depends on the next until the base case is reached.

The correctness of this recursive function is easy to verify from the standard definition of the mathematical function for factorial:

$$\begin{aligned}
 (n-1)! &= (n-1) \cdot (n-2) \cdot \dots \cdot 1 \\
 n! &= n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1 \\
 n! &= n \cdot (n-1)!
 \end{aligned}$$

$$(n-1)!n! = (n-1) \cdot (n-2) \cdot \dots \cdot 1 = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1 = n \cdot (n-1)!$$

(1)

While we can unwind the recursion using our model of computation, it is often clearer to think about recursive calls as functional abstractions. That is, we should not care about how `fact(n-1)` is implemented in the body of `fact`; we should simply trust that it computes the factorial of `n-1`. Treating a recursive call as a functional abstraction has been called a *recursive leap of faith*. We define a function in terms of itself, but simply trust that the simpler cases will work correctly when verifying the correctness of the function. In this example, we trust that `fact(n-1)` will correctly compute `(n-1)!`; we must only check that `n!` is computed correctly if this assumption holds. In this way, verifying the correctness of a recursive function is a form of proof by induction.

The functions `fact_iter` and `fact` also differ because the former must introduce two additional names, `total` and `k`, that are not required in the recursive implementation. In general, iterative functions must maintain some local state that changes throughout the course of computation. At any point in the iteration, that state characterizes the result of completed work and the amount of work remaining. For example, when `k` is 3 and `total` is 2, there are still two terms remaining to be processed, 3 and 4. On the other hand, `fact` is characterized by its single argument `n`. The state of the computation is entirely contained within the structure of the environment, which has return values that take the role of `total`, and binds `n` to different values in different frames rather than explicitly tracking `k`.

Recursive functions leverage the rules of evaluating call expressions to bind names to values, often avoiding the nuisance of correctly assigning local names during iteration. For this reason, recursive functions can be easier to define correctly. However, learning to recognize the computational processes evolved by recursive functions certainly requires practice.

Mutual Recursion

When a recursive procedure is divided among two functions that call each other, the functions are said to be *mutually recursive*. As an example, consider the following definition of even and odd for non-negative integers:

- a number is even if it is one more than an odd number
- a number is odd if it is one more than an even number
- 0 is even

Using this definition, we can implement mutually recursive functions to determine whether a number is even or odd:

```
env_diagram(r'''
def is_even(n):
    if n == 0:
        return True
    else:
        return is_odd(n-1)

def is_odd(n):
    if n == 0:
        return False
    else:
        return is_even(n-1)

result = is_even(4)
''' , step=0, cumulative=False)
```

Loading...

Mutually recursive functions can be turned into a single recursive function by breaking the abstraction boundary between the two functions. In this example, the body of `is_odd` can be incorporated into that of `is_even`, making sure to replace `n` with `n-1` in the body of `is_odd` to reflect the argument passed into it:

```
>>> def is_even(n):
    if n == 0:
        return True
    else:
        if (n-1) == 0:
            return False
        else:
            return is_even((n-1)-1)
```

As such, mutual recursion is no more mysterious or powerful than simple recursion, and it provides a mechanism for maintaining abstraction within a complicated recursive program.

Printing in Recursive Functions

The computational process evolved by a recursive function can often be visualized using calls to `print`. As an example, we will implement a function `cascade` that prints all prefixes of a number from largest to smallest to largest.

```
>>> def cascade(n):
    """Print a cascade of prefixes of n."""
    if n < 10:
        print(n)
    else:
        print(n)
        cascade(n//10)
        print(n)

>>> cascade(2013)
2013
201
20
2
20
201
2013
```

In this recursive function, the base case is a single-digit number, which is printed. Otherwise, a recursive call is placed between two calls to `print`.

It is not a rigid requirement that base cases be expressed before recursive calls. In fact, this function can be expressed more compactly by observing that `print(n)` is repeated in both clauses of the conditional statement, and therefore can precede it.

```
>>> def cascade(n):
    """Print a cascade of prefixes of n."""
    print(n)
    if n >= 10:
        cascade(n//10)
    print(n)
```

As another example of mutual recursion, consider a two-player game in which there are `n` initial pebbles on a table. The players take turns, removing either one or two pebbles from the table, and the player who removes the final pebble wins. Suppose that Alice and Bob play this game, each using a simple strategy:

- Alice always removes a single pebble
- Bob removes two pebbles if an even number of pebbles is on the table, and one otherwise

Given `n` initial pebbles and Alice starting, who wins the game?

A natural decomposition of this problem is to encapsulate each strategy in its own function. This allows us to modify one strategy without affecting the other, maintaining the abstraction barrier between the two. In order to incorporate the turn-by-turn nature of the game, these two functions call each other at the end of each turn.

```
>>> def play_alice(n):
    if n == 0:
        print("Bob wins!")
    else:
        play_bob(n-1)

>>> def play_bob(n):
    if n == 0:
        print("Alice wins!")
    elif is_even(n):
        play_alice(n-2)
    else:
        play_alice(n-1)

>>> play_alice(20)
Bob wins!
```

In `play_bob`, we see that multiple recursive calls may appear in the body of a function. However, in this example, each call to `play_bob` calls `play_alice` at most once. In the next section, we consider what happens when a single function call makes multiple direct recursive calls.

Tree Recursion¶

Another common pattern of computation is called tree recursion, in which a function calls itself more than once. As an example, consider computing the sequence of Fibonacci numbers, in which each number is the sum of the preceding two.

```
env_diagram(r'''
def fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    else:
        return fib(n-2) + fib(n-1)

result = fib(6)
''', step=0, cumulative=False)
Loading...
```

This recursive definition is tremendously appealing relative to our previous attempts: it exactly mirrors the familiar definition of Fibonacci numbers. A function with multiple recursive calls is said to be *tree recursive* because each call branches into multiple smaller calls, each of which branches into yet smaller calls, just as the branches of a tree become smaller but more numerous as they extend from the trunk.

We were already able to define a function to compute Fibonacci numbers without tree recursion. In fact, our previous attempts were more efficient, a topic discussed later in the text. Next, we consider a problem for which the tree recursive solution is substantially simpler than any

iterative alternative.

Example: Partitions

The number of partitions of a positive integer n , using parts up to size m , is the number of ways in which n can be expressed as the sum of positive integer parts up to m in increasing order. For example, the number of partitions of 6 using parts up to 4 is 9.

1. $6 = 2 + 4$
2. $6 = 1 + 1 + 4$
3. $6 = 3 + 3$
4. $6 = 1 + 2 + 3$
5. $6 = 1 + 1 + 1 + 3$
6. $6 = 2 + 2 + 2$
7. $6 = 1 + 1 + 2 + 2$
8. $6 = 1 + 1 + 1 + 1 + 2$
9. $6 = 1 + 1 + 1 + 1 + 1 + 1$

We will define a function `count_partitions(n, m)` that returns the number of different partitions of n using parts up to m . This function has a simple solution as a tree-recursive function, based on the following observation:

The number of ways to partition n using integers up to m equals

1. the number of ways to partition $n-m$ using integers up to m , and
2. the number of ways to partition n using integers up to $m-1$.

To see why this is true, observe that all the ways of partitioning n can be divided into two groups: those that include at least one m and those that do not. Moreover, each partition in the first group is a partition of $n-m$, followed by m added at the end. In the example above, the first two partitions contain 4, and the rest do not.

Therefore, we can recursively reduce the problem of partitioning n using integers up to m into two simpler problems: (1) partition a smaller number $n-m$, and (2) partition with smaller components up to $m-1$.

To complete the implementation, we need to specify the following base cases:

1. There is one way to partition 0: include no parts.
2. There are 0 ways to partition a negative n .

3. There are 0 ways to partition any `n` greater than 0 using parts of size 0 or less.

```
>>> def count_partitions(n, m):
    """Count the ways to partition n using parts up to m."""
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        return count_partitions(n-m, m) + count_partitions(n, m-1)

>>> count_partitions(6, 4)
9
>>> count_partitions(5, 5)
7
>>> count_partitions(10, 10)
42
>>> count_partitions(15, 15)
176
>>> count_partitions(20, 20)
627
```

We can think of a tree-recursive function as exploring different possibilities. In this case, we explore the possibility that we use a part of size `m` and the possibility that we do not. The first and second recursive calls correspond to these possibilities.

Implementing this function without recursion would be substantially more involved. Interested readers are encouraged to try.



[Chapter 1: Functions](#)

[1.6 Higher-Order Functions](#)

[Chapter 1: Functions](#)

[Functions Reference](#)



Functions Reference



Functions Reference

The first chapter discusses expressions, names, assignment, control, and functions. This reference section contains the chapter's most important definitions, along with a list of built-in functions and some mathematical background material that appears in examples.

Expressions^[1]

An *expression* is code that can be *evaluated* and therefore has a *value*.

Learning Python involves learning its different forms of expressions and how they are evaluated. Important kinds of expressions include:

- Numbers, which evaluate to themselves: `3.14` evaluates to the number `3.14`
- Names, which evaluate to their values in the current environment
- Call expressions, which call a function on some arguments

Names^[1]

Programs can have different behavior (such as displaying messages, sending email, etc.), but mostly exist to manipulate values. A *value* is a general term for some piece of data, which could be a single number or a whole document. This course starts by focusing on only a few types of values: numbers, boolean values (`True` and `False`), the `None` value, strings, and functions.

Code includes names, which refer to values. All expressions are evaluated in some *environment*, which is the part of the Python interpreter that tracks which names refer to which values. When a name and value are *bound* together in the environment, the name will evaluate to the value in that environment.

It means the same thing to say that a name is *bound* to a value or to say that a value is *assigned* to a name.

Call Expressions^[1]

Here are two example call expressions and their values:

```
pow(2, 10)
```

```
1024
```

```
pow(1 + 1, min(3, abs(-4)))
```

```
8
```

A call expression has an expression before its parentheses, which in this course we call the *operator* expression ^[1]. The operator is almost always a name, and it evaluates to the function being called.

A call expression also has expressions within its parentheses separated by commas, which in this course we call *operand* expressions. These evaluate to the arguments of the function.

Python evaluates a call expression by evaluating the operator, then evaluating the operands, then calling the function (the value of the operator) on its arguments (the values of the operands). Calling a function on some arguments returns a value, and the value of the call expression is the value returned by the function called.

For example, in `pow(1 + 1, min(3, abs(-4)))` :

- the operator is the name `pow`, and its value is the built-in `pow` function.
- the operands are `1 + 1` and `min(3, abs(-4))`, and their values are 2 and 3.
- calling the `pow` function on 2 and 3 returns 8, which is the value of the whole expression.

Built-in Functions^[1]

Python starts running all programs in the global environment. Initially, a lot of names are bound to built-in functions. For example, the name `max` is initially bound to the built-in `max` function. The built-in functions used in the first unit of the course are:

- `pow` always takes two numbers. `pow(x, y)` returns `x` raised to the power of `y`. `pow(2, 3)` returns 8.

- `abs` takes a single number and returns its absolute value. `abs(-5)` returns 5.
- `bool` takes a single argument and returns `True` if the argument is a true value (sometimes called a *truthy* value), otherwise it returns `False`. Some false values: `0`, `False`, `''`, `None`. All other numbers and strings are true values, as are all functions.
- `str` returns a string for any value. `str(123)` returns `'123'`.
- `round` takes a number and optionally the number of decimal places to round to, and returns the rounded number. `round(3.14159, 2)` returns 3.14, `round(3.5)` returns 4.
- `max` takes two or more arguments and returns the largest one. `max(1, 5, 3)` returns 5.
- `min` takes two or more arguments and returns the smallest one. `min(1, 5, 3)` returns 1.
- `len` takes one argument that has a length, such as a string. It returns the length of the string. It cannot be called on a number.

In Chapter 2 we'll learn about other ways to call `max` and `min`. We'll also learn about many other built-in functions, such as `sum`. A common pitfall: in Python, `sum(2, 3)` does not return 5. Use `add(2, 3)` or `2 + 3`.

Some functions need to be imported. The ones used in the first unit include:

- `add` always takes two arguments, which could both be numbers or both be strings. `add(2, 3)` returns 5 and `add('two', 'three')` returns `twothree`. For any two expressions `x` and `y`, `add(x, y)` has the same value and behavior as `x + y`.
- `sub(x, y)` is the same as `x - y`.
- `mul(x, y)` is the same as `x * y`.
- `floordiv(x, y)` is the same as `x // y`.
- `truediv(x, y)` is the same as `x / y`.

Numbers¹

Many examples in this course involve concepts related to numbers, such as multiples and factors.

The definitions that appear on assignments and exams aim to be short, general, and precise rather than intuitive and conversational. For example, when defining the term *multiple*, you won't see: "The multiples of 10 are 20, 30, 40, 50, and so on. They are what you get by repeatedly adding the same number." That definition is problematic because "and so on" isn't precise, and it leaves open a lot of questions, such as whether 0, 10, or -20 are multiples of 10. Instead, you'll see: "`n` is a multiple of integer `m` if `n` equals `m * b` for some integer `b`." This definition is shorter and clarifies that 0, 10, and -20 are all multiples of 10, just like 20, 30, 40, and 50.

Here are some useful concepts related to numbers:

A function `f` is *increasing* if `f(y) > f(x)` whenever `y > x`. Some properties of an increasing function `f` include:

- For any `x` and any positive `a`, `f(x + a) > f(x)`.
- `f(max(x, y)) = max(f(x), f(y))`

A closely related concept is *non-decreasing*: A function `f` is *non-decreasing* if `f(y) >= f(x)` whenever `y > x`. The properties are similar for a non-decreasing function `f`:

- For any `x` and any positive `a`, `f(x + a) >= f(x)`.
- `f(max(x, y)) = max(f(x), f(y))`

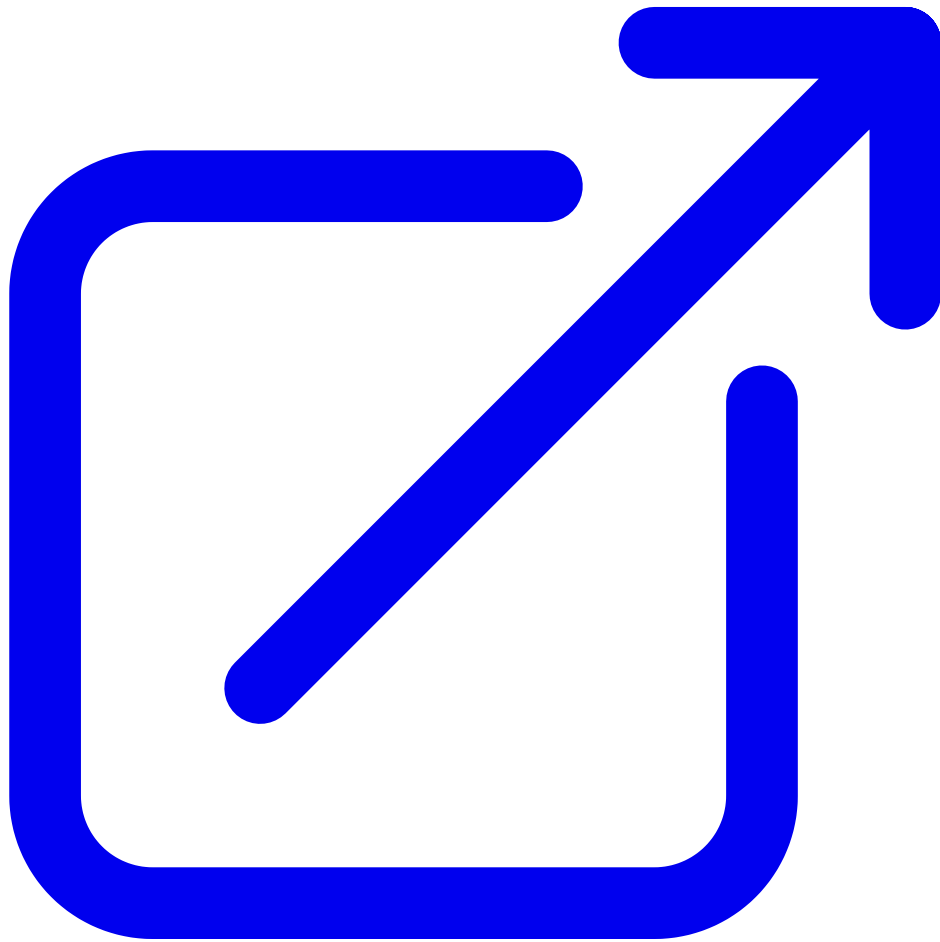
A positive integer `k` is a *factor* of a positive integer `n` if `n/k` is an integer. When `n/k` is an integer, we say that `k` *evenly divides* `n`. Some properties:

- `k` is a factor of `n` if `n` is a multiple of `k`.
- For any `n`, there are at least two factors: 1 and `n`. If these are the only two factors, then `n` is prime.

The remainder when dividing integer `n` by integer `k` is the smallest `r` such that `k*d + r` equals `n` for some integer `d`. The remainder when dividing `n` ...

- by 10 is the last digit of `n`
- by 100 is the last two digits of `n`
- by `pow(10, p)` is the last `p` digits of `n` for positive integer `p`.

1. The operator expression has different names in different documentation. For example, in the [Python Docs](#)



it is called the *primary*.



[Chapter 1: Functions](#)

[1.7 Recursive Functions](#)

[Chapter 2: Data](#)

[2.1 Introduction](#)



We concentrated in Chapter 1 on computational processes and on the role of functions in program design. We saw how to use primitive data (numbers) and primitive operations (arithmetic), how to form compound functions through composition and control, and how to create functional abstractions by giving names to processes. We also saw that higher-order functions enhance the expressiveness of a language by enabling us to manipulate, and thereby to reason, in terms of general methods of computation. This is much of the essence of programming.

Native Data Types

```
>>> type(2)
<class 'int'>
```

1. There are expressions that evaluate to values of native types, called *literals*.
2. There are built-in functions and operators to manipulate values of native types.

[illegible]

```
>>> type(1.5)
<class 'float'>
>>> type(1+1j)
<class 'complex'>
```

The name `float` comes from the way in which real numbers are represented in Python and many other programming languages: a “floating point” representation. While the details of how numbers are represented are not a topic for this text, some high-level differences between `int` and `float` objects are important to know. In particular, `int` objects represent integers exactly, without any approximation or limits on their size. On the other hand, `float` objects can represent a wide range of fractional numbers, but not all numbers can be represented exactly, and there are minimum and maximum values. Therefore, `float` values should be treated as approximations to real values. These approximations have only a finite amount of precision. Combining `float` values can lead to approximation errors; both of the following expressions would evaluate to `7` if not for approximation.

```
>>> 7 / 3 * 3
7.0

>>> 1 / 3 * 7 * 3
6.999999999999999
```

Although `int` values are combined above, dividing one `int` by another yields a `float` value: a truncated finite approximation to the actual ratio of the two integers divided.

```
>>> type(1/3)
<class 'float'>
>>> 1/3
0.3333333333333333
```

Problems with this approximation appear when we conduct equality tests.

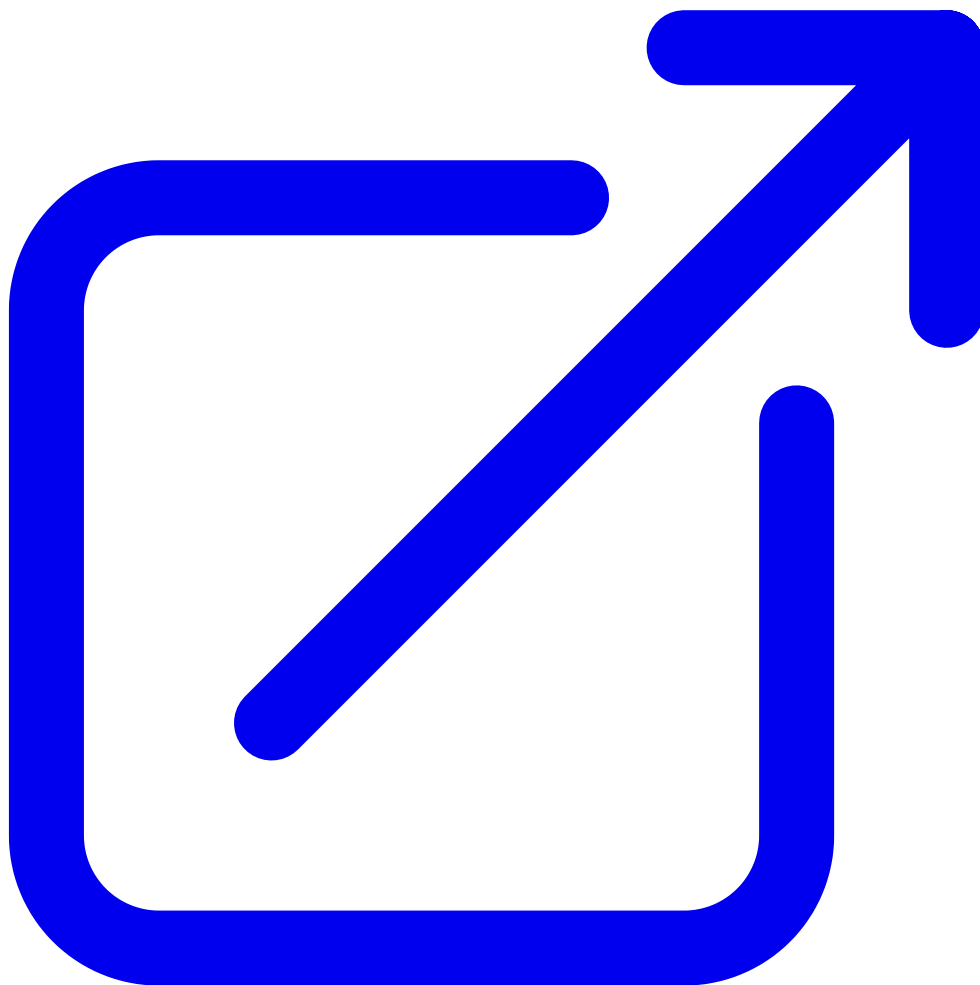
```
>>> 1/3 == 0.333333333333333312345 # Beware of float approximation
True
```

These subtle differences between the `int` and `float` class have wide-ranging consequences for writing programs, and so they are details that must be memorized by programmers. Fortunately, there are only a handful of native data types, limiting the amount of memorization required to become proficient in a programming language. Moreover, these same details are consistent across many programming languages, enforced by community guidelines such as the [IEEE 754 floating point standard](#).

Non-Numeric Types

Values can represent many other types of data, such as sounds, images, locations, web addresses, network connections, and more. Some are represented by native data types, such as the `bool` class for values `True` and `False` and the `str` class for text values such as `'Hello, world!'`. The type for many other values must be defined by programmers using the means of combination and abstraction that we will develop in this chapter.

This chapter introduces many of Python's native data types, as well as new ones we will create. For those interested in further details, a chapter on [native data types](#)



in the online book *Dive Into Python 3* gives a pragmatic overview of all Python's native data types and how to manipulate them, including numerous usage examples and practical tips.

Working with Types

Programming languages vary in their treatment of data types. Python is organized around types, but does not require that those types be written down explicitly in the code. Values of similar types can in many cases be used interchangeably by the same functions (such as `int` and `float` values). However, there are cases when examining, annotating, or conditioning on the type of a value is useful.

Type Inspection

The built-in function `isinstance` takes a value and a class. It returns true if the value is an instance of the class. For example, `show` rounds `float` values before printing them, but prints all other values as they are.

```
>>> def show(n):
    if isinstance(n, float):
        print(round(n, 2))
    else:
        print(n)
>>> show(100 / 3)
33.33
>>> show(100 > 5)
True
```

Type Coercion

The process of converting a value of one type to an equivalent value of another type is called *type coercion*. A value of one native type can in some circumstances be converted to another. All values can be converted to a `str`. A string containing only a number can be converted to an `int` (if there is no decimal component) or `float`. Call a type on a value to perform the conversion.

```
>>> str(12)
'12'
>>> float('34.56')
34.56
>>> int(78.0)
78
>>> int('9')
9
```

Type Hints

Most functions are designed to be called on values of a particular type. A function can be annotated with the intended types of its formal parameters (the names of its arguments), as well as its return value following a `->` symbol. These annotations are called *type hints* in Python. In the example below, the formal parameter `x` and the return value are annotated as `int` values.

```
>>> def square(x: int) -> int:
    return x * x
>>> square(5)
25
```

Type hints are not restrictions on what the function can take and return, but instead documentation of the function's intended use. This `square` function could take and return `float` values, despite the type hints, and it would return successfully.

```
>>> square(1.4142)
1.9999616399999998
```

The `square` function could also be called on `str` arguments, such as `square('three')`, which would cause a `TypeError`, but not because of the type hints. Instead, the call to `square` would be allowed, then an error would occur when the Python interpreter attempted to evaluate `x * x` with `x` bound to a `str` value.

Union types. A function that can take either a `str` or a `float` can be annotated with the union type `str | float`. This `double` function doubles a number, converting a `str` argument to a `float` and back.

```
>>> def double(x: str | float) -> str | float:
    if isinstance(x, str):
        return str(2 * float(x))
    else:
        return 2 * x
>>> double('3')
'6.0'
>>> double(3.5)
7.0
```

Type aliases. One way to give a type a name is to use a `type` statement. User-defined types are conventionally capitalized. Once a new type name is introduced, it can be used in type hints.

```
type Quantity = str | float

def double(x: Quantity) -> Quantity:
    if isinstance(x, str):
        return str(2 * float(x))
    else:
        return 2 * x
```

Type variables. A type variable is used to describe a function that takes and returns related types. A name, usually `T`, is used to indicate that the same type is involved in two or more formal parameters or return values. For example, the `identity` function takes an argument of any type and returns it.

```
def identity[T](x: T) -> T:
    return x
```

Furthermore, a type variable can be constrained to be among a collection of other types. This hint for the `square` function indicates that it can take either an `int` or `float` value, and that its return value is the same type as its argument.

```
def square[T: (int, float)](x: T) -> T:
    return x * x
```

This syntax for type variables and type aliases was only introduced recently, in Python 3.12 released in 2023. If you try these examples and see a `SyntaxError`, upgrade to a more recent version of Python.

Type checkers. Type hints can be *type checked*, which is a process that inspects the code in a Python file without executing it to determine whether names annotated with a type will be bound to compatible values. Likewise, it checks whether all return values of a function will be compatible with the annotated return type of that function. The Visual Studio Code editor with the Microsoft Python extension installed can detect type incompatibilities as you edit a Python source file, but this type checking is turned off by default. To turn it on, select *Settings* from the *Code* menu (Mac) or *Preferences* from the *File* menu (Windows), then change the setting called *Python › Analysis: Type Checking Mode* from `off` to `standard`. Search for “type checking mode” to find the setting. Then, type incompatibilities will be underlined as errors. Type checking for Python is not limited to VS Code and can be configured for many editors.

Type hints are optional and can in some cases make programs more difficult to read. Type checkers can often infer the types of formal parameters and return values even without adding type hints. Type hints are becoming more and more common in large Python programs, but remember that annotating a function with types does not change its behavior.



[Chapter 1: Functions](#)

[Functions Reference](#)

[Chapter 2: Data](#)

[2.2 Sequences](#)



2.2 Sequences



2.2 Sequences

```
from pytor import env_diagram
```

A sequence is an ordered collection of values. A sequence is not an instance of a particular built-in type in Python, but instead an abstraction: a collection of behaviors.

Length. A sequence has a finite length. An empty sequence has length 0.

Item selection. A sequence has an item corresponding to any non-negative integer index less than its length, starting at 0 for the first item.

While sequences are collections of items, which are values, each sequence itself is also a value. That is, a sequence can be given a name, passed as an argument, or returned from a function.

These behaviors are shared among several different data types, the most important of which is the `list`.

Lists

A list can be constructed by placing expressions within square brackets separated by commas. Such an expression is called a list literal.

```
>>> [10, 20]
[10, 20]
```

The items of a list can be accessed in two ways. The first way is via our familiar method of multiple assignment, which unpacks a list into its items and binds each item to a different name.

```
>>> pair = [10, 20]
>>> pair
[10, 20]
>>> x, y = pair
>>> x
10
>>> y
20
```

A second method for accessing the items in a list is by the item selection operator, also expressed using square brackets. Unlike a list literal, a square-brackets expression directly following another expression does not evaluate to a `list` value, but instead selects an item from the value of the preceding expression.

```
>>> pair[0]
10
>>> pair[1]
20
```

Lists in Python (and sequences in most other programming languages) are 0-indexed, meaning that the index 0 selects the first item, index 1 selects the second, and so on. One intuition that supports this indexing convention is that the index represents how far an item is offset from the beginning of the list.

The equivalent function for the item selection operator is called `getitem`, and it also uses 0-indexed positions to select items from a list.

```
>>> from operator import getitem
>>> getitem(pair, 0)
10
>>> getitem(pair, 1)
20
```

A `list` can have any finite length, and modern computers can work with lists that contain millions of items.

The built-in `len` function returns the length of a sequence. Below, `digits` is a list with four items. The item at index 3 is 8.

```
>>> digits = [1, 8, 2, 8]
>>> len(digits)
4
>>> digits[len(digits) - 1]
8
```

Lists can be added together and multiplied by integers. For sequences, addition and multiplication do not add or multiply items, but instead combine and replicate the sequences themselves. That is, the `+` operator (and the `add` function in the `operator` module) returns a list that is the concatenation of the added list arguments. The `*` operator (and the `mul` function in `operator`) can take a list and an integer `k` to return the list that consists of `k` repetitions of the original list.

```
>>> [2, 7] + digits * 2
[2, 7, 1, 8, 2, 8, 1, 8, 2, 8]
```

Any values can be included in a list, including another list. Item selection can be applied multiple times in order to select a deeply nested item in a list containing lists.

```
>>> pairs = [[10, 20], [30, 40]]
>>> pairs[1]
[30, 40]
>>> pairs[1][0]
30
```

Allowing lists to contain other lists enables us to create hierarchical structures --- structures made up of parts, which themselves are made up of parts, and so on.

```
>>> dog = [['head', ['face', ['nose', 'eyes']], ['ears']], ['body', ['legs', ['paws']], 'tail']]
>>> dog[0]
['head', ['face', ['nose', 'eyes']], ['ears']]
```

We can visualize lists in environment diagrams using *box-and-pointer* notation. A list is depicted as adjacent boxes that contain the items of the list. Primitive values such as numbers, strings, boolean values, and `None` appear within an item box. Compound values, such as function values and other lists, are indicated by an arrow.

```
env_diagram(r'''
one_two = [1, 2]
nested = [[1, 2], [],
          [[3, False, None],
           [4, lambda: 5]]]
''', cumulative=False)
```

Loading...

Sequence Iteration¹

In many cases, we would like to iterate over the items of a sequence and perform some computation for each item in turn. This pattern is so common that Python has an additional control statement to process sequential data: the `for` statement.

Consider the problem of counting how many times a value appears in a sequence. We can implement a function to compute this count using a `while` loop.

```
>>> def count(s, value):
    """Count the number of occurrences of value in sequence s."""
    total, index = 0, 0
    while index < len(s):
        if s[index] == value:
            total = total + 1
            index = index + 1
    return total

>>> count(digits, 8)
2
```

The Python `for` statement can simplify this function body by iterating over the item values directly without introducing the name `index` at all.

```
>>> def count(s, value):
    """Count the number of occurrences of value in sequence s."""
    total = 0
    for item in s:
        if item == value:
            total = total + 1
    return total

>>> count(digits, 8)
2
```

A `for` statement consists of a single clause with the form:

```
for <name> in <expression>:
    <suite>
```

A `for` statement is executed by the following procedure:

1. Evaluate the header `<expression>`, which must yield an iterable value.
2. For each item value in that iterable value, in order: A. Bind `<name>` to that value in the current frame. B. Execute the `<suite>`.

This execution procedure refers to *iterable values*. Lists are a type of sequence, and sequences are iterable values. Their items are considered in their sequential order. Python includes other iterable types, but we will focus on sequences for now; the general definition of the term “iterable” appears in the section on iterators in Chapter 4.

An important consequence of this evaluation procedure is that `<name>` will be bound to the last item of the sequence after the `for` statement is executed (unless the sequence is empty). The `for` loop introduces yet another way in which the environment can be updated by a statement.

Sequence Unpacking

A common pattern in programs is to have a sequence of items that are themselves sequences, but all of a fixed length. A `for` statement may include multiple names in its header to “unpack” each item sequence into its respective items. For example, we may have a list of two-item lists.

```
>>> pairs = [[1, 2], [2, 2], [2, 3], [4, 4]]
```

and wish to find the number of these pairs that have the same first and second item.

```
>>> same_count = 0
```

The following `for` statement with two names in its header will bind each name `x` and `y` to the first and second items in each pair, respectively.

```
>>> for x, y in pairs:
    if x == y:
        same_count = same_count + 1
>>> same_count
2
```

This pattern of binding multiple names to multiple values in a fixed-length sequence is called *sequence unpacking*; it is the same pattern that we see in assignment statements that bind multiple names to multiple values.

Ranges

A `range` is another built-in type of sequence in Python, which represents a range of integers. Ranges are created with `range`, which takes two integer arguments: the first number and one beyond the last number in the desired range.

```
>>> range(1, 10) # Includes 1, but not 10
range(1, 10)
```

Calling the `list` constructor on a range evaluates to a list with the same items as the range, so that the items can be easily inspected.

```
>>> list(range(5, 8))
[5, 6, 7]
```

If only one argument is given, it is interpreted as one beyond the last value for a range that starts at 0.

```
>>> list(range(4))
[0, 1, 2, 3]
```

Ranges commonly appear as the expression in a `for` header to specify the number of times that the suite should be executed. A common convention is to use a single underscore character for the name in the `for` header if the name is unused in the suite.

```
>>> for _ in range(3):
    print('Go Bears!')

Go Bears!
Go Bears!
Go Bears!
```

This underscore is just another name in the environment as far as the interpreter is concerned, but has a conventional meaning among programmers that indicates the name will not appear in any future expressions.

Sequence Processing

Sequences are such a common form of compound data that whole programs are often organized around this single abstraction. Modular components that have sequences as both inputs and outputs can be mixed and matched to perform data processing. Complex components can be defined by chaining together a pipeline of sequence processing operations, each of which is simple and focused.

List Comprehensions

Many sequence processing operations can be expressed by evaluating a fixed expression for each item in a sequence and collecting the resulting values in a result sequence. In Python, a list comprehension is an expression that performs such a computation.

```
>>> odds = [1, 3, 5, 7, 9]
>>> [x+1 for x in odds]
[2, 4, 6, 8, 10]
```

The `for` keyword above is not part of a `for` statement, but instead part of a list comprehension because it is contained within square brackets. The sub-expression `x+1` is evaluated with `x` bound to each item of `odds` in turn, and the resulting values are collected into a list.

Another common sequence processing operation is to select a subset of values that satisfy some condition. List comprehensions can also express this pattern, for instance selecting all items of `odds` that evenly divide `25`.

```
>>> [x for x in odds if 25 % x == 0]
[1, 5]
```

The general form of a list comprehension is:

```
[<map expression> for <name> in <sequence expression> if <filter expression>]
```

To evaluate a list comprehension, Python evaluates the `<sequence expression>`, which must return an iterable value. Then, for each item in order, the item value is bound to `<name>`, the filter expression is evaluated (if present), and if it evaluates to a true value (or there is no filter), the map expression is evaluated. The values of the map expression are collected into a list.

Aggregation

A third common pattern in sequence processing is to aggregate all values in a sequence into a single value. The built-in functions `sum`, `min`, and `max` are all examples of aggregation functions.

By combining the patterns of evaluating an expression for each item, selecting a subset of items, and aggregating items, we can solve problems using a sequence processing approach.

A perfect number is a positive integer that is equal to the sum of its divisors. The divisors of `n` are positive integers less than `n` that divide evenly into `n`. Listing the divisors of `n` can be expressed with a list comprehension.

```
>>> def divisors(n):
    return [1] + [x for x in range(2, n) if n % x == 0]

>>> divisors(4)
[1, 2]
>>> divisors(12)
[1, 2, 3, 4, 6]
```

Using `divisors`, we can compute all perfect numbers up to 1000 with another list comprehension. (The range begins at 2 because `divisors(1)` is `[1]`, which sums to 1, but 1 is not a perfect number.)

```
>>> [n for n in range(2, 1000) if sum(divisors(n)) == n]
[6, 28, 496]
```

We can reuse our definition of `divisors` to solve another problem, finding the minimum perimeter of a rectangle with integer side lengths, given its area. The area of a rectangle is its height times its width. Therefore, given the area and height, we can compute the width. We can assert that the height evenly divides the area to ensure that the side lengths are integers.

```
>>> def width(area, height):
    assert area % height == 0
    return area // height
```

The perimeter of a rectangle is the sum of its side lengths.

```
>>> def perimeter(width, height):
    return 2 * width + 2 * height
```

The height of a rectangle with integer side lengths must be a divisor of its area. We can compute the minimum perimeter by considering all heights.

```
>>> def minimum_perimeter(area):
    heights = divisors(area)
    perimeters = [perimeter(width(area, h), h) for h in heights]
    return min(perimeters)

>>> area = 80
>>> width(area, 5)
16
>>> perimeter(16, 5)
42
>>> perimeter(10, 8)
36
>>> minimum_perimeter(area)
36
>>> [minimum_perimeter(n) for n in range(1, 10)]
[4, 6, 8, 8, 12, 10, 16, 12, 12]
```

Type hints. It is possible to indicate that a formal parameter or return value is a `list` and also that it is a list of items that all have a common type. For example, the type hint `list[int]` indicates that a name should be used for a list of integers.

A type hint describing `[[1, 2], [3, 4], [5, 6]]` would be `list[list[int]]`, since it is a list containing lists of integers. The fact that each list item contains exactly two integers is not something that a list type hint can specify in Python.

```
>>> def divisible_by(s: list[int], k: int) -> list[int]:
    "The items in s that are divisible by k."
    return [n for n in s if n % k == 0]

>>> divisible_by([3, 4, 5, 6, 7, 8], 4)
[4, 8]
```

Higher-Order Functions

The common patterns we have observed in sequence processing can be expressed using higher-order functions. First, evaluating an expression for each item in a sequence can be expressed by applying a function to each item.

```
>>> def apply_to_all(map_fn, s):
    return [map_fn(x) for x in s]
```

Selecting only items for which some expression is true can be expressed by applying a function to each item.

```
>>> def keep_if(filter_fn, s):
    return [x for x in s if filter_fn(x)]
```

Finally, many forms of aggregation can be expressed as repeatedly applying a two-argument function to the `reduced` value so far and each item in turn.

```
>>> def reduce(reduce_fn, s, initial):
    reduced = initial
    for x in s:
        reduced = reduce_fn(reduced, x)
    return reduced
```

Using `mul` as the `reduce_fn` and 1 as the `initial` value, `reduce` can be used to multiply together a sequence of numbers.

```
>>> from operator import mul
>>> reduce(mul, [2, 4, 8], 1)
64
```

We can find perfect numbers using these higher-order functions as well.

```
>>> def divisors_of(n):
    divides_n = lambda x: n % x == 0
    return [1] + keep_if(divides_n, range(2, n))

>>> divisors_of(12)
[1, 2, 3, 4, 6]
>>> from operator import add
>>> def sum_of_divisors(n):
    return reduce(add, divisors_of(n), 0)

>>> def perfect(n):
    return sum_of_divisors(n) == n

>>> keep_if(perfect, range(2, 1000))
[6, 28, 496]
```

Conventional names. In the computer science community, the more common name for `apply_to_all` is `map` and the more common name for `keep_if` is `filter`. In Python, the built-in `map` and `filter` are generalizations of these functions that do not return lists. These functions are discussed in Chapter 2's section on lazy evaluation. The `apply_to_all` and `keep_if` functions above are equivalent to applying the `list` constructor to the result of built-in `map` and `filter` calls.

```
>>> apply_to_all = lambda map_fn, s: list(map(map_fn, s))
>>> keep_if = lambda filter_fn, s: list(filter(filter_fn, s))
```

The `reduce` function is built into the `functools` module of the Python standard library. In this version, the `initial` argument is optional.

```
>>> from functools import reduce
>>> from operator import mul
>>> def product(s):
>>>     return reduce(mul, s)
>>> product([1, 2, 3, 4, 5])
120
```

In Python programs, it is more common to use list comprehensions directly rather than higher-order functions, but both approaches to sequence processing are widely used.

Sequence Abstraction

We have introduced two native data types that satisfy the conditions with which we began this section: length and item selection. Python includes two more behaviors of sequence types that extend the sequence abstraction.

Membership. A value can be tested for membership in a sequence. Python has two operators `in` and `not in` that evaluate to `True` or `False` depending on whether an item appears in a sequence.

```
>>> digits
[1, 8, 2, 8]
>>> 2 in digits
True
>>> 1828 not in digits
True
```

Slicing. Sequences contain smaller sequences within them. A *slice* of a sequence is any contiguous span of the original sequence, designated by a pair of integers. As with the `range` constructor, the first integer indicates the starting index of the slice and the second indicates one beyond the ending index.

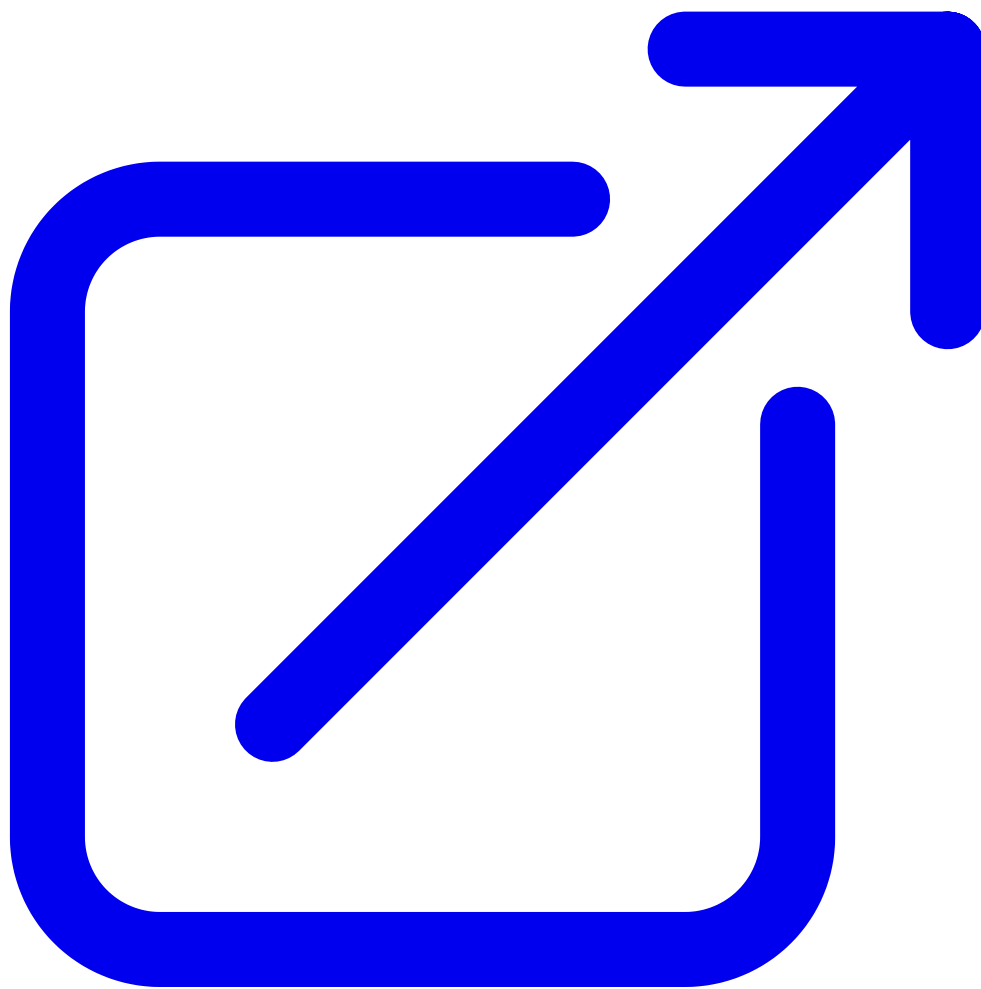
In Python, sequence slicing is expressed similarly to item selection, using square brackets. A colon separates the starting and ending indices. Any bound that is omitted is assumed to be an extreme value: 0 for the starting index, and the length of the sequence for the ending index.

```
>>> digits[0:2]
[1, 8]
>>> digits[1:]
[8, 2, 8]
```

Enumerating these additional behaviors of the Python sequence abstraction gives us an opportunity to reflect upon what constitutes a useful data abstraction in general. The richness of an abstraction (that is, how many behaviors it includes) has consequences. For users of an abstraction, additional behaviors can be helpful. On the other hand, complicated abstractions take longer for users to learn.

Sequences have a rich abstraction (with length, item selection, membership, and slicing) because they are so ubiquitous in computing that learning a few complex behaviors is justified. In general, most user-defined abstractions should be kept as simple as possible.

Further reading. Slice notation admits a variety of special cases, such as negative starting values, ending values, and step sizes. A complete description appears in the subsection called [slicing a list](#)



in Dive Into Python 3. In this chapter, we will only use the basic features described above.

Strings¹

Text values are perhaps more fundamental to computer science than even numbers. Python programs are written and stored as text. The native data type for text in Python is called a string, and corresponds to the constructor `str`.

There are many details of how strings are represented, expressed, and manipulated in Python. Strings are another example of a rich abstraction, one that requires a substantial commitment on the part of the programmer to master. This section serves as a condensed introduction to essential string behaviors.

String literals can express arbitrary text, surrounded by either single or double quotation marks.

```
>>> 'I am string!'
'I am string!'
>>> "I've got an apostrophe"
"I've got an apostrophe"
>>> '您好'
'您好'
```

We have seen strings already in our code, as docstrings, in calls to `print`, and as error messages in `assert` statements.

Strings satisfy the two basic conditions of a sequence that we introduced at the beginning of this section: they have a length and they support item selection.

```
>>> city = 'Berkeley'
>>> len(city)
8
>>> city[3]
'k'
```

The items of a string are themselves strings that have only a single character. A character is any single letter of the alphabet, punctuation mark, or other symbol. Unlike many other programming languages, Python does not have a separate character type; any text is a string, and strings that represent single characters have a length of 1.

Like lists, strings can also be combined via addition and multiplication.

```
>>> 'Berkeley' + ', CA'
'Berkeley, CA'
>>> 'Shabu ' * 2
'Shabu Shabu '
```

Membership. The behavior of strings diverges from other sequence types in Python. The string abstraction does not conform to the full sequence abstraction that we described for lists and ranges. In particular, the membership operator `in` applies to strings, but has an entirely different behavior than when it is applied to other sequences. It matches substrings rather than items.

```
>>> 'here' in "Where's Waldo?"
True
```

Multiline literals. Strings aren't limited to a single line. Triple quotes delimit string literals that span multiple lines. We have used this triple quoting extensively already for docstrings.

```
>>> """The Zen of Python
claims, "Readability counts."
Read more: import this."""
'The Zen of Python\nclaims, "Readability counts."\nRead more: import this.'
```

In the printed result above, the `\n` (pronounced "*backslash en*") is a single item that represents a new line. Although it appears as two characters (backslash and "n"), it is considered a single character for the purposes of length and item selection.

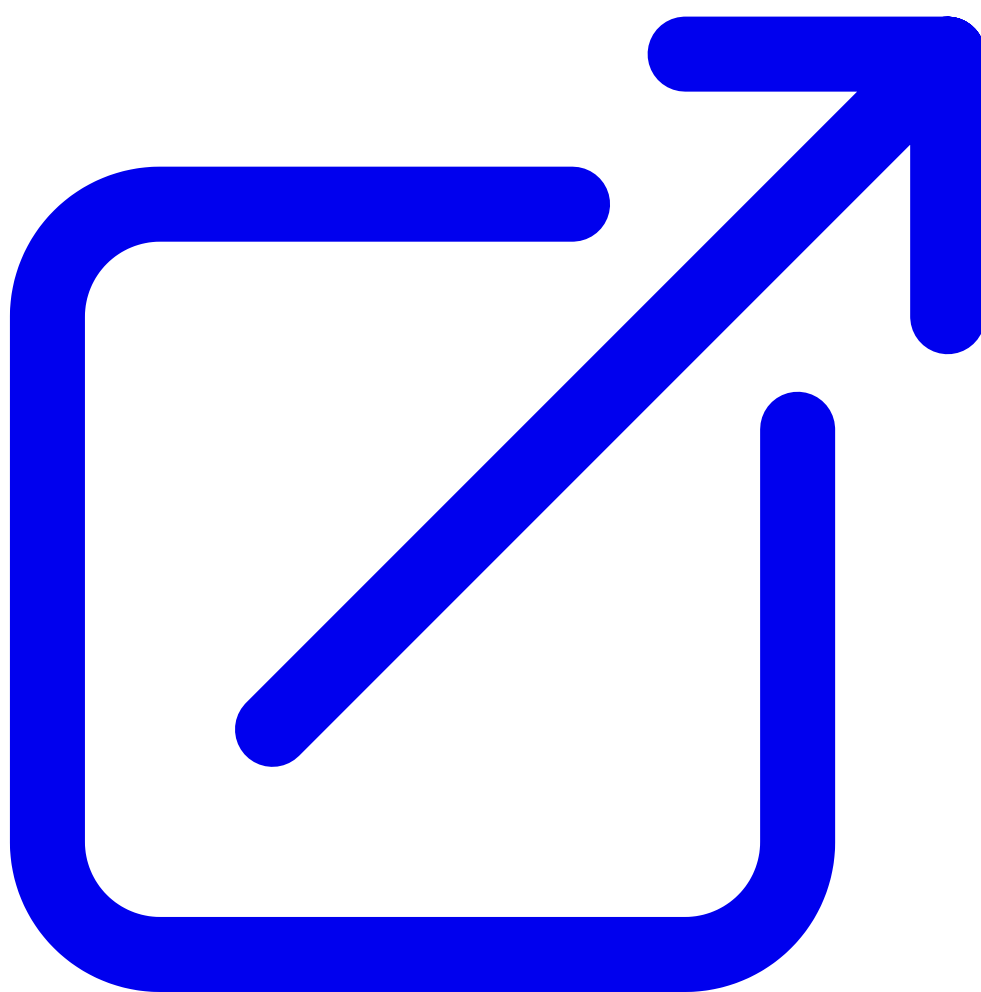
String coercion. A string can be created from any object in Python by calling the `str` constructor function with an object value as its argument. This feature of strings is useful for constructing descriptive strings from objects of various types.

```
>>> str(1 + 1) + ' is an item of ' + str(digits)
'2 is an item of [1, 8, 2, 8]'
```

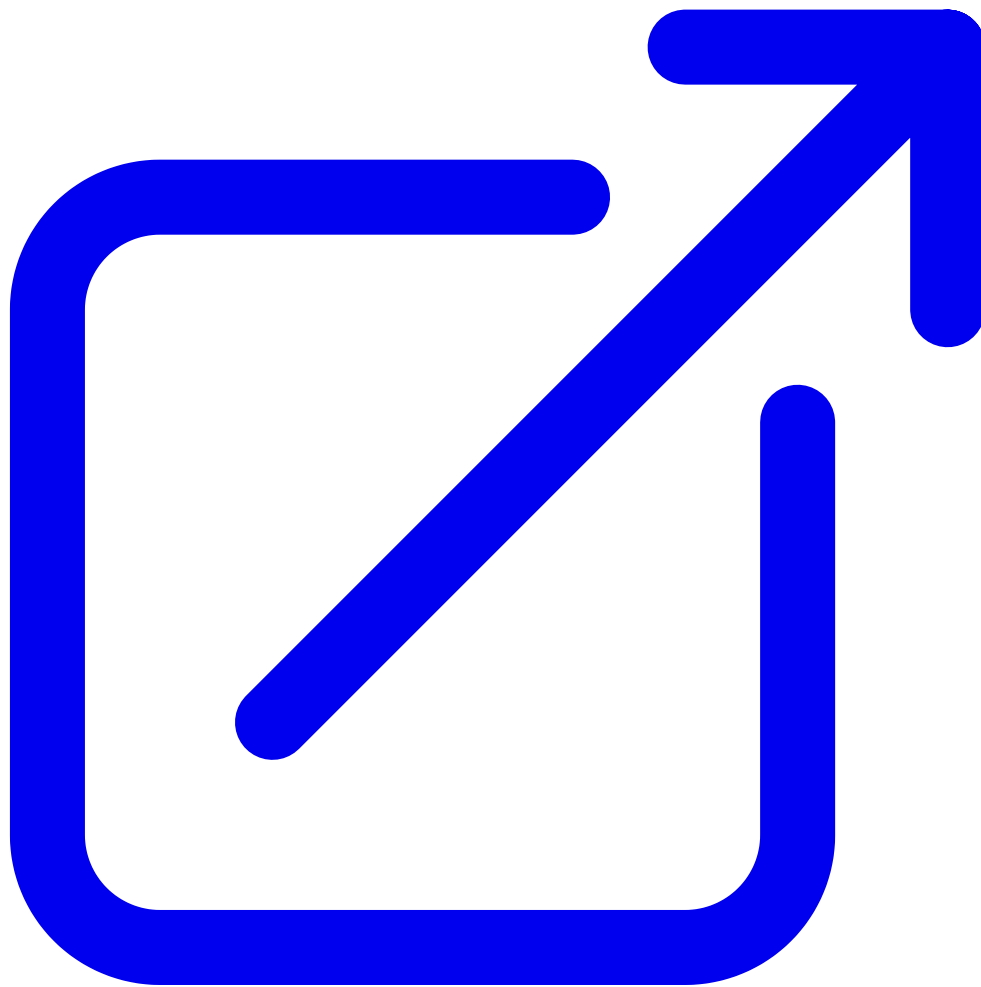
String interpolation. A string can also be created by inserting the string representation of the values of expressions into a larger template, called an f-string or *formatted string literal*. Preceding a string literal by the letter `f` directs Python to evaluate all expressions contained within braces and include the string representations of their values.

```
>>> f'{1 + 1} is an item of {digits}'
'2 is an item of [1, 8, 2, 8]'
```

Further reading. Encoding text in computers is a complex topic. In this chapter, we will abstract away the details of how strings are represented. However, for many applications, the particular details of how strings are encoded by computers are essential knowledge. [The strings chapter of Dive Into Python 3](#)



provides a description of character encodings and Unicode. The [Python tutorial section on f-strings](#)



describes the many ways that the format of values can be specified.

Dictionaries¹

Dictionaries are Python's built-in data type for storing and manipulating correspondence relationships. A dictionary is an ordered collection of key-value pairs. The purpose of a dictionary is to provide an abstraction for storing and retrieving values that are looked up not by consecutive integers, but by descriptive keys.

Strings commonly serve as keys, because strings are our conventional representation for names of things. This dictionary literal gives the values of various Roman numerals.

```
>>> numerals = {'I': 1, 'V': 5, 'X': 10}
>>> numerals
{'I': 1, 'V': 5, 'X': 10}
>>> len(numerals)
3
```

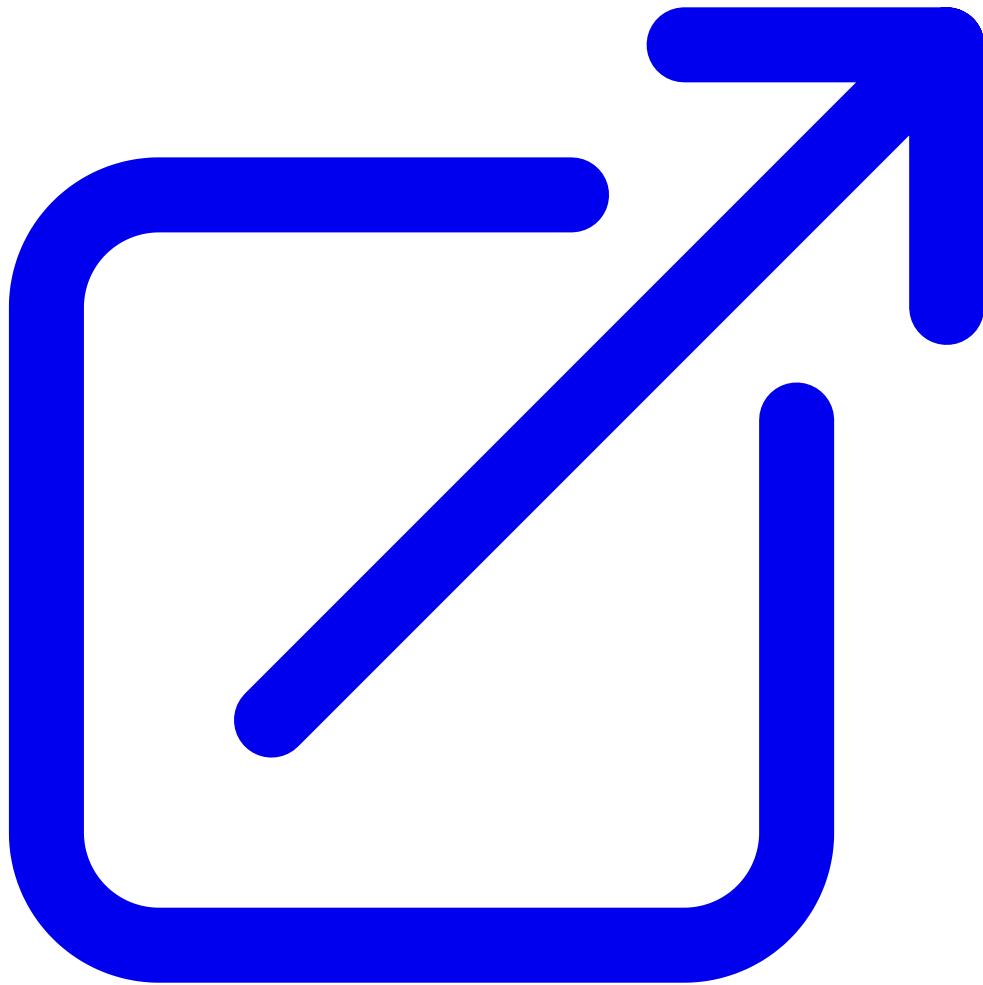
Looking up values by their keys uses the item selection operator that we previously applied to sequences.

```
>>> numerals['X']
10
```

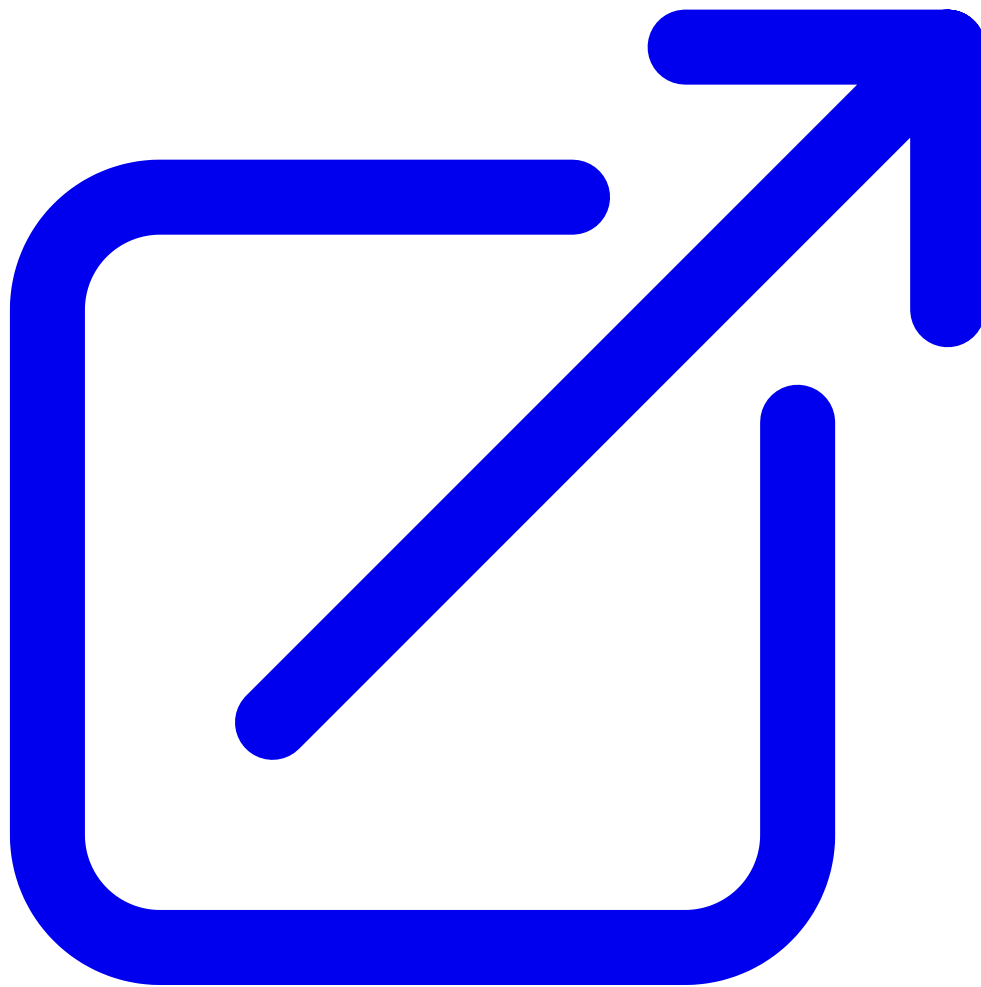
A dictionary can have at most one value for each key. The order of key-value pairs is determined by the order that the keys first appear, but only the last value is retained.

```
>>> {'X': 11, 'V': 5, 'X': 10}  
{'X': 10, 'V': 5}
```

A key of a dictionary cannot be a list or another dictionary (or, more generally, any other [unhashable](#)



value). This restriction is tied to the underlying implementation of [dictionaries in Python](#)



Dictionaries also support ways of iterating over the key-value pairs of a dictionary. The simplest approach iterates over the keys using a `for` statement.

```
>>> for k in numerals:
    print('key:', k, 'value:', numerals[k])
key: I value: 1
key: V value: 5
key: X value: 10
```

Similarly, it's possible to create a list of a dictionary's keys.

```
>>> list(numerals)
['I', 'V', 'X']
```

Dictionaries can appear in environment diagrams as well.

```
env_diagram(r'''
numerals = {'I': 1, 'V': 5, 'X': 10}
''', cumulative=False)
```

Loading...

The type hint for a dictionary can be just `dict` or specify a type for the keys and a type for the values. For example, the type hint describing the `numerals` dictionary would be `dict[str, int]`.

Dot Notation

Using a *dot expression*, it is also possible to iterate over the values or key-value pairs of a dictionary. A dot expression is a way to access parts of compound data or access behavior that is specific to the type of the value. The many details of dot notation are discussed in section 2.5 on object-oriented programming. For now, it is sufficient to know that `.values()` accesses the values of a dictionary and `.items()` accesses the key-value pairs. These are called *methods* of the dictionary.

```
>>> for v in numerals.values():
    print(v)
1
5
10
>>> for k, v in numerals.items():
    print(k, v)
I 1
V 5
X 10
```

Another useful method on dictionaries is `get`, which returns either the value for a key, if the key is present, or a default value. The arguments to `get` are the key and the default value.

```
>>> numerals.get('A', 0)
0
>>> numerals.get('V', 0)
5
```

A list of key-value pairs can be converted into a dictionary by calling `dict`.

```
>>> dict([[3, 9], [4, 16], [5, 25]])
{3: 9, 4: 16, 5: 25}
```

Dictionaries also have a comprehension syntax analogous to those of lists. A key expression and a value expression are separated by a colon. Evaluating a dictionary comprehension creates a new dictionary object.

```
>>> {x: x*x for x in range(3,6)}
{3: 9, 4: 16, 5: 25}
```

Tuples

A tuple is a sequence that can be the key of a dictionary (as long as the tuple contains no disallowed values such as lists or dictionaries). It is created using a tuple literal that separates item expressions by commas. Parentheses are optional but almost always included in practice. Any values can be placed within tuples.

```
>>> 1, 2 + 3
(1, 5)
>>> ("the", 1, ("and", "only"))
('the', 1, ('and', 'only'))
>>> type( (10, 20) )
<class 'tuple'>
```

Empty and one-item tuples have special literal syntax.

```
>>> () # 0 items
()
>>> (10,) # 1 item
(10,)
```

Like lists, tuples have a finite length and support item selection, slicing, and membership. Their items can also be unpacked using assignment.

```
>>> code = ('up', 'up', 'down', 'down') + ('left', 'right') * 2
>>> len(code)
8
>>> code[3]
'down'
>>> 'left' in code
True
>>> code[1:3]
('up', 'down')
>>> u, d = code[1:3]
>>> u
'up'
>>> d
'down'
```

The type hint for a tuple can specify the type of value in each position, since tuples are often used to hold multiple types. The example below contains a type hint `tuple[str, int]` for a two-item tuple holding a `str` and then an `int`. This format of type hint differs from list type hints,

which only specify one shared type for all the items in the list. As a consequence, the type hint for a tuple can specify the length of a tuple, while the type hint for a list cannot.

```
>>> def display_age(person: tuple[str, int], year: int) -> None:
    name, year_born = person
    print(name, 'was', year - year_born, 'years old in', year)

>>> keanu = ("Keanu Reeves", 1964)
>>> display_age(keanu, 1999)
Keanu Reeves was 35 years old in 1999
```

If a dictionary needs compound keys, tuples are a common choice. This dictionary distinguishes between Michael Jordan the basketball player and Michael B. Jordan the actor by associating each with his birth year.

```
>>> top_movies = {
    ('Michael Jordan', 1963): 'Space Jam',
    ('Michael Jordan', 1987): 'Black Panther',
}

>>> top_movies[('Michael Jordan', 1963)]
'Space Jam'
```

A precise type hint describing the `top_movies` dictionary would be `dict[tuple[str, int], str]`, which describes that the keys are two-element tuples and the values are strings.



[Chapter 2: Data](#)

[2.1 Introduction](#)

[Chapter 2: Data](#)

[2.3 Compound Data](#)



2.3 Compound Data



2.3 Compound Data

```
from pytutor import env_diagram
```

As we consider the wide set of things in the world that we would like to represent in our programs, we find that most of them have compound structure. For example, a geographic position has latitude and longitude coordinates. A date has a year, month, and day. Introducing dedicated types to represent concepts in the world, such as positions and dates, has proven to be an excellent way to organize complex programs.

The Object Metaphor

An object combines values and behavior. The central characteristic of objects is that they represent information but also *behave* like the information that they represent. When an object is printed, it provides text to describe itself. If an object is a combination of parts, it provides access to those parts. Objects are both information and functionality, bundled together to provide abstract representations.

In Python, all values are objects. Rather than data consisting of only numbers manipulated by functions, data are objects that have types which endow those objects with behavior. Python's `date` type provides an excellent illustration. While a program could use a list of integers (a year, month, and day) to represent a date, a `date` object bundles in a great deal of date-related functionality.

```
>>> from datetime import date
>>> date
<class 'datetime.date'>
>>> final = date(2026, 12, 18)
>>> final
datetime.date(2026, 12, 18)
```

The name `date` is bound to a *class*, which represents a kind of value. Individual dates are called *instances* or *objects* of that class. Instances are *constructed* by calling the class on arguments that describe the instance.

While `final` was constructed from primitive numbers, it behaves like a date. For instance, subtracting it from another date will give a time difference, which we can print.

```
>>> print(date(2026, 12, 21) - final)
3 days, 0:00:00
```

Objects have *attributes*, which are named values that are part of the object. In Python, like many other programming languages, we use dot notation to designate an attribute of an object.

`<expression> . <name>`

Above, the `<expression>` evaluates to an object, and `<name>` is the name of an attribute for that object.

Attribute names are not available in the global environment. Instead, they are particular to the object instance preceding the dot.

```
>>> final.year
2026
>>> final.day
18
```

Objects also have *methods*, which are function-valued attributes. Metaphorically, we say that the object “knows” how to carry out those methods. By implementation, methods are functions that compute their results from both their arguments and their object. For example, the `strftime` method (an abbreviation for “string format of time”) of `final` takes a single argument that specifies how to display a date (e.g., `%A` means that the day of the week should be spelled out in full).

```
>>> final.strftime('%A, %B %d')
'Friday, December 18'
```

Computing the return value of `strftime` requires two inputs: the string that describes the format of the output and the date information bundled into `final`. Date-specific logic is applied within this method to yield this result. We never stated that the 18th of December, 2026, was a Friday, but knowing the corresponding weekday is part of what it means to be a date. By bundling behavior and information together, this Python object offers us a convincing, self-contained abstraction of a date.

Dates are objects, but numbers, strings, lists, and ranges are all objects as well. They represent values, but also behave in a manner that befits the values they represent. They also have attributes and methods. For instance, strings have several methods that facilitate text processing.

```
>>> '1234'.isnumeric()
True
>>> 'rOBERT dE nIRO'.swapcase()
'Robert De Niro'
>>> 'eyes'.upper().endswith('YES')
True
```

Data Classes

To represent positions as objects, we would like our programming language to have the capacity to introduce a new class, the `Position`, which couples together a latitude and longitude into a *compound* object that our programs can manipulate as a single conceptual unit, but which also has two coordinates that can be accessed using dot expressions.

We could use a two-element list to represent a position, but then we would have to remember whether the latitude or longitude came first everywhere that we want to access its coordinates.

We can instead create a `Position` type that has attributes called `lat` and `lon` using a `class` statement with a `dataclass` decorator. The `class` statement is used to create new user-defined types. Many more details of `class` statements will be described in Section 2.5 on object-oriented programming. The `dataclass` decorator simplifies the creation of classes for compound data. The lines indented below the `class` header declare that every `Position` object has two attributes `lat` and `lon` that are intended to be numbers.

```
>>> from dataclasses import dataclass

>>> @dataclass
class Position:
    "A geographic position."
    lat: float
    lon: float
```

Once this class statement is executed, new `Position` objects can be created by calling `Position` on a pair of numbers. The coordinates of a position are accessed using dot expressions.

```
>>> sydney = Position(-33.87, 151.21)
>>> sydney.lat
-33.87
>>> sydney.lon
151.21
>>> sydney
Position(lat=-33.87, lon=151.21)
>>> type(sydney) == Position
True
>>> isinstance(sydney, Position)
True
```

A `Position` can be formatted to include cardinal directions (N, S, E, W) by defining a function that takes a `Position` and returns a `str`. ^[1]

```
>>> def format_pos(p: Position) -> str:
    "Format the position as a string with directional suffixes."
    assert abs(p.lat) <= 90 and abs(p.lon) <= 180, f'{p!r} is not on Earth!'
    if p.lat >= 0:
        lat_dir = "N"
    else:
        lat_dir = "S"

    if p.lon >= 0:
        lon_dir = "E"
    else:
        lon_dir = "W"

    return f"{abs(p.lat)}° {lat_dir}, {abs(p.lon)}° {lon_dir}"

>>> format_pos(sydney)
'33.87° S, 151.21° E'
```

We can redefine the `Position` class to include a `__str__` method, which is a function defined within a class. This special name `__str__` directs Python to call `format_pos` each time any `Position` is printed or passed to `str`. A full description of how to add behavior to classes appears in Section 2.5 on object-oriented programming. For now, we will focus on ensuring that our user-defined data types provide useful information when printed.

```

>>> @dataclass
class Position:
    "A geographic position."
    lat: float
    lon: float

    def __str__(self):
        return format_pos(self)
>>> sydney = Position(-33.87, 151.21)
>>> str(sydney)
'33.87° S, 151.21° E'
>>> print('The position of Sydney is', sydney)
The position of Sydney is 33.87° S, 151.21° E

```

Data Abstraction

Before continuing with further examples of user-defined types, it is worth observing the role of abstraction in the position example. By defining a class, we can use a `Position` ---accessing its named attributes (`lat` and `lon`) or printing it---without needing to specify or decide how its primitive components are stored and combined. Without using a `class` statement, we would need to decide whether a position is represented as a list, a dictionary, or some other way, and each time we wanted to print a position we would need to call `format_pos`.

In fact, all the native classes we have used are also abstractions. A Python list has some way in which its items are stored and combined, but we can use a list---accessing its length and items---without reasoning about its representation in the computer's memory. We can print a list without thinking about the process that constructs its printed representation. Likewise, dictionaries and strings are abstractions that allow us to interact with compound data while abstracting away the details of exactly how a dictionary looks up a value by its key or how a string can represent letters in many different alphabets. Even `int` and `float` types are abstractions that allow us to work with numbers without always reasoning about the details of how numbers are stored by the transistors of a computer.

Data abstraction involves giving a name to a type of data and interacting with compound data values according to a fixed set of operations that are specific to that type of data, without making assumptions or reasoning about how those operations are implemented. It is analogous to functional abstraction, which involves giving a name to a computational process and using it (by calling a function) without making assumptions or reasoning about how the process is implemented.

Eventually learning about the implementation of data abstractions is both interesting and important for building useful software. But it is also helpful to learn to work with data abstractly. Expert programmers not only understand how numbers, lists, dictionaries, and classes are implemented, but also learn to use these types of objects without always thinking about their implementation. The concept of an object, which is shared by many programming languages, is designed to help separate the use and implementation of compound data.

Linked Lists

So far, we have used only native types to represent sequences. However, we can also develop sequence representations that are not built into Python. For example, a common and useful representation of a sequence is the *linked list*.

A linked list combines a `first` item of a sequence and the `rest` of the sequence. Linked lists have a recursive structure: the rest of a linked list is another linked list. It is possible to represent a linked list just with two-element tuples, using the empty tuple to represent an empty linked list. In this way, we see that any method of combining two values can be used to represent a sequence of any length.

```

env_diagram(r'''
s = (9, (7, (5, (3, ())))))
''', cumulative=False)

```

Loading...

Linked lists differ from two-item tuples because of their regular structure: each first item has the same type and each second item is another linked list. We can express this structure using a data class for linked lists.

```

>>> type LinkedList[T] = Link[T] | tuple[()]

>>> @dataclass
class Link[T]:
    "A Link has a first value of type T and the rest of the linked list."
    first: T
    rest: LinkedList[T] = () # rest defaults to an empty linked list

```

The type `tuple[()]` has only one compatible value: the empty tuple.

The examples below demonstrate how to construct a linked list containing the numbers 3, 4, and 5 and how to access its items using `first` and `rest` attributes.

```

>>> s = Link(3, Link(4, Link(5)))
>>> s.first
3
>>> s.rest.first
4
>>> s.rest.rest.first
5
>>> s.rest.rest.rest == ()
True
>>> s
Link(first=3, rest=Link(first=4, rest=Link(first=5, rest=())))
>>> s.rest.rest
Link(first=5, rest=())

```

There is no one standardized way to display a linked list, but a common one is to show the items it contains within parentheses and without commas. This notation was introduced by the Lisp programming language, one of the oldest languages still widely used today.

Adding a `__str__` method to the `Link` class generates this string representation each time a `Link` object is printed or passed to `str`.

```

>>> def format_link(s: Link):
    """Return a Link s formatted as items within parentheses."""
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'

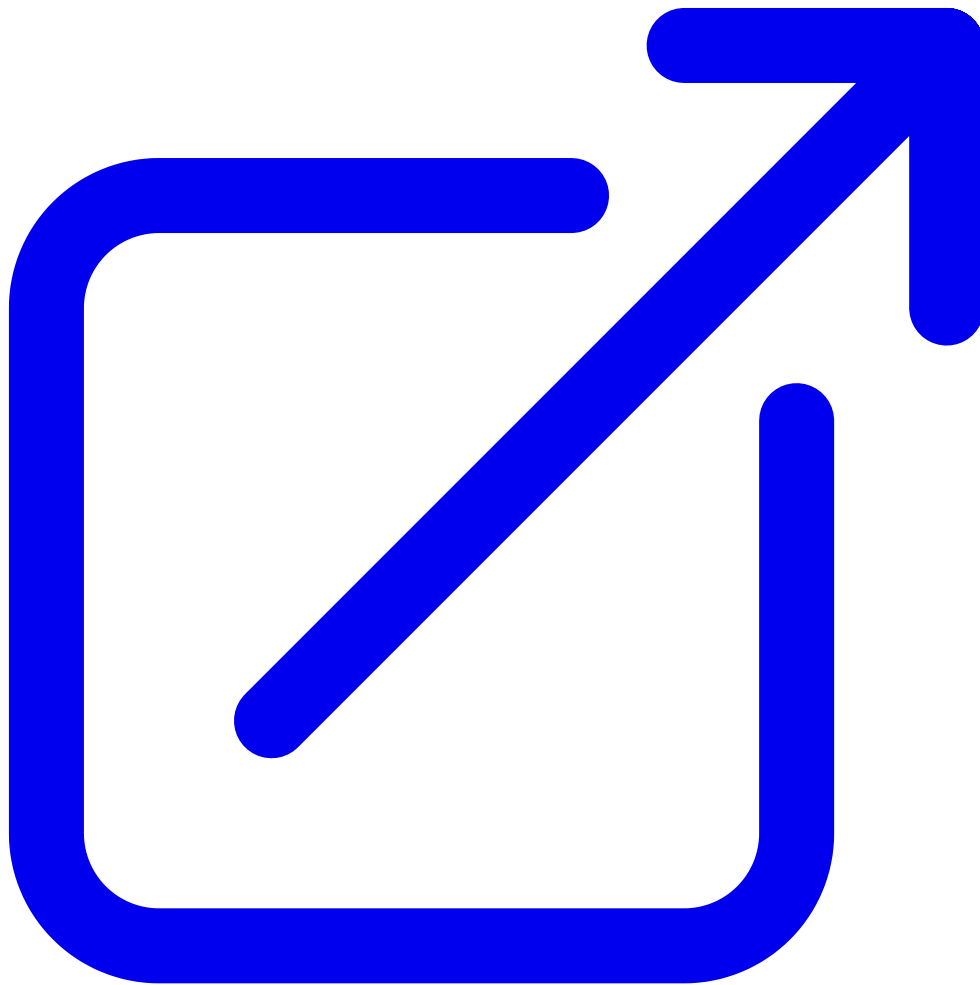
>>> @dataclass
class Link[T]:
    """A Link has a first value of type T and the rest of the linked list."""
    first: T
    rest: LinkedList[T] = () # rest defaults to an empty linked list

    def __str__(self):
        return format_link(self)

>>> s = Link(3, Link(4, Link(5)))
>>> print(s)
(3 4 5)

```

The complete [Link class](#)



appears in the examples appendix.

The most common way to visualize any linked list, whether created from tuples or a data class, is to use a box-and-pointer diagram in which each `Link` is a pair of boxes holding its `first` and `rest` and the empty list is denoted by a slash through the rest of the final `Link` in a linked list. Here is the diagram for `Link(3, Link(4, Link(5)))`, which would be printed as `(3 4 5)`.



Here are two example functions that take linked lists: `len_link` returns the number of items contained in a linked list and `getitem_link` returns an item by its position.

```

>>> def len_link(s: LinkedList):
    """Return the length of a linked list."""
    length = 0
    while isinstance(s, Link):
        length += 1
        s = s.rest
    return length
>>> len_link(s)
3

>>> def getitem_link(s: LinkedList, i: int):
    """Return the item at index i of a non-empty linked list."""
    while i > 0:
        assert isinstance(s, Link), 'Index out of range'
        s = s.rest
        i -= 1
    assert isinstance(s, Link), 'Index out of range'
    return s.first
>>> getitem_link(s, 1)
4

```

Type variables. Like a `list`, a `LinkedList` typically holds items that all share a type. The `LinkedList[T]` and `Link[T]` types have a type variable `T`, which represents a shared type for all of the items they contain. It can be replaced by any type. For example, a `LinkedList[int]` describes a `LinkedList` that contains `int` values as all of its items. Functions that take or return linked lists can optionally specify the type of items contained within those lists.

```

>>> def sum_link(s: LinkedList[float]) -> float:
    """Return the sum of the items in a linked list of numbers."""
    total = 0
    while isinstance(s, Link):
        total += s.first
        s = s.rest
    return total
>>> sum_link(s)
12

```

These examples demonstrate a common pattern of computation with linked lists, where each step in an iteration operates on an increasingly shorter suffix of the original list.

Recursive manipulation. `len_link`, `getitem_link`, and `sum_link` are iterative. They peel away each `Link` until the end of the list (in `len_link` and `sum_link`) or the desired item (in `getitem_link`) is reached. We can also implement all three using recursion.

```

>>> four = Link(1, Link(2, Link(3, Link(4))))
>>> print(four)
(1 2 3 4)
>>> def len_link_recursive(s: LinkedList) -> int:
    """Return the length of a linked list s."""
    if not isinstance(s, Link):
        return 0
    return 1 + len_link_recursive(s.rest)
>>> len_link_recursive(four)
4

>>> def getitem_link_recursive(s: LinkedList, i: int):
    """Return the item at index i of linked list s."""
    assert isinstance(s, Link), 'Index out of range'
    if i == 0:
        return s.first
    return getitem_link_recursive(s.rest, i - 1)
>>> getitem_link_recursive(four, 1)
2

>>> def sum_link_recursive(s: LinkedList[float]) -> float:
    """Return the sum of the items in a linked list of numbers."""
    if not isinstance(s, Link):
        return 0
    return s.first + sum_link_recursive(s.rest)
>>> sum_link_recursive(s)
12

```

Recursion is also useful for transforming and combining linked lists.

```

>>> def extend_link(s: LinkedList, t: LinkedList) -> LinkedList:
    """Return a linked list with the items of s followed by those of t."""
    if not isinstance(s, Link):
        return t
    else:
        return Link(s.first, extend_link(s.rest, t))

>>> print(extend_link(four, four))
(1 2 3 4 1 2 3 4)

>>> def apply_to_all_link(f, s: LinkedList) -> LinkedList:
    """Return a linked list of f applied to each item of s."""
    if not isinstance(s, Link):
        return s
    else:
        return Link(f(s.first), apply_to_all_link(f, s.rest))

>>> print(apply_to_all_link(lambda x: x*x, four))
(1 4 9 16)

>>> def keep_if_link(f, s: LinkedList) -> LinkedList:
    """Return a linked list with the items of s for which f returns a true value."""
    if not isinstance(s, Link):
        return s
    else:
        kept = keep_if_link(f, s.rest)
        if f(s.first):
            return Link(s.first, kept)
        else:
            return kept

>>> print(keep_if_link(lambda x: x%2 == 0, four))
(2 4)

>>> def join_link(s: LinkedList, separator: str) -> str:
    """Return a string of all items in s separated by separator."""
    if not isinstance(s, Link):
        return ""
    elif not isinstance(s.rest, Link):
        return str(s.first)
    else:
        return str(s.first) + separator + join_link(s.rest, separator)

>>> print(join_link(four, ", "))
1, 2, 3, 4

```

Recursive construction. Linked lists are particularly useful when constructing sequences incrementally, a situation that arises often in recursive computations.

The `count_partitions` function from Chapter 1 counted the number of ways to partition an integer `n` using parts up to size `m` via a tree-recursive process. With sequences, we can also enumerate these partitions explicitly using a similar process.

We follow the same recursive analysis of the problem as we did while counting: partitioning `n` using integers up to `m` involves either

1. partitioning `n-m` using integers up to `m`, or
2. partitioning `n` using integers up to `m-1`.

For base cases, we find that 0 has one partition that is empty, while partitioning a negative integer or using parts smaller than 1 is impossible.

```

>>> def partitions(n: int, m: int) -> list[LinkedList[int]]:
    """Return a list of partitions of n using parts of up to m.
    Each partition is represented as a linked list.
    """
    if n == 0:
        return [()] # A list containing the empty partition
    elif n < 0 or m == 0:
        return []
    else:
        with_m = [Link(m, s) for s in partitions(n-m, m)]
        without_m = partitions(n, m-1)
        return with_m + without_m

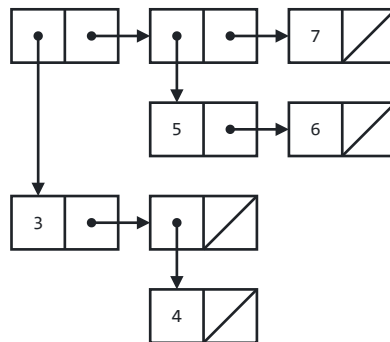
```

In the recursive case, we construct two sublists of partitions. The first uses `m`, and so we prepend `m` to each item returned by the recursive call `partitions(n-m, m)` to form `with_m`. This use of linked lists is more efficient than using native Python lists because prepending can be achieved just by constructing one new `Link`.

The result of `partitions` is a (regular Python) list of linked lists. Using `join_link`, we can display each partition in a human-readable manner.

```
>>> def print_partitions(n: int, m: int) -> None:
    "Print the partitions of n using parts up to size m."
    for p in partitions(n, m):
        print(join_link(p, " + "))
>>> print_partitions(6, 4)
4 + 2
4 + 1 + 1
3 + 3
3 + 2 + 1
3 + 1 + 1 + 1
2 + 2 + 2
2 + 2 + 1 + 1
2 + 1 + 1 + 1 + 1
1 + 1 + 1 + 1 + 1 + 1
```

Nested linked lists. The `first` item in a linked list can be any object, including another linked list. Nested structures can be created by placing linked lists within linked lists, and box-and-pointer diagrams are useful for visualizing these structures. Here is the diagram for `Link(Link(3, Link(Link(4))), Link(Link(5, Link(6)), Link(7)))`, which would be printed as `((3 (4)) (5 6) 7)`.



Trees

Nesting lists within lists is a way to create hierarchical structures, but the *tree* is a fundamental data abstraction that imposes regularity on how hierarchical values are structured and manipulated.

A tree has a root label (which can be any value) and a sequence of branches. Each branch of a tree is a tree. A tree with no branches is called a leaf. Any tree contained within a tree is called a sub-tree of that tree (such as a branch of a branch). The root of each sub-tree of a tree is called a node in that tree.

Here is a simple data class for a `Tree`.

```
>>> from __future__ import annotations # for Python 3.12 and 3.13
>>> from dataclasses import field
>>> @dataclass
    class Tree[T]:
        "A Tree has a label (of type T) and a list of branches, which are trees."
        label: T
        branches: list[Tree[T]] = field(default_factory=list) # branches defaults to []
```

For Python versions 3.12 and 3.13, executing `from __future__ import annotations` before this definition is necessary to allow a `Tree` to contain a `list[Tree[T]]` attribute using this type hint syntax. For Python 3.14 and later, this import is unnecessary.

The complicated expression `field(default_factory=list)` is required to provide a default empty list value for the `branches` attribute, rather than just `[]`, due to the concept of mutation that will be discussed in Section 2.4. This expression ensures that each leaf gets its own empty list of branches rather than sharing the same list with other `Tree` objects.

Trees can be constructed by nested expressions. The following `Tree` object `t` has root label 3 and two branches.

```
>>> t = Tree(3, [Tree(1), Tree(2, [Tree(1), Tree(1)])])
>>> t.branches[0]
Tree(label=1, branches=[])
>>> len(t.branches)
2
>>> t.branches[1].label
2
>>> len(t.branches[1].branches)
2
```

Here we introduce a string representation that shows all of the labels within the tree indented to show how deep within the tree's branches they appear. The newly added `__str__` method of the `Tree` class ensures that `format_tree` will be called whenever a `Tree` object is printed or passed to `str`.

```
>>> def format_tree(t: Tree, indent='') -> str:
    "Format a tree with each branch indented below its label."
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + ' ')
    return string

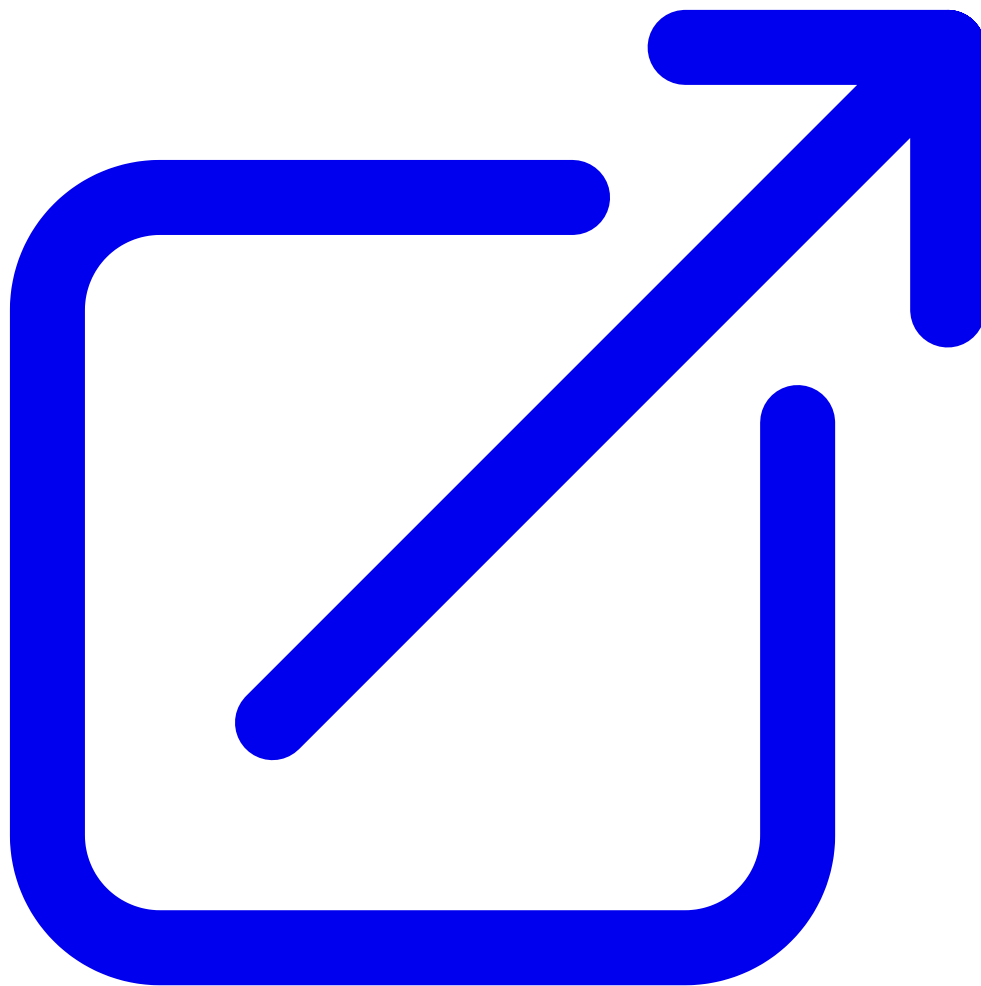
>>> @dataclass
class Tree[T]:
    "A Tree has a label (of type T) and a list of branches, which are trees."
    label: T
    branches: list[Tree[T]] = field(default_factory=list) # branches defaults to []

    def __str__(self):
        return format_tree(self)
```

The `is_leaf` function takes a `Tree` and returns whether it is a leaf. It makes programs that use trees easier to follow.

```
def is_leaf(t: Tree) -> bool:
    "Return whether a Tree t is a leaf with no branches."
    return not t.branches
```

The complete [Tree class](#)



appears in the examples appendix.

The example below constructs a tree representing the evaluation of the expression `((2 * 3) + 4)`.

```
>>> expr = Tree(10, [Tree(6, [Tree(2), Tree('*'), Tree(3)]), Tree('+'), Tree(4)])
>>> print(expr)
10
 6
 2
 *
 3
 +
 4
```

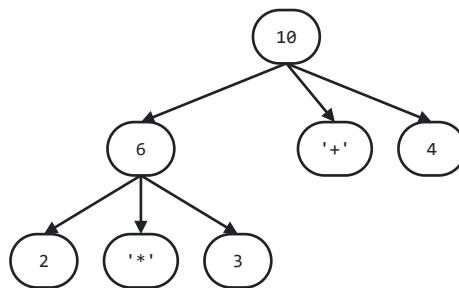
The Tree Abstraction¶

Trees are used to represent a variety of hierarchical structures, such as the organizational charts of institutions, the structure of web pages, and even Python code. There are two complementary and compatible ways to view a tree. The recursive description matches our `Tree` class and uses arboreal terminology: trees have branches and leaves.

A tree can also be described as a collection of *nodes*, which are the `Tree` objects. Each node has a label and relations to other nodes. Genealogical terminology applies to this description: the parent of a node is the `Tree` that contains it as a branch, and the children of a node are the `Tree` objects in its list of branches. The ancestors of a node include its parent and the parent of any ancestor. The descendants of a node include its children and any children of its descendants.

Both ways of describing a tree use the term *root* to describe the `Tree` object that contains the entire tree, and the *root label* is the label of that object.

A tree is typically visualized by placing each label in a node and drawing an arrow to each child node. The root node is at the top.



Tree-Recursive Functions¶

Tree-recursive functions can be used to construct trees. For example, the *nth* Fibonacci tree has a root label of the *nth* Fibonacci number and, for $n > 1$, two branches that are also Fibonacci trees. A Fibonacci tree illustrates the tree-recursive computation of a Fibonacci number.

```
>>> def fib_tree(n: int) -> Tree[int]:
    if n == 0 or n == 1:
        return Tree(n)
    else:
        left, right = fib_tree(n-2), fib_tree(n-1)
        fib_n = left.label + right.label
        return Tree(fib_n, [left, right])
>>> print(fib_tree(5))
5
 2
 1
 1
 0
 1
 3
 1
 0
 1
 2
 1
 1
 0
 1
```

This `fib_tree(5)` object can be described recursively: a Fibonacci tree has a Fibonacci number as its label and two Fibonacci trees as its branches. Its contents can also be described in terms of its node relations: the label of each node in a Fibonacci tree is the sum of the labels of its

children.

Tree-recursive functions are also used to process trees. For example, the `sum_leaves` function sums the labels of the leaves of a tree.

```
>>> def sum_leaves(t: Tree) -> int:
    if is_leaf(t):
        return t.label
    else:
        branch_sums = [sum_leaves(b) for b in t.branches]
        return sum(branch_sums)
>>> sum_leaves(Tree(5, [Tree(100), Tree(1000, [Tree(3), Tree(20)])]))
123
>>> sum_leaves(fib_tree(5))
5
>>> sum_leaves(fib_tree(6))
8
```

Footnotes¹

1. The `f'{p!r} ...'` syntax directs the f-string to use the built-in representation for a `Position` rather than the one defined by `format_pos`. This directive ensures that generating the error message within `format_pos` won't trigger another call to `format_pos` once we install `format_pos` as the function that generates the output of calling `str` or `print` on a `Position`.

[↩](#)

[←](#)

[Chapter 2: Data](#)

[2.2 Sequences](#)

[Chapter 2: Data](#)

[2.4 Mutable Data](#)

[→](#)

2.4 Mutable Data



2.4 Mutable Data

```
from pytutor import env_diagram
```

We have seen how abstraction is vital in helping us to cope with the complexity of large systems. Effective programming also requires organizational principles that can guide us in formulating the overall design of a program. In particular, we need strategies to help us structure large systems to be modular, meaning that they divide naturally into coherent parts that can be separately developed and maintained.

One powerful technique for creating modular programs is to incorporate data that may change state over time. In this way, a single data object can represent something that evolves independently of the rest of the program. The behavior of a changing object may be influenced by its history, just like an entity in the world. Adding state to data is a central ingredient of a paradigm called object-oriented programming.

Mutable Containers

Instances of primitive built-in values such as numbers are *immutable*. The values themselves cannot change over the course of program execution. Lists and dictionaries, on the other hand, are *mutable*.

Mutable objects can be used to represent values that change over time. A person is the same person from one day to the next, despite having aged, received a haircut, or otherwise changed in some way. Similarly, an object may have changing properties due to *mutation* operations. For example, it is possible to change the contents of a list. Most changes are performed by invoking methods on list objects.

We can introduce many list modification operations through an example that illustrates the history of playing cards (drastically simplified). Comments in the examples describe the effect of each method invocation.

Playing cards were invented in China, perhaps around the 9th century. An early deck had three suits, which corresponded to denominations of money.

```
>>> chinese = ['coin', 'string', 'myriad'] # A list literal
>>> suits = chinese # Two names refer to the same list
```

As cards migrated to Europe (perhaps through Egypt), only the suit of coins remained in Spanish decks (*oro*).

```
>>> suits.pop() # Remove and return the final item
'myriad'
>>> suits.remove('string') # Remove the first item that equals the argument
```

Three more suits were added (they evolved in name and design over time),

```
>>> suits.append('cup') # Add an item to the end
>>> suits.extend(['sword', 'club']) # Add all items of a sequence to the end
```

and Italians called swords *spades*.

```
>>> suits[2] = 'spade' # Replace an item
```

giving the suits of a traditional Italian deck of cards.

```
>>> suits
['coin', 'cup', 'spade', 'club']
```

The French variant used today in the U.S. changes the first two suits:

```
>>> suits[0:2] = ['heart', 'diamond'] # Replace a slice
>>> suits
['heart', 'diamond', 'spade', 'club']
```

Methods also exist for inserting, sorting, and reversing lists. All of these mutation operations change the value of the list; they do not create new list objects.

Sharing and Identity. Because we have been changing a single list rather than creating new lists, the object bound to the name `chinese` has also changed, because it is the same list object that was bound to `suits` !

```
>>> chinese # This name co-refers with "suits" to the same changing list
['heart', 'diamond', 'spade', 'club']
```

This behavior is new. Previously, if a name did not appear in a statement, then its value would not be affected by that statement. With mutable data, methods called on one name can affect another name at the same time.

The environment diagram for this example shows how the value bound to `chinese` is changed by statements involving only `suits`. Step through each line of the following example to observe these changes.

```
env_diagram(r'''
chinese = ['coin', 'string', 'myriad']
suits = chinese
suits.pop()
suits.remove('string')
suits.append('cup')
suits.extend(['sword', 'club'])
suits[2] = 'spade'
suits[0:2] = ['heart', 'diamond']
''', step=3, cumulative=False)
```

Loading...

Lists can be copied using the `list` constructor function. Changes to one list do not affect another, unless they share structure.

```
>>> nest = list(suits) # Bind "nest" to a second list with the same items
>>> nest[0] = suits    # Create a nested list
```

According to this environment, changing the list referenced by `suits` will affect the nested list that is the first item of `nest`, but not the other items.

```
>>> suits.insert(2, 'Joker') # Insert an item at index 2, shifting the rest
>>> nest
[['heart', 'diamond', 'Joker', 'spade', 'club'], 'diamond', 'spade', 'club']
```

And likewise, undoing this change in the first item of `nest` will change `suits` as well.

```
>>> nest[0].pop(2)
'Joker'
>>> suits
['heart', 'diamond', 'spade', 'club']
```

Stepping through this example line by line will show the representation of a nested list.

```
env_diagram(r'''
suits = ['heart', 'diamond', 'spade', 'club']
nest = list(suits)
nest[0] = suits
suits.insert(2, 'Joker')
joke = nest[0].pop(2)
''', step=0, cumulative=False)
```

Loading...

Because two lists may have the same contents but in fact be different lists, we require a means to test whether two objects are the same. Python includes two comparison operators, called `is` and `is not`, that test whether two expressions in fact evaluate to the identical object. Two objects are identical if they are equal in their current value, and any change to one will always be reflected in the other. Identity is a stronger condition than equality.

```
>>> suits is nest[0]
True
>>> suits is ['heart', 'diamond', 'spade', 'club']
False
>>> suits == ['heart', 'diamond', 'spade', 'club']
True
```

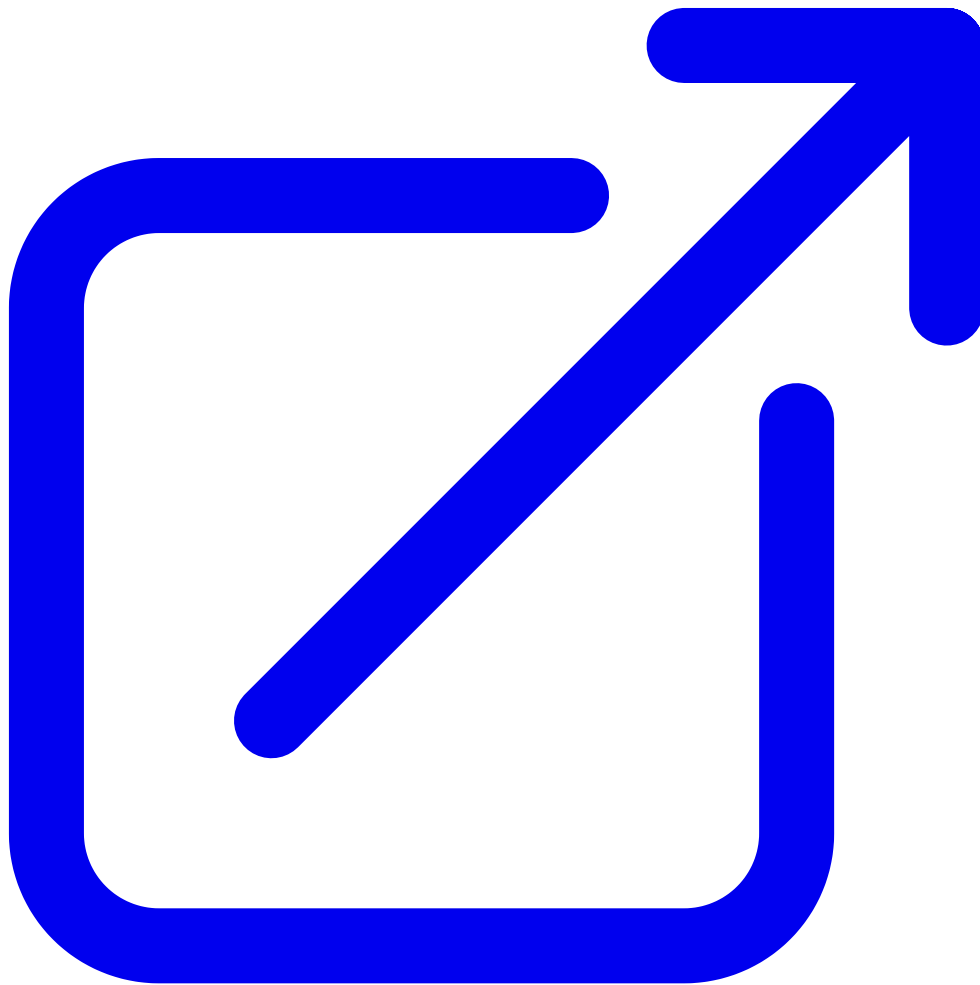
The final two comparisons illustrate the difference between `is` and `==`. The former checks for identity, while the latter checks for the equality of contents.

List comprehensions. A list comprehension always creates a new list. For example, the `unicodedata` module tracks the official names of every character in the Unicode alphabet. We can look up the characters corresponding to names, including those for card suits.

```
>>> from unicodedata import lookup
>>> [lookup('WHITE ' + s.upper()) + ' SUIT') for s in suits]
['♥', '♦', '♠', '♣']
```

This resulting list does not share any of its contents with `suits`, and evaluating the list comprehension does not modify the `suits` list.

You can read more about the Unicode standard for representing text in the [Unicode section](#)



of Dive into Python 3.

Tuples. Tuples are immutable sequences. A new tuple can be created based on an existing one, but modifications cannot be made to an existing tuple's length or the identity of the items it contains.

While it is not possible to change which items are in a tuple, it is possible to change the value of a mutable item contained within a tuple.

```
env_diagram(r'''
nest = (10, 20, [30, 40])
nest[2].pop()
''', cumulative=False)
```

Loading...

Tuples are used implicitly in multiple assignment. An assignment of two values to two names creates a two-item tuple and then unpacks it.

Dictionaries. Dictionaries are mutable. Adding new key-value pairs and changing the existing value for a key can both be achieved with assignment statements.

```
>>> numerals = {'I': 'one', 'V': 5, 'X': 10}
>>> numerals['I'] = 1
>>> numerals['L'] = 50
>>> numerals
{'I': 1, 'V': 5, 'X': 10, 'L': 50}
>>> numerals.pop('X')
10
>>> numerals
{'I': 1, 'V': 5, 'L': 50}
```

Notice that 'I' kept its position in the output above even though its value changed, while the new key 'L' was added to the end. Dictionaries preserve insertion order: when we print a dictionary, keys appear in the order in which they were first added, and assigning a new value to an existing key does not change its position.

Local State¹

Lists and dictionaries have *local state*: they are changing values that have some particular contents at any point in the execution of a program. The word “state” implies an evolving process in which that state may change.

Functions can also have local state, which is stored in their parent frame. For instance, let us define a function that models the process of withdrawing money from a bank account. We will create a function called `withdraw`, which takes as its argument an amount to be withdrawn. If there is enough money in the account to accommodate the withdrawal, then `withdraw` will return the amount remaining after the withdrawal. Otherwise, `withdraw` will return the message `'Insufficient funds'`. For example, if we begin with \$100 in the account, we would like to obtain the following sequence of return values by calling `withdraw`:

```
>>> withdraw(25)
75
>>> withdraw(25)
50
>>> withdraw(60)
'Insufficient funds'
>>> withdraw(15)
35
```

Above, the expression `withdraw(25)`, evaluated twice, yields different values. Thus, this user-defined function is non-pure. Calling the function not only returns a value, but also has the side effect of changing the function in some way, so that the next call with the same argument will return a different result. This side effect is a result of `withdraw` changing the contents of a list that is bound to a name outside of the current frame.

For `withdraw` to make sense, it must be created with an initial account balance. The function `make_withdraw` is a higher-order function that takes a starting balance as an argument. The function `withdraw` is its return value.

```
>>> withdraw = make_withdraw(100)
```

The implementation of `make_withdraw` below uses a one-element mutable list called `state` to store the state of `withdraw`. When we call `make_withdraw`, we place the `initial_balance` in the `state` list. We then define and return a local function, `withdraw`, which updates and returns the contents of `state` when called.

```
>>> def make_withdraw(initial_balance):
    """Return a withdraw function whose state starts at initial_balance and decreases with each call."""
    state = [initial_balance]
    def withdraw(amount):
        if amount > state[0]:
            return 'Insufficient funds'
        state[0] = state[0] - amount
        return state[0]
    return withdraw
```

The following environment diagrams illustrate the effects of multiple calls to a function created by `make_withdraw`.

```
env_diagram(r'''
def make_withdraw(initial_balance):
    state = [initial_balance]
    def withdraw(amount):
        if amount > state[0]:
            return 'Insufficient funds'
        state[0] = state[0] - amount
        return state[0]
    return withdraw

wd = make_withdraw(20)
wd(5)
wd(3)
''' , step=6, cumulative=True)
```

Loading...

The first `def` statement has the usual effect: it creates a new user-defined function and binds the name `make_withdraw` to that function in the global frame. The subsequent call to `make_withdraw` creates and returns a locally defined function `withdraw`, which is bound to the name `wd` in the global frame. In the `make_withdraw` frame, the name `state` is bound to a list containing the starting balance 20. Crucially, there will only be this single binding for the name `state` throughout the rest of this example.

Next, we evaluate an expression that calls this function, bound to the name `wd`, on an amount 5. The body of `withdraw` is executed in a new environment that extends the environment in which `withdraw` was defined.

```
env_diagram(r'''
def make_withdraw(initial_balance):
    state = [initial_balance]
    def withdraw(amount):
        if amount > state[0]:
            return 'Insufficient funds'
        state[0] = state[0] - amount
        return state[0]
    return withdraw

wd = make_withdraw(20)
wd(5)
wd(3)
''' , step=9, cumulative=True)
```

Loading...

Instead of using `initial_balance`, the `withdraw` function uses and updates `state[0]`. An assignment statement with `state[0]` on the left-hand side of `=` does not bind the name `state`. Instead, it looks up `state` in the current environment, finds it in the `make_withdraw` frame, and changes the contents of that list. Updating an item of a list is allowed, even if the name for that list does not appear in the local frame but instead in some non-local or global frame of the current environment.

By virtue of changing the contents of `state`, we have changed the `withdraw` function as well. The next time it is called, `state[0]` will evaluate to 15 instead of 20. Hence, when we call `withdraw` a second time, we see that its return value is 12 and not 17. The change to `state` from the first call affects the result of the second call.

```
env_diagram(r'''
def make_withdraw(initial_balance):
    state = [initial_balance]
    def withdraw(amount):
        if amount > state[0]:
            return 'Insufficient funds'
        state[0] = state[0] - amount
        return state[0]
    return withdraw

wd = make_withdraw(20)
wd(5)
wd(3)
''' , cumulative=True)
```

Loading...

The second call to `withdraw` does create a second local frame, as usual. However, both `withdraw` frames have the same parent. That is, they both extend the environment for `make_withdraw`, which contains the binding for `state`. Hence, they share the contents of this list. Calling `withdraw` has the side effect of altering the environment that will be extended by future calls to `withdraw`. Using a mutable value in the parent allows `withdraw` to change its future behavior.

Python Particulars. Python has an unusual restriction regarding the lookup of names: within the body of a function, all instances of a name must refer to the same frame. As a result, Python cannot look up the value of a name in a non-local frame, then bind that same name in the local frame, because the same name would be accessed in two different frames in the same function. This restriction allows Python to pre-compute which frame contains each name before executing the body of a function. When this restriction is violated, a confusing error message results. To demonstrate, the `make_withdraw` example is repeated below without a `state` list.

```
env_diagram(r'''
def make_withdraw(initial_balance):
    balance = initial_balance
    def withdraw(amount):
        if amount > balance:
            return 'Insufficient funds'
        balance = balance - amount
        return balance
    return withdraw

wd = make_withdraw(20)
wd(5)
''' , step=8, cumulative=True)
```

Loading...

This `UnboundLocalError` appears because `balance` is assigned locally in line 6, and so Python assumes that all references to `balance` must appear in the local frame as well. This error occurs *before* line 6 is ever executed, implying that Python has considered line 6 in some way before

executing line 4. Storing the balance in a `state` list avoids this error, because `withdraw` never changes what object `state` refers to, instead it only changes the contents of that object.

The Benefits of Mutation¶

Writing a function that updates its own state is a first step toward viewing a complex program as a collection of independent and autonomous *objects*, which interact with each other but each manage their own internal state.

List mutation has given us the ability to maintain some state that is local to a function, but evolves over successive calls to that function. The `state` associated with a particular `withdraw` function is shared among all calls to that function. However, this state associated with a particular `withdraw` function is inaccessible to the rest of the program. Only `wd` is associated with the frame for `make_withdraw` in which it was defined. If `make_withdraw` is called again, then it will create a separate frame with a separate binding for `state`.

We can extend our example to illustrate this point. A second call to `make_withdraw` returns a second `withdraw` function that has a different parent. We bind this second function to the name `wd2` in the global frame.

```
env_diagram(r'''
def make_withdraw(initial_balance):
    state = [initial_balance]
    def withdraw(amount):
        if amount > state[0]:
            return 'Insufficient funds'
        state[0] = state[0] - amount
        return state[0]
    return withdraw

wd = make_withdraw(20)
wd2 = make_withdraw(7)
wd2(6)
wd(8)
''', step=11, cumulative=True)
```

Loading...

Now, we see that there are in fact two bindings for the name `state` in two different frames, and each `withdraw` function has a different parent. The name `wd` is bound to a function whose `state` list contains 20, while `wd2` is bound to a different function whose `state` list contains 7.

Calling `wd2` changes the contents of the `state` list in its parent frame, but does not affect the function bound to the name `wd`. As a result, calling `wd` uses a `state` list that still contains 20.

```
env_diagram(r'''
def make_withdraw(initial_balance):
    state = [initial_balance]
    def withdraw(amount):
        if amount > state[0]:
            return 'Insufficient funds'
        state[0] = state[0] - amount
        return state[0]
    return withdraw

wd = make_withdraw(20)
wd2 = make_withdraw(7)
wd2(6)
wd(8)
''', step=16, cumulative=True)
```

Loading...

In this way, each instance of `withdraw` maintains its own state, but that state is inaccessible to any other function in the program. Viewing this situation at a higher level, we have created an abstraction of a bank account that manages its own internals but behaves in a way that models accounts in the world: it changes over time based on its own history of withdrawal requests.

The Cost of Mutation¶

Our environment model of computation cleanly extends to explain the effects of mutation. However, mutation introduces some important nuances in the way we think about names and values.

Previously, our values did not change; only our names and bindings changed. When two names `a` and `b` were both bound to the value 4, it did not matter whether they were bound to the same 4 or different 4's. As far as we could tell, there was only one 4 object that never changed.

However, functions with state do not behave this way. When two names `wd` and `wd2` are both bound to a `withdraw` function, it *does* matter whether they are bound to the same function or different instances of that function. Consider the following example, which contrasts the one we

just analyzed. In this case, calling the function named by `wd2` did change the state of the function named by `wd`, because both names refer to the same function.

```
env_diagram(r'''
def make_withdraw(initial_balance):
    state = [initial_balance]
    def withdraw(amount):
        if amount > state[0]:
            return 'Insufficient funds'
        state[0] = state[0] - amount
        return state[0]
    return withdraw

wd = make_withdraw(12)
wd2 = wd
wd2(1)
wd(1)
''', cumulative=True)
```

Loading...

It is not unusual for two names to co-refer to the same value in the world, and so it is in our programs. But, as values change over time, we must be very careful to understand the effect of a change on other names that might refer to those values.

Sameness and change. These subtleties arise because, by introducing non-pure functions that change values referenced from their parent frames, we have changed the nature of expressions. An expression that contains only pure function calls is *referentially transparent*; its value does not change if we substitute one of its subexpressions with the value of that subexpression.

Mutation operations violate the conditions of referential transparency because they do more than return a value; they change the environment. When we introduce mutation, we encounter a thorny epistemological issue: what it means for two values to be the same. In our environment model of computation, two separately defined functions are not the same, because changes to one may not be reflected in the other.

In general, so long as we never modify data objects, we can regard a compound data object to be precisely the totality of its pieces. For example, a geographic position is determined by giving its latitude and longitude. But this view is no longer valid in the presence of change, where a compound data object has an “identity” that is something different from the pieces of which it is composed. A bank account is still “the same” bank account even if we change the balance by making a withdrawal; conversely, we could have two bank accounts that happen to have the same balance, but are different objects.

Despite the complications it introduces, mutation is a powerful tool for creating modular programs. Different parts of a program, which correspond to different environment frames, can evolve separately throughout program execution.



[Chapter 2: Data](#)

[2.3 Compound Data](#)

[Chapter 2: Data](#)

[2.5 Object-Oriented Programming](#)



2.5 Object-Oriented Programming



2.5 Object-Oriented Programming

Object-oriented programming (OOP) is a method for organizing programs that brings together many of the ideas introduced in this chapter. Objects have types that determine their behavior (Section 2.1), objects can contain other objects (Section 2.2), class statements can introduce new types that bundle together data and behavior (Section 2.3), and objects have local mutable state (Section 2.4). Two final components of OOP remain: customizing object behavior and sharing behavior among related types. Together, these ideas allow programs to be organized around creating and manipulating objects and types. Python provides convenient syntax to promote the use of these techniques, and much of this syntax is shared among other object-oriented programming languages.

Object syntax offers more than just convenience. It enables a new metaphor for designing programs in which several independent components interact within the computer. Objects bundle together local state and behavior in a way that provides an abstraction over the details of how the hardware of a computer or the language interpreter stores and manipulates data. Objects communicate with each other, and useful results are computed as a consequence of their interaction. Objects can also share behavior among other objects of the same type and inherit behavior from related types.

Objects and Classes

A class serves as a template for all objects whose type is that class. Every object is an instance of some class. User-defined classes can customize many details of how their instances behave. The `@dataclass` decorator for a class provides a reasonable default behavior for how objects behave, but we will now switch to defining every aspect of a class ourselves. A class definition specifies the attributes and methods shared among objects of that class. We can start learning about the details of class definition by revisiting the example of a bank account.

When introducing local state, we saw that bank accounts are naturally modeled as mutable values that have a `balance`. A bank account object should have a `withdraw` method that updates the account balance and returns the requested amount, if it is available. To complete the abstraction: a bank account should be able to return its current `balance`, return the name of the account `holder`, and an amount for `deposit`.

An `Account` class allows us to create multiple instances of bank accounts. The act of creating a new object instance is known as *instantiating* the class. The syntax in Python for instantiating a class is identical to the syntax of calling a function. In this case, we call `Account` with the argument `'Kirk'`, the account holder's name.

```
>>> a = Account('Kirk')
```

An *attribute* of an object is a name-value pair associated with the object, which is accessible via dot notation. The attributes specific to a particular object, as opposed to all objects of a class, are called *instance attributes*. Each `Account` has its own balance and account holder name, which are examples of instance attributes. In the broader programming community, instance attributes may also be called *fields*, *member variables*, or *instance variables*.

```
>>> a.holder
'Kirk'
>>> a.balance
0
```

Functions that operate on the object or perform object-specific computations are called methods. The return values and side effects of a method can depend upon and change other attributes of the object. For example, `deposit` is a method of our `Account` object `a`. It takes one argument, the amount to deposit, changes the `balance` attribute of the object, and returns the resulting balance.

```
>>> a.deposit(15)
15
```

We say that methods are *invoked* on a particular object. As a result of invoking the `withdraw` method, either the withdrawal is approved and the amount is deducted, or the request is declined and the method returns an error message.

```
>>> a.withdraw(10) # The withdraw method returns the balance after withdrawal
5
>>> a.balance     # The balance attribute has changed
5
>>> a.withdraw(10)
'Insufficient funds'
```

As illustrated above, the behavior of a method can depend upon the changing attributes of the object. Two calls to `withdraw` with the same argument return different results.

Defining Classes

User-defined classes are created by `class` statements, which define the class name, then include a suite of statements to define the attributes and methods of the class:

```
class <name>:  
    <suite>
```

When a class statement is executed, a new class is created and bound to `<name>` in the first frame of the current environment. The suite is then executed. Any names bound within the `<suite>` of a `class` statement, through `def` or assignment statements, create or modify attributes of the class.

Classes are typically organized around manipulating instance attributes, which are the name-value pairs associated with each instance of that class. The class specifies the instance attributes of its objects by defining a method for initializing new objects. For example, part of initializing an object of the `Account` class is to assign it a starting balance of 0.

The `<suite>` of a `class` statement contains `def` statements that define new methods for objects of that class. The method that initializes objects has a special name in Python, `__init__` (two underscores on each side of the word "init"), and is called the *constructor* for the class.

```
>>> class Account:  
    def __init__(self, account_holder):  
        self.balance = 0  
        self.holder = account_holder
```

The `__init__` method for `Account` has two formal parameters. The first one, `self`, is bound to the newly created `Account` object. The second parameter, `account_holder`, is bound to the argument passed to the class when it is called to be instantiated.

The constructor binds the instance attribute name `balance` to 0. It also binds the attribute name `holder` to the argument named `account_holder`. The formal parameter `account_holder` is a local name in the `__init__` method. On the other hand, the name `holder` that is bound via the final assignment statement persists, because it is stored as an attribute of `self` using dot notation.

Having defined the `Account` class, we can instantiate it.

```
>>> a = Account('Kirk')
```

This "call" to the `Account` class creates a new object that is an instance of `Account`, then calls the constructor function `__init__` with two arguments: the newly created object and the string `'Kirk'`. By convention, we use the parameter name `self` for the first argument of a constructor, because it is always bound to the object being instantiated. This convention is adopted in virtually all Python code.

Now, we can access the object's `balance` and `holder` using dot notation.

```
>>> a.balance  
0  
>>> a.holder  
'Kirk'
```

Unlike a class decorated with `@dataclass`, a standard class statement contains no fixed list of instance attributes. Instead, it is up to `__init__` to add instance attributes to a newly constructed object, and different instances of the same class could have different instance attributes, although it is most common that `__init__` will add the same attributes to every instance of the class.

Identity. Each new account instance has its own `balance` attribute, the value of which is independent of other objects of the same class.

```
>>> b = Account('Spock')  
>>> b.balance = 200  
>>> [acc.balance for acc in (a, b)]  
[0, 200]
```

To enforce this separation, every object that is an instance of a user-defined class has a unique identity. Object identity is compared using the `is` and `is not` operators.

```
>>> a is a  
True  
>>> a is not b  
True
```

Binding an object to a new name using assignment does not create a new object. Instead, it gives an additional name to an existing object in the current environment.

```
>>> c = a  
>>> c is a  
True
```

Creating another `Account` instance would require calling `Account`.

Methods. Object methods are also defined by a `def` statement in the suite of a `class` statement. Below, `deposit` and `withdraw` are both defined as methods on objects of the `Account` class.

```
>>> class Account:
    def __init__(self, account_holder):
        self.balance = 0
        self.holder = account_holder
    def deposit(self, amount):
        self.balance = self.balance + amount
        return self.balance
    def withdraw(self, amount):
        if amount > self.balance:
            return 'Insufficient funds'
        self.balance = self.balance - amount
        return self.balance
```

While method definitions do not differ from standard function definitions in how they are declared, the fact that they appear within a `class` statement means that they become methods of the class. Methods can then be invoked on an instance of the class using dot notation.

Each method definition again includes a special first parameter `self`, which is bound to the object on which the method is invoked. For example, let us say that `deposit` is invoked on a particular `Account` object and passed a single `int` argument: the amount deposited. The object itself is bound to `self`, while the argument is bound to `amount`. All invoked methods have access to the object via the `self` parameter, and so they can all access and manipulate the object's state.

To invoke these methods, we again use dot notation, as illustrated below.

```
>>> spock_account = Account('Spock')
>>> spock_account.deposit(100)
100
>>> spock_account.withdraw(90)
10
>>> spock_account.withdraw(90)
'Insufficient funds'
>>> spock_account.holder
'Spock'
```

When a method is invoked via dot notation, the object itself (bound to `spock_account`, in this case) plays a dual role. First, it determines what the name `withdraw` means; `withdraw` is not a name in the environment, but instead a name that is local to the `Account` class. Second, it is bound to the first parameter `self` when the `withdraw` method is called.

Message Passing and Dot Expressions

Methods, which are defined in classes, and instance attributes, which are typically initialized in constructors, are the fundamental elements of object-oriented programming. A useful metaphor is that invoking a method is sending a message to the object, and the return value is its response. Classes define which messages its instances can receive (which methods are available). The central idea in this metaphor of message passing is that objects should have behavior by responding to messages that are relevant to the abstract type they represent.

Dot expressions. The code fragment `spock_account.deposit` is called a *dot expression*. A dot expression consists of an expression, a dot, and a name:

```
<expression> . <name>
```

The `<expression>` can be any valid Python expression, but the `<name>` must be a simple name (not an expression that evaluates to a name). A dot expression evaluates to the value of the attribute with the given `<name>`, for the object that is the value of the `<expression>`.

The built-in function `getattr` also returns an attribute for an object by name. It is the function equivalent of dot notation. Using `getattr`, we can look up an attribute using a string.

```
>>> getattr(spock_account, 'balance')
10
```

We can also test whether an object has a named attribute with `hasattr`.

```
>>> hasattr(spock_account, 'deposit')
True
```

The attributes of an object include all of its instance attributes, along with all of the attributes (including methods) defined in its class. Methods are attributes of the class that require special handling.

Methods and functions. When a method is invoked on an object, that object is implicitly passed as the first argument to the method. That is, the object that is the value of the `<expression>` to the left of the dot is passed automatically as the first argument to the method named on the right side of the dot expression. As a result, the object is bound to the parameter `self`.

To achieve automatic `self` binding, Python distinguishes between *functions*, which we have been creating since the beginning of the text, and *bound methods*, which couple together a function and the object on which that method will be invoked. A bound method value is already associated with its first argument, the instance on which it was invoked, which will be named `self` when the method is called.

We can see the difference in the interactive interpreter by calling `type` on the returned values of dot expressions. As an attribute of a class, a method is just a function, but as an attribute of an instance, it is a bound method:

```
>>> type(Account.deposit)
<class 'function'>
>>> type(spock_account.deposit)
<class 'method'>
```

These two results differ only in the fact that the first is a standard two-argument function with parameters `self` and `amount`. The second is a one-argument method, where the name `self` will be bound to the object named `spock_account` automatically when the method is called, while the parameter `amount` will be bound to the argument passed to the method. Both of these values, whether function values or bound method values, are associated with the same `deposit` function body.

We can call `deposit` in two ways: as a function and as a bound method. In the former case, we must supply an argument for the `self` parameter explicitly. In the latter case, the `self` parameter is bound automatically.

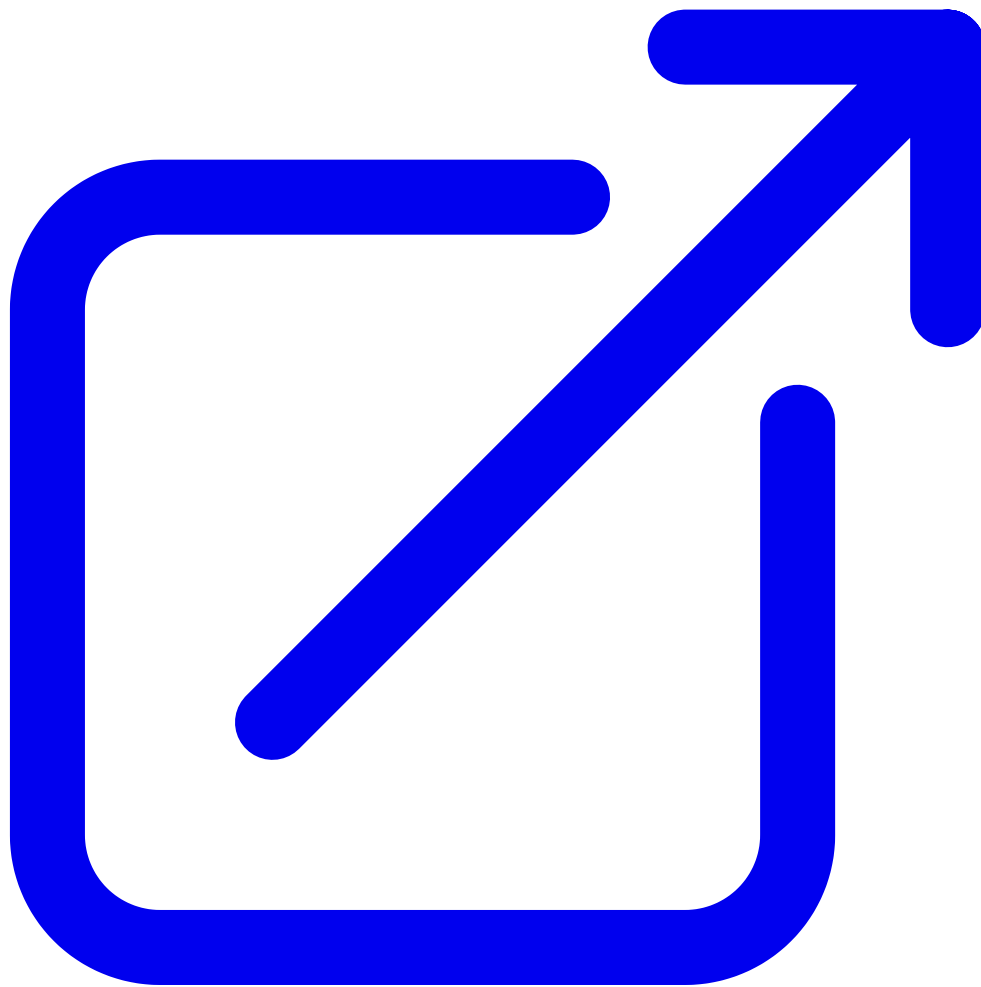
```
>>> Account.deposit(spock_account, 1001) # The deposit function takes 2 arguments
1011
>>> spock_account.deposit(1000)          # The deposit method takes 1 argument
2011
```

The function `getattr` behaves exactly like dot notation: if its first argument is an object but the name is a method defined in the class, then `getattr` returns a bound method value. On the other hand, if the first argument is a class, then `getattr` returns the attribute value directly, which is a plain function.

Naming Conventions. Class names are conventionally written using the CapWords convention (also called CamelCase because the capital letters in the middle of a name look like humps). Method names follow the standard convention of naming functions using lowercased words separated by underscores.

In some cases, there are instance attributes and methods that are related to the maintenance and consistency of an object that we don't want users of the object to see or use. They are not part of the abstraction defined by a class, but instead part of the implementation. Python's convention dictates that if an attribute name starts with an underscore, it should only be accessed within methods of the class itself, rather than by users of the class.

Attribute access. In some programming languages, accessing or changing attribute values directly (rather than invoking methods) is either discouraged or prohibited. Python also supports this programming style, described in the Python Tutorial section on [private variables](#)



, in which instance attributes are not accessed directly but instead via methods. But Python programs in the world often do access and update attributes directly.

Class Attributes¹

Some attribute values are shared across all instances of a class. Such attributes are associated with the class itself, rather than any individual instance of the class. For instance, let us say that a bank pays interest on the balance of accounts at a fixed interest rate. That interest rate may change, but it is a single value shared across all accounts.

Class attributes are created by assignment statements in the suite of a `class` statement, outside of any method definition. In the broader programming community, class attributes may also be called class variables or static fields. The following class statement creates a class attribute for `Account` with the name `interest`.

```
>>> class Account:
    interest = 0.02          # A class attribute
    def __init__(self, account_holder):
        self.balance = 0
        self.holder = account_holder
    # Additional methods would be defined here
```

This attribute can still be accessed from any instance of the class.

```
>>> spock_account = Account('Spock')
>>> kirk_account = Account('Kirk')
>>> spock_account.interest
0.02
>>> kirk_account.interest
0.02
```

However, a single assignment statement to a class attribute changes the value of the attribute for all instances of the class.

```
>>> Account.interest = 0.04
>>> spock_account.interest
0.04
>>> kirk_account.interest
0.04
```

Methods are also class attributes. They are functions shared by all instances of a class. However, the term class attribute is most often used to describe attributes that are not function-valued, and function-valued class attributes are called methods.

Attribute names. We now need to specify how names are resolved to particular attributes, since we could easily have a class attribute and an instance attribute with the same name.

As we have seen, a dot expression consists of an expression, a dot, and a name:

```
<expression> . <name>
```

To evaluate a dot expression:

1. Evaluate the `<expression>` to the left of the dot, which yields the *object* of the dot expression.
2. `<name>` is matched against the instance attributes of that object; if an attribute with that name exists, its value is returned.
3. If `<name>` does not appear among instance attributes, then `<name>` is looked up in the class, which yields a class attribute value.
4. That value is returned unless it is a function, in which case a bound method is returned instead.

In this evaluation procedure, instance attributes are found before class attributes, just as local names have priority over global in an environment. Methods defined within the class are combined with the object of the dot expression to form a bound method during the fourth step of this evaluation procedure.

Attribute assignment. All assignment statements that contain a dot expression on their left-hand side affect attributes for the object of that dot expression. If the object is an instance, then assignment sets an instance attribute. If the object is a class, then assignment sets a class attribute. As a consequence of this rule, assignment to an attribute of an object cannot affect the attributes of its class. The examples below illustrate this distinction.

If we assign to the named attribute `interest` of an account instance, we create a new instance attribute that has the same name as the existing class attribute.

```
>>> kirk_account.interest = 0.08
```

and that attribute value will be returned from a dot expression.

```
>>> kirk_account.interest
0.08
```

However, the class attribute `interest` still retains its original value, which is returned for all other accounts.

```
>>> spock_account.interest
0.04
```

Changes to the class attribute `interest` will affect `spock_account`, but the instance attribute for `kirk_account` will be unaffected.

```
>>> Account.interest = 0.05 # changing the class attribute
>>> spock_account.interest # changes instances without like-named instance attributes
0.05
>>> kirk_account.interest # but the existing instance attribute is unaffected
0.08
```

Inheritance

When working in the object-oriented programming paradigm, we often find that different types are related and should share behavior. In particular, similar classes may differ in their amount of specialization. Two classes may have similar attributes, but one represents a special case of the other.

For example, here is a checking account that is a bit different from a standard account. A checking account charges an extra \$1 for each withdrawal and has a lower interest rate. Here, we demonstrate the desired behavior.

```
>>> ch = CheckingAccount('Spock')
>>> ch.interest      # Lower interest rate for checking accounts
0.01
>>> ch.deposit(20)   # Deposits are the same
20
>>> ch.withdraw(5)   # withdrawals decrease balance by an extra charge
14
```

A `CheckingAccount` is a specialization of an `Account`. In OOP terminology, the generic account will serve as the base class of `CheckingAccount`, while `CheckingAccount` will be a subclass of `Account`. (The terms *parent class* and *superclass* are also used for the base class, while *child class* is also used for the subclass.)

A subclass *inherits* the class attributes of its base class, but may *override* certain attributes, including certain methods. With inheritance, we only specify what is different between the subclass and the base class. Anything that we leave unspecified in the subclass is automatically assumed to behave just as it would for the base class.

Inheritance also has a role in our object metaphor, in addition to being a useful organizational feature. Inheritance is meant to represent *is-a* relationships between classes, which contrast with *has-a* relationships. A checking account *is-a* specific type of account, so having a `CheckingAccount` inherit from `Account` is an appropriate use of inheritance. On the other hand, a bank *has-a* list of bank accounts that it manages, so neither should inherit from the other. Instead, a list of account objects would be naturally expressed as an instance attribute of a bank object.

Using Inheritance

First, we give a full implementation of the `Account` class, which includes docstrings for the class and its methods.

```
>>> class Account:
    """A bank account that has a non-negative balance."""
    interest = 0.02
    def __init__(self, account_holder):
        self.balance = 0
        self.holder = account_holder
    def deposit(self, amount):
        """Increase the account balance by amount and return the new balance."""
        self.balance = self.balance + amount
        return self.balance
    def withdraw(self, amount):
        """Decrease the account balance by amount and return the new balance."""
        if amount > self.balance:
            return 'Insufficient funds'
        self.balance = self.balance - amount
        return self.balance
```

A full implementation of `CheckingAccount` appears below. We specify inheritance by placing an expression that evaluates to the base class in parentheses after the class name.

```
>>> class CheckingAccount(Account):
    """A bank account that charges for withdrawals."""
    withdraw_charge = 1
    interest = 0.01
    def withdraw(self, amount):
        return super().withdraw(amount + self.withdraw_charge)
```

Here, we introduce a class attribute `withdraw_charge` that is specific to the `CheckingAccount` class. We assign a lower value to the `interest` attribute. We also define a new `withdraw` method to override the behavior defined in the `Account` class. With no further statements in the class suite, all other behavior is inherited from the base class `Account`.

```
>>> checking = CheckingAccount('Sam')
>>> checking.deposit(10)
10
>>> checking.withdraw(5)
4
>>> checking.interest
0.01
```

The expression `checking.deposit` evaluates to a bound method for making deposits, which was defined in the `Account` class. When Python resolves a name in a dot expression that is not an attribute of the instance, it looks up the name in the class. In fact, the act of “looking up” a name in a class tries to find that name in every base class in the inheritance chain for the original object’s class. We can define this procedure recursively. To look up a name in a class.

1. If it names an attribute in the class, return the attribute value.
2. Otherwise, look up the name in the base class, if there is one.

In the case of `deposit`, Python would have looked for the name first on the instance, and then in the `CheckingAccount` class. Finally, it would look in the `Account` class, where `deposit` is defined. According to our evaluation rule for dot expressions, since `deposit` is a function looked up in the class for the `checking` instance, the dot expression evaluates to a bound method value. That method is invoked with the argument 10, which calls the deposit method with `self` bound to the `checking` object and `amount` bound to 10.

The class of an object stays constant throughout. Even though the `deposit` method was found in the `Account` class, `deposit` is called with `self` bound to an instance of `CheckingAccount`, not of `Account`.

Calling ancestors. Attributes that have been overridden are still accessible via `super()`, which returns a variant of `self` that finds class attributes in ancestor classes, skipping the class in which the method is defined. For instance, we implemented the `withdraw` method of `CheckingAccount` by calling `super().withdraw`, which evaluates to a bound method for the `Account.withdraw` function. Since `CheckingAccount` is a subclass of `Account`, calling `super().withdraw(x)` within a `CheckingAccount` method is equivalent to calling `Account.withdraw(self, x)`. The `super()` version is now the preferred syntax in Python, since it doesn't repeat the base class name and often won't require updating when class names or inheritance patterns change.

Extensibility. We used `self.withdraw_charge` rather than the equivalent `CheckingAccount.withdraw_charge`. The benefit of the former over the latter is that a class that inherits from `CheckingAccount` might override the withdrawal charge. If that is the case, we would like our implementation of `withdraw` to find that new value instead of the old one. In general, it is best to look up class attributes via `self` so that a subclass can later override the class attribute value without having to override the methods that use it as well.

Interfaces. It is extremely common in object-oriented programs that different types of objects will share the same attribute names. An *object interface* is a collection of attributes and conditions on those attributes. For example, all accounts must have `deposit` and `withdraw` methods that take numerical arguments, as well as a `balance` attribute. The classes `Account` and `CheckingAccount` both implement this interface. Inheritance specifically promotes name sharing in this way. In some programming languages such as Java, interface implementations must be explicitly declared. In others such as Python, Ruby, and Go, any object with the appropriate names implements an interface.

The parts of your program that use objects (rather than implementing them) are most robust to future changes if they do not make assumptions about object types, but instead only about their attribute names. That is, they use the object abstraction, rather than assuming anything about its implementation.

For example, let us say that we run a lottery, and we wish to deposit \$5 into each of a list of accounts. The following implementation does not assume anything about the types of those accounts, and therefore works equally well with any type of object that has a `deposit` method:

```
>>> def deposit_all(winners, amount=5):
    for account in winners:
        account.deposit(amount)
```

The function `deposit_all` above assumes only that each `account` satisfies the account object abstraction, and so it will work with any other account classes that also implement this interface. Assuming a particular class of account would violate the abstraction barrier of the account object abstraction. For example, the following implementation will not necessarily work with new kinds of accounts:

```
>>> def deposit_all(winners, amount=5):
    for account in winners:
        Account.deposit(account, amount)
```

Multiple Inheritance

Python supports the concept of a subclass inheriting attributes from multiple base classes, a language feature called *multiple inheritance*.

Suppose that we have a `SavingsAccount` that inherits from `Account`, but charges customers a small fee every time they make a deposit.

```
>>> class SavingsAccount(Account):
    deposit_charge = 2
    def deposit(self, amount):
        return super().deposit(amount - self.deposit_charge)
```

Then, a clever executive conceives of an `AsSeenOnTVAccount` account with the best features of both `CheckingAccount` and `SavingsAccount`: withdrawal fees, deposit fees, and a low interest rate. It's both a checking and a savings account in one! "If we build it," the executive reasons, "someone will sign up and pay all those fees. We'll even give them a dollar."

```
>>> class AsSeenOnTVAccount(CheckingAccount, SavingsAccount):
    def __init__(self, account_holder):
        self.holder = account_holder
        self.balance = 1          # A free dollar!
```

In fact, this implementation is complete. Both withdrawal and deposits will generate fees, using the function definitions in `CheckingAccount` and `SavingsAccount` respectively.

```

>>> such_a_deal = AsSeenOnTVAccount("John")
>>> such_a_deal.balance
1
>>> such_a_deal.deposit(20)          # $2 fee from SavingsAccount.deposit
19
>>> such_a_deal.withdraw(5)         # $1 fee from CheckingAccount.withdraw
13

```

Non-ambiguous references are resolved correctly as expected:

```

>>> such_a_deal.deposit_charge
2
>>> such_a_deal.withdraw_charge
1

```

But what about when the reference is ambiguous, such as the reference to the `withdraw` method that is defined in both `Account` and `CheckingAccount`? The figure below depicts an *inheritance graph* for the `AsSeenOnTVAccount` class. Each arrow points from a subclass to a base class.



For a simple “diamond” shape like this, Python resolves names from left to right, then upwards. In this example, Python checks for an attribute name in the following classes, in order, until an attribute with that name is found:

```
AsSeenOnTVAccount, CheckingAccount, SavingsAccount, Account, object
```

There is no correct solution to the inheritance ordering problem, as there are cases in which we might prefer to give precedence to certain inherited classes over others. However, any programming language that supports multiple inheritance must select some ordering in a consistent way, so that users of the language can predict the behavior of their programs.

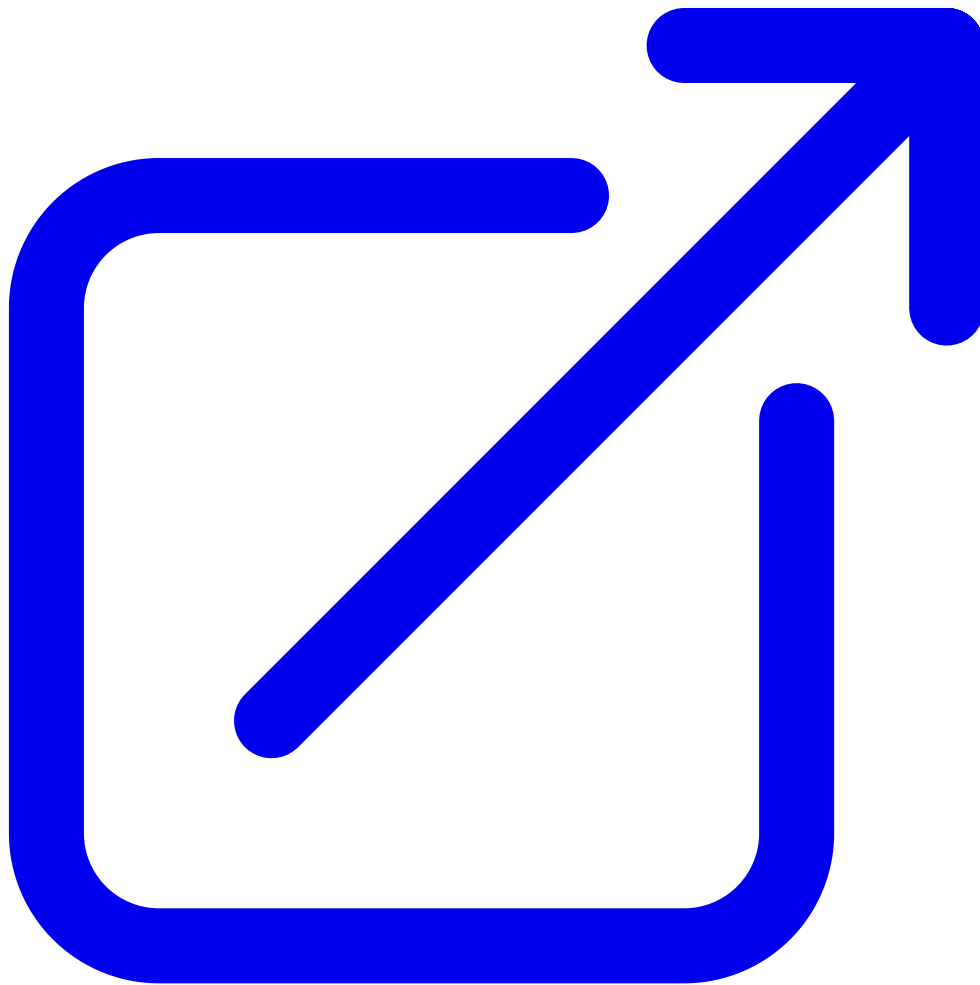
Further reading. Python resolves this name using a recursive algorithm called the C3 Method Resolution Ordering. The method resolution order of any class can be queried using the `mro` method on all classes.

```

>>> [c.__name__ for c in AsSeenOnTVAccount.mro()]
['AsSeenOnTVAccount', 'CheckingAccount', 'SavingsAccount', 'Account', 'object']

```

The precise algorithm for finding method resolution orderings is not a topic for this text, but is [described by Python's primary author](#)



with a reference to the original paper.

The Role of Objects¹

The Python object system is designed to make data abstraction and message passing both convenient and flexible. The specialized syntax of classes, methods, inheritance, and dot expressions all enable us to adopt the object metaphor in our programs, which improves our ability to organize large programs.

In particular, we would like our object system to promote a *separation of concerns* among the different aspects of the program. Each object in a program encapsulates and manages some part of the program's state, and each class statement defines the functions that implement some part of the program's overall logic.

Object-oriented programming is particularly well-suited to programs that model systems that have separate but interacting parts. For instance, different users interact in a social network, different characters interact in a game, and different shapes interact in a physical simulation. When representing such systems, the objects in a program often map naturally onto objects in the system being modeled, and classes represent their types and relationships.

On the other hand, classes may not provide the best mechanism for implementing certain abstractions. Functional abstractions provide a more natural metaphor for representing relationships between inputs and outputs. One should not feel compelled to fit every bit of logic in a program within a class, especially when defining independent functions for manipulating data is more natural. Functions can also enforce a separation of concerns.

Multi-paradigm languages such as Python allow programmers to match organizational paradigms to appropriate problems. Learning to identify when to introduce a new class, as opposed to a new function, in order to simplify or modularize a program, is an important design skill in

software engineering that deserves careful attention.



[Chapter 2: Data](#)

[2.4 Mutable Data](#)

[Chapter 2: Data](#)

[2.6 Implementing Classes and Objects](#)



2.6 Implementing Classes and Objects



2.6 Implementing Classes and Objects

When working in the object-oriented programming paradigm, we use the object metaphor to guide the organization of our programs. Most logic about how to represent and manipulate data is expressed within class declarations. In this section, we see that classes and objects can themselves be represented using just functions and dictionaries. The purpose of implementing an object system in this way is to illustrate that using the object metaphor does not require a special programming language. Programs can be object-oriented, even in programming languages that do not have a built-in object system.

In order to implement objects, we will abandon dot notation (which does require built-in language support), but create dictionaries that behave in much the same way as built-in classes and instances. Mirroring Python's built-in object system, we will send messages between instances, classes, and base classes. In our implementation, the messages are strings rather than attribute names, and the attributes themselves are stored in dictionaries.

Our implementation is not meant to follow every detail of the Python type system, but instead to capture the core functionality that supports object-oriented programming.

Dispatch Dictionaries

Our implementation will be designed around *dispatch dictionaries*, which are dictionaries with string keys and function values that are used to respond to messages. An example of a dispatch dictionary appears below. This one responds to messages "add" and "subtract" that manipulate a count stored as the item in a one-item list.

```
>>> def counter():
    state = [0]
    def add(k):
        state[0] += k
        return state[0]
    def subtract(k):
        state[0] -= k
        return state[0]
    return {"add": add, "subtract": subtract}

>>> first = counter()
>>> first["add"](3)
3
>>> first["add"](3)
6
>>> first["subtract"](2)
4
>>> second = counter()
>>> second["add"](5)
5
>>> second["add"](5)
10
>>> first["subtract"](2)
2
>>> second["subtract"](3)
7
```

Instances

We begin with instances. An instance has named attributes, such as the balance of an account, which can be set and retrieved. We implement an instance using a dispatch dictionary that responds to messages that "get" and "set" attribute values. Attributes themselves are stored in a local dictionary called `attributes`.

To begin our implementation, we assume that we have a class implementation that can look up any names that are not part of the instance. We pass in a class to `make_instance` as the parameter `cls`.

```
>>> def make_instance(cls):
    """Return a new object instance, which is a dispatch dictionary."""
    def get_value(name):
        if name in attributes:
            return attributes[name]
        else:
            value = cls['get'](name)
            return bind_method(value, instance)
    def set_value(name, value):
        attributes[name] = value
    attributes = {}
    instance = {'get': get_value, 'set': set_value}
    return instance
```

The `instance` is a dispatch dictionary that responds to the messages `get` and `set`. The `set` message corresponds to attribute assignment in Python's object system: all assigned attributes are stored directly within the object's local attribute dictionary. In `get`, if `name` does not appear in the local `attributes` dictionary, then it is looked up in the class. If the `value` returned by `cls` is a function, it must be bound to the instance.

Bound method values. The `get_value` function in `make_instance` finds a named attribute in its class with `get`, then calls `bind_method`. Binding a method only applies to function values, and it creates a bound method value from a function value by inserting the instance as the first argument:

```
>>> def bind_method(value, instance):
    """Return a bound method if value is callable, or value otherwise."""
    if callable(value):
        def method(*args):
            return value(instance, *args)
        return method
    else:
        return value
```

The `def method(*args)` syntax allows `method` to take any number of arguments and binds the name `args` to a tuple containing all of the arguments passed in. Similarly, the `value(instance, *args)` syntax calls `value` on multiple arguments, including `instance` as the first argument and each item within `args` as an additional argument. The built-in `callable` function returns whether its argument can be called.

When a method is called, the first parameter `self` will be bound to the value of `instance` by this definition.

Classes

A class is also an object, both in Python's object system and the system we are implementing here. For simplicity, we say that classes do not themselves have a class. (In Python, classes do have classes; almost all classes share the same class, called `type`.) A class can respond to `get` and `set` messages, as well as the `new` message:

```
>>> def make_class(attributes, base_class=None):
    """Return a new class, which is a dispatch dictionary."""
    def get_value(name):
        if name in attributes:
            return attributes[name]
        elif base_class is not None:
            return base_class['get'](name)
    def set_value(name, value):
        attributes[name] = value
    def new(*args):
        return init_instance(cls, *args)
    cls = {'get': get_value, 'set': set_value, 'new': new}
    return cls
```

Unlike an instance, the `get` function for classes does not query its class when an attribute is not found, but instead queries its `base_class`. No method binding is required for classes.

Initialization. The `new` function in `make_class` calls `init_instance`, which first makes a new instance, then invokes a method called `__init__`.

```
>>> def init_instance(cls, *args):
    """Return a new object with type cls, initialized with args."""
    instance = make_instance(cls)
    init = cls['get']('__init__')
    if init:
        init(instance, *args)
    return instance
```

This final function completes our object system. We now have instances, which `set` locally but fall back to their classes on `get`. After an instance looks up a name in its class, it binds itself to function values to create methods. Finally, classes can create `new` instances, and they apply

their `__init__` constructor function immediately after instance creation.

In this object system, the only function that should be called by the user is `make_class`. All other functionality is enabled through message passing. Similarly, Python's object system is invoked via the `class` statement, and all of its other functionality is enabled through dot expressions and calls to classes.

Using Implemented Objects

We now return to use the bank account example from the previous section. Using our implemented object system, we will create an `Account` class, a `CheckingAccount` subclass, and an instance of each.

The `Account` class is created through a `make_account_class` function, which has structure similar to a `class` statement in Python, but concludes with a call to `make_class`.

```
>>> def make_account_class():
    """Return the Account class, which has deposit and withdraw methods."""
    interest = 0.02
    def __init__(self, account_holder):
        self['set']('holder', account_holder)
        self['set']('balance', 0)
    def deposit(self, amount):
        """Increase the account balance by amount and return the new balance."""
        new_balance = self['get']('balance') + amount
        self['set']('balance', new_balance)
        return self['get']('balance')
    def withdraw(self, amount):
        """Decrease the account balance by amount and return the new balance."""
        balance = self['get']('balance')
        if amount > balance:
            return 'Insufficient funds'
        self['set']('balance', balance - amount)
        return self['get']('balance')
    return make_class(locals())
```

The final call to `locals` returns a dictionary with string keys that contains the name-value bindings in the current local frame.

The `Account` class is finally instantiated via assignment.

```
>>> Account = make_account_class()
```

Then, an account instance is created via the `new` message, which requires a name to go with the newly created account.

```
>>> kirk_account = Account['new']('Kirk')
```

Then, `get` messages passed to `kirk_account` retrieve properties and methods. Methods can be called to update the balance of the account.

```
>>> kirk_account['get']('holder')
'Kirk'
>>> kirk_account['get']('interest')
0.02
>>> kirk_account['get']('deposit')(20)
20
>>> kirk_account['get']('withdraw')(5)
15
```

As with the Python object system, setting an attribute of an instance does not change the corresponding attribute of its class.

```
>>> kirk_account['set']('interest', 0.04)
>>> Account['get']('interest')
0.02
```

Inheritance. We can create a subclass `CheckingAccount` by overloading a subset of the class attributes. In this case, we change the `withdraw` method to impose a fee, and we reduce the interest rate.

```
>>> def make_checking_account_class():
    """Return the CheckingAccount class, which imposes a $1 withdrawal fee."""
    interest = 0.01
    withdraw_fee = 1
    def withdraw(self, amount):
        fee = self['get']('withdraw_fee')
        return Account['get']('withdraw')(self, amount + fee)
    return make_class(locals(), Account)
```

In this implementation, we call the `withdraw` function of the base class `Account` from the `withdraw` function of the subclass, as we would in Python's built-in object system. We can create the subclass itself and an instance, as before.

```
>>> CheckingAccount = make_checking_account_class()
>>> spock_acct = CheckingAccount['new']('Spock')
```

Deposits behave identically, as does the constructor function. withdrawals impose the \$1 fee from the specialized `withdraw` method, and `interest` has the new lower value from `CheckingAccount`.

```
>>> spock_acct['get']('interest')
0.01
>>> spock_acct['get']('deposit')(20)
20
>>> spock_acct['get']('withdraw')(5)
14
```

Our object system built upon dictionaries is quite similar in implementation to the built-in object system in Python. In Python, an instance of any user-defined class has a special attribute `__dict__` that stores the local instance attributes for that object in a dictionary, much like our `attributes` dictionary. Python differs because it distinguishes certain special methods that interact with built-in functions to ensure that those functions behave correctly for arguments of many different types. Functions that operate on different types are the subject of the next section.



[Chapter 2: Data](#)

[2.5 Object-Oriented Programming](#)

[Chapter 2: Data](#)

[2.7 Object Abstraction](#)



2.7 Object Abstraction



2.7 Object Abstraction

The object system allows programmers to build and use abstract data representations. A final step in designing the behavior of user-defined classes is to specify how their instances can use built-in Python syntax, such as printing, displaying, adding, calling, and indexing into objects.

A central concept in object abstraction is a *generic function*, which is a function that can accept values of multiple different types. Python's most common way of implementing generic functions is through shared interfaces.

String Conversion¶

To represent data effectively, an object value should behave like the kind of data it is meant to represent, including producing a string representation of itself. String representations of data values are especially important in an interactive language such as Python that automatically displays the string representation of the values of expressions in an interactive session.

String values provide a fundamental medium for communicating information among humans. Sequences of characters can be rendered on a screen, printed to paper, read aloud, converted to braille, or broadcast as Morse code. Strings are also fundamental to programming because they can represent Python expressions.

Python stipulates that all objects should produce two different string representations: one that is human-interpretable text and one that is a Python-interpretable expression. The constructor function for strings, `str`, returns a human-readable string. Where possible, the `repr` function returns a Python expression that evaluates to an equal object. The docstring for `repr` explains this property:

```
repr(object) -> string

Return the canonical string representation of the object.
For most object types, eval(repr(object)) == object.
```

The result of calling `repr` on the value of an expression is what Python prints in an interactive session.

```
>>> 12e12
1200000000000.0
>>> print(repr(12e12))
1200000000000.0
```

In cases where no representation exists that evaluates to the original value, Python typically produces a description surrounded by angled brackets.

```
>>> repr(min)
'<built-in function min>'
```

The `str` constructor often coincides with `repr`, but provides a more interpretable text representation in some cases. For instance, we see a difference between `str` and `repr` with dates.

```
>>> from datetime import date
>>> tues = date(2011, 9, 12)
>>> repr(tues)
'datetime.date(2011, 9, 12)'
>>> str(tues)
'2011-09-12'
```

Defining the `repr` function presents a new challenge: we would like it to apply correctly to all data types, even those that did not exist when `repr` was implemented. We would like it to be a generic or *polymorphic function*, one that can be applied to many (*poly*) different forms (*morph*) of data.

The object system provides an elegant solution in this case: the `repr` function always invokes a method called `__repr__` on its argument.

```
>>> tues.__repr__()
'datetime.date(2011, 9, 12)'
```

By implementing this same method in user-defined classes, we can extend the applicability of `repr` to any class we create in the future. This example highlights another benefit of dot expressions in general, that they provide a mechanism for extending the domain of existing functions to new object types.

The `str` constructor is implemented in a similar manner: it invokes a method called `__str__` on its argument.

```
>>> tues.__str__()
'2011-09-12'
```

These polymorphic functions are examples of a more general principle: certain functions should apply to multiple data types. Moreover, one way to create such a function is to use a shared method name with a different definition in each class.

Special Methods¹

In Python, certain *special names* are invoked by the Python interpreter in special circumstances. For instance, the `__init__` method of a class is automatically invoked whenever an object is constructed. The `__str__` method is invoked automatically when printing, and `__repr__` is invoked in an interactive session to display values.

There are special names for many other behaviors in Python. Some of those used most commonly are described below.

True and false values. We saw previously that numbers in Python have a truth value; more specifically, 0 is a false value and all other numbers are true values. In fact, all objects in Python have a truth value. By default, objects of user-defined classes are considered to be true, but the special `__bool__` method can be used to override this behavior. If an object defines the `__bool__` method, then Python calls that method to determine its truth value.

As an example, suppose we want a bank account with 0 balance to be false. We can add a `__bool__` method to the `Account` class to create this behavior.

```
>>> Account.__bool__ = lambda self: self.balance != 0
```

We can call the `bool` constructor to see the truth value of an object, and we can use any object in a boolean context.

```
>>> bool(Account('Spock'))
False
>>> if not Account('Spock'):
    print('Spock is broke')
Spock is broke
```

Sequence operations. We have seen that we can call the `len` function to determine the length of a sequence.

```
>>> len('Go Bears!')
9
```

The `len` function invokes the `__len__` method of its argument to determine its length. All built-in sequence types implement this method.

```
>>> 'Go Bears!'.__len__()
9
```

Python uses a sequence's length to determine its truth value, if it does not provide a `__bool__` method. Empty sequences are false, while non-empty sequences are true.

```
>>> bool('')
False
>>> bool([])
False
>>> bool('Go Bears!')
True
```

The `__getitem__` method is invoked by the item selection operator, but it can also be invoked directly.

```
>>> 'Go Bears!'[3]
'B'
>>> 'Go Bears!'.__getitem__(3)
'B'
```

Callable objects. In Python, functions are first-class objects, so they can be passed around as data and have attributes like any other object. Python also allows us to define objects that can be “called” like functions by including a `__call__` method. With this method, we can define a class that behaves like a higher-order function.

As an example, consider the following higher-order function, which returns a function that adds a constant value to its argument.

```
>>> def make_adder(n):
    def adder(k):
        return n + k
    return adder

>>> add_three = make_adder(3)
>>> add_three(4)
7
```

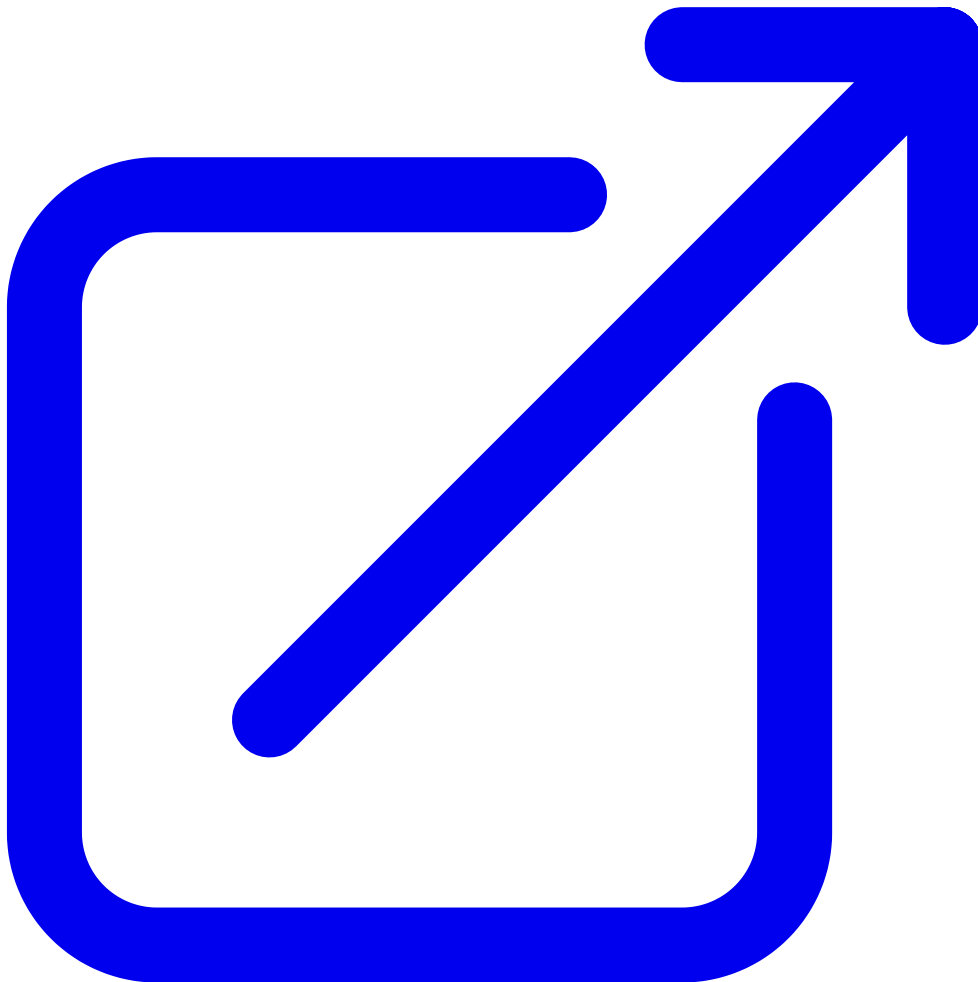
We can create an `Adder` class that defines a `__call__` method to provide the same functionality.

```
>>> class Adder:
    def __init__(self, n):
        self.n = n
    def __call__(self, k):
        return self.n + k

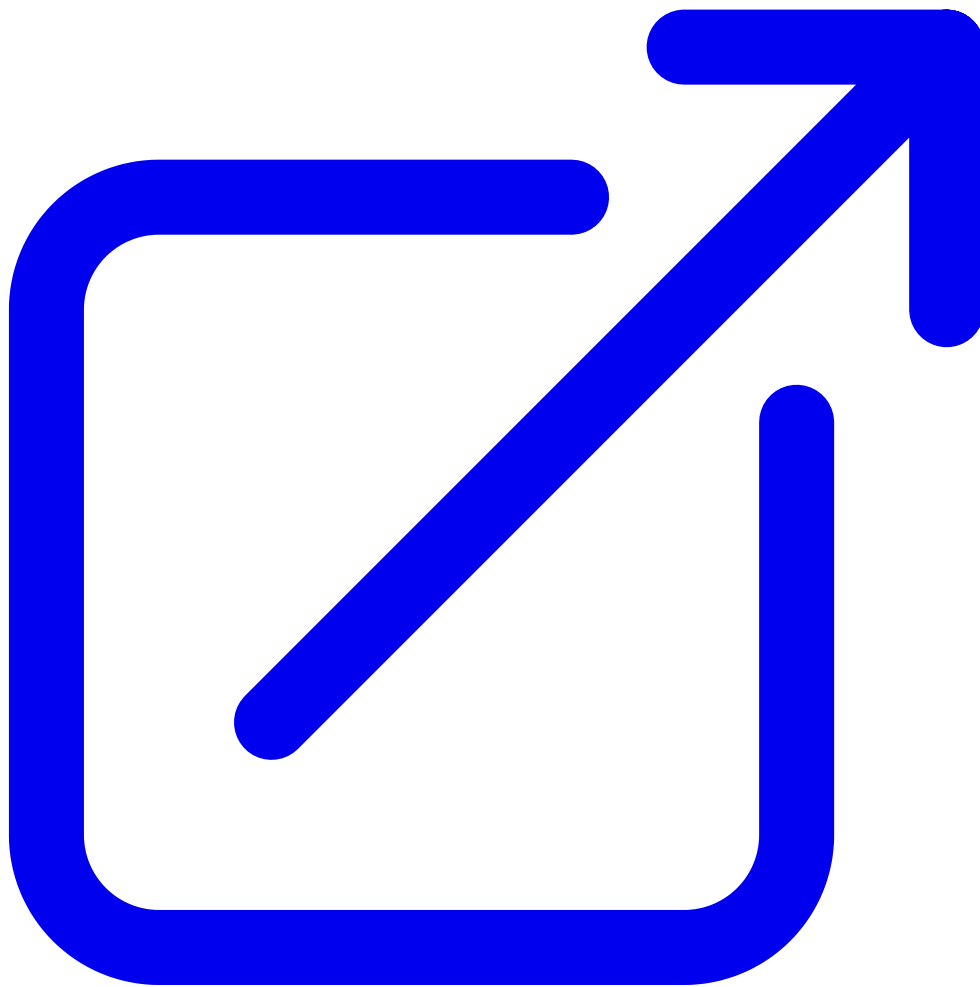
>>> add_three_obj = Adder(3)
>>> add_three_obj(4)
7
```

Here, the `Adder` class behaves like the `make_adder` higher-order function, and the `add_three_obj` object behaves like the `add_three` function. We have further blurred the line between data and functions.

Arithmetic. Special methods can also define the behavior of built-in operators such as `+` and `*` when applied to user-defined objects. In order to provide this generality, Python follows specific protocols to apply each operator. For example, to evaluate expressions that contain the `+` operator, Python checks for special methods on both the left and right operands of the expression. First, Python checks for an `__add__` method on the value of the left operand, then checks for an `__radd__` method on the value of the right operand. If either is found, that method is invoked with the value of the other operand as its argument. An example follows below. For readers interested in further details, the Python documentation describes the exhaustive set of [method names for operators](#).



. Dive into Python 3 has a chapter on [special method names](#)



that describes how many of these special method names are used.

Example: Ratios

A ratio of integers, also called a rational number, is an important kind of number. A rational number such as $1/3$ or $17/29$ is typically written as:

```
<numerator>/<denominator>
```

where both the `<numerator>` and `<denominator>` are placeholders for integer values. Both parts are needed to exactly characterize the value of the ratio. Actually dividing integers produces a `float` approximation, losing the exact precision of integers.

```
>>> 1/3
0.3333333333333333
>>> 1/3 == 0.33333333333333330123456789 # Dividing integers yields an approximation
True
```

However, we can create an exact representation for ratios using a class. The class defined below stores the ratio in lowest terms, dividing both the passed in numerator and denominator by their greatest common divisor. The special methods provide appropriate behavior when displaying, adding, and multiplying ratios.

```

>>> from __future__ import annotations
>>> import math
>>> class Ratio:
>>>     "A ratio of integers: number / denom"
>>>     def __init__(self, n: int, d: int):
>>>         g = math.gcd(n, d)
>>>         self.number = n // g
>>>         self.denom = d // g
>>>     def __str__(self):
>>>         return f'{self.number}/{self.denom}'
>>>     def __repr__(self):
>>>         return f'Ratio({self.number}, {self.denom})'
>>>     def __add__(self, other: Ratio):
>>>         return Ratio(self.number * other.denom + other.number * self.denom, self.denom * other.denom)
>>>     def __mul__(self, other: Ratio):
>>>         return Ratio(self.number * other.number, self.denom * other.denom)

>>> half = Ratio(2, 4)
>>> half
Ratio(1, 2)
>>> print(half)
1/2
>>> fifth = Ratio(1, 5)
>>> print(half * fifth)
1/10
>>> print(fifth + fifth)
2/5

```

As the example above shows, our rational number implementation reduces ratios to lowest terms each time one is constructed, so `fifth + fifth` is two fifths, rather than ten twenty-fifths.



[Chapter 2: Data](#)

[2.6 Implementing Classes and Objects](#)

[Chapter 2: Data](#)

[Appendix: Examples](#)



Appendix: Examples



Appendix: Examples

Companion code for the text. Each Python file can be downloaded and run on its own; its docstrings include doctests (`python3 -m doctest -v link.py`).

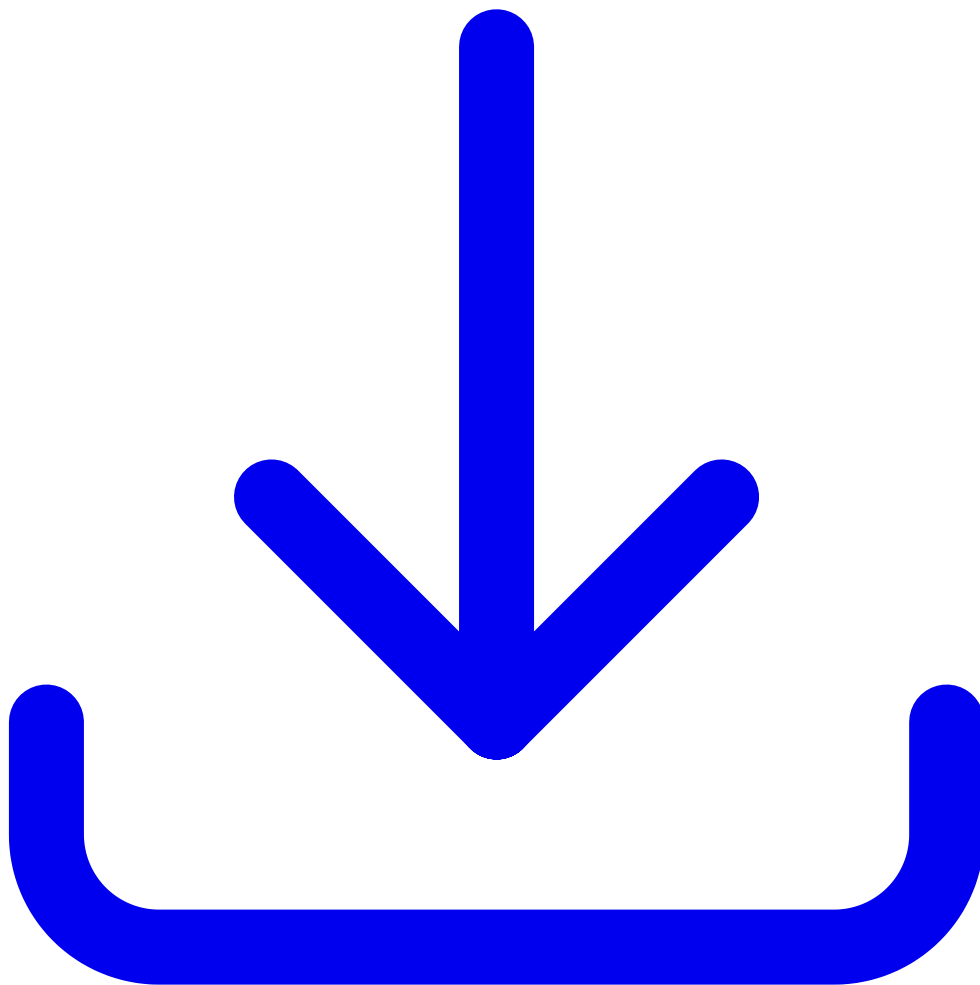
Hog Two Ways[¶]

[Hog Two Ways](#) is a Gleam project that replaces the mutable dice functions of a Python Hog game with explicit state handling and with state passing. Download the whole project as [dice.zip](#).

link.py[¶]

The `Link` data class for linked lists from [Section 2.3](#), with a `__str__` method that formats a linked list as its items within parentheses.

Source: [link.py](#)



To see a linked list as a box-and-pointer diagram, paste its repr or str into the [drawing tool](#).



link.py

```
from __future__ import annotations
from dataclasses import dataclass

type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    """A Link has a first value of type T and the rest of the linked list.

    The rest is either another Link or the empty tuple.

    >>> s = Link(3, Link(4, Link(5)))
    >>> s.first
    3
    >>> s.rest.first
    4
    >>> s.rest.rest.first
    5
    >>> s.rest.rest.rest == ()
    True
    >>> s
    Link(first=3, rest=Link(first=4, rest=Link(first=5, rest=())))
    >>> s.rest.rest
    Link(first=5, rest=())
    >>> print(s)
    (3 4 5)
    """
    first: T
    rest: LinkedList[T] = () # rest defaults to an empty linked list

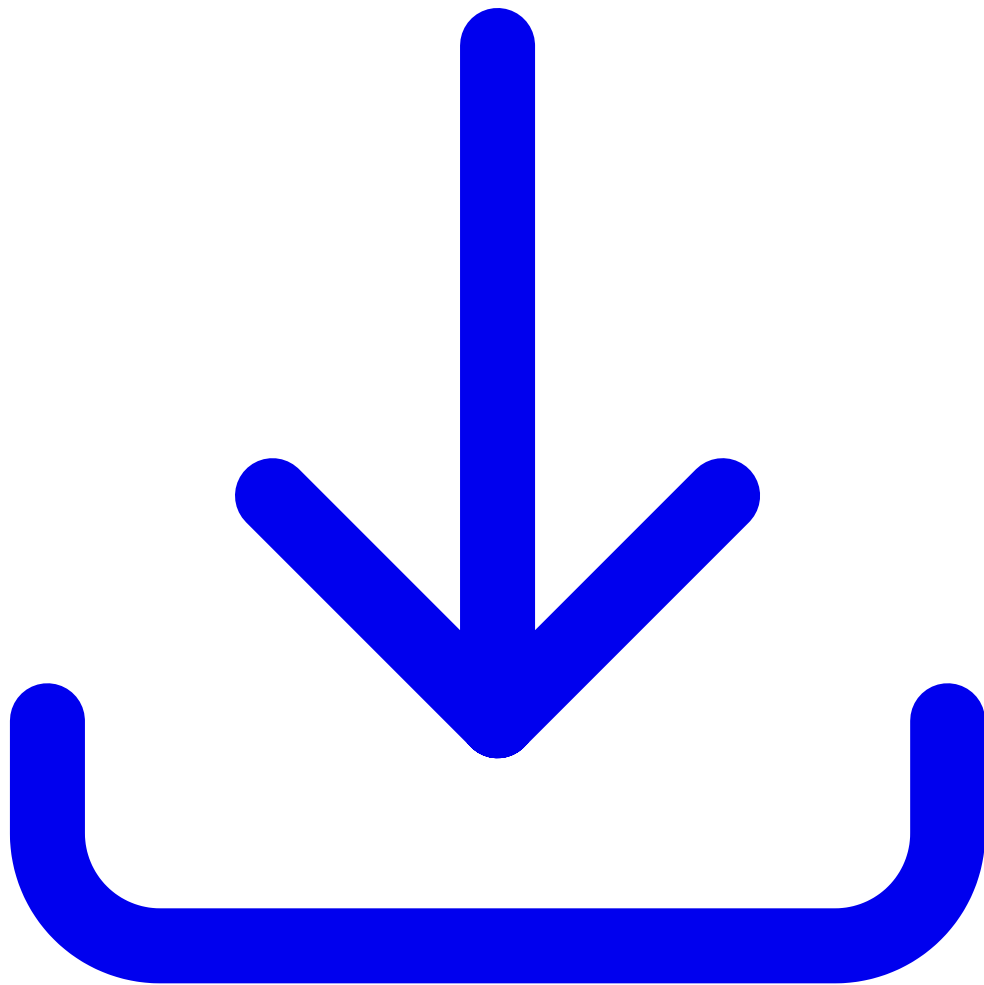
    def __str__(self):
        return format_link(self)

def format_link(s: Link):
    """Return a Link s formatted as items within parentheses."""
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'
```

tree.py

The `Tree` data class from [Section 2.3](#) with a `__str__` method that prints each branch indented below its label.

Source: [tree.py](#).



The [drawing tool](#) draws a tree from its repr or str.

📄
tree.py

```

from __future__ import annotations
from dataclasses import dataclass, field

@dataclass
class Tree[T]:
    """A Tree has a label (of type T) and a list of branches, which are trees.

    >>> t = Tree(3, [Tree(1, []), Tree(2, [Tree(1, []), Tree(1, [])])]
    >>> t.branches[0]
    Tree(label=1, branches=[])
    >>> len(t.branches)
    2
    >>> t.branches[1].label
    2
    >>> len(t.branches[1].branches)
    2
    >>> expr = Tree(10, [Tree(6, [Tree(2), Tree('*'), Tree(3)]), Tree('+'), Tree(4)])
    >>> print(expr)
    10
    6
    2
    *
    3
    +
    4
    """
    label: T
    branches: list[Tree[T]] = field(default_factory=list) # branches defaults to []

    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    "Format a tree with each branch indented below its label."
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + ' ')
    return string

def is_leaf(t: Tree) -> bool:
    "Return whether a Tree t is a leaf with no branches."
    return not t.branches

```



[Chapter 2: Data](#)

[2.7 Object Abstraction](#)